

PLAYLIST NOTES

Backend from first principles

These comprehensive study notes were automatically generated by NoteForge.

Table of Contents

01 Lecture 1: 1. Roadmap for backend from first principles

Chapter 1

02 Lecture 2: 2. Walk the path of a true backend engineer

Chapter 2

03 Lecture 3: 3. What is a Backend, how do they work and why do we need them?

Chapter 3

04 Lecture 4: 4. Benefits of learning backend engineering from first principles

Chapter 4

05 Lecture 5: 5. Understanding HTTP for backend engineers, where it all starts

Chapter 5

06 Lecture 6: 6. What is Routing in Backend? How Requests Find Their Way Home

Chapter 6

07 Lecture 7: 7. Serialization and Deserialization for backend engineers

Chapter 7

08 Lecture 8: 8. Authentication and authorization for backend engineers

Chapter 8

09 Lecture 9: 9. Validations and transformations for backend engineers

Chapter 9

10 Lecture 10: 10. What are controllers, services, repositories, middlewares and request context?

Chapter 10

11 Lecture 11: 11. Complete REST API Design

Chapter 11

12 Lecture 12: 12. Mastering Databases with Postgres

Chapter 12

13 Lecture 13: 13. Caching, the secret behind it all

Chapter 13

14 Lecture 14: 14. Task queues and background jobs

Chapter 14

15 Lecture 15: 15. Full text search using Elasticsearch for blazingly fast search

Chapter 15

16 Lecture 16: 16. Error Handling and Building Fault Tolerant Systems

Chapter 16

17 Lecture 17: 17. Production-grade Configuration Management

Chapter 17

18 Lecture 18: 18. Logging, Monitoring and Observability

Chapter 18

19 Lecture 19: 19. Graceful Shutdown

Chapter 19

20 Lecture 20: 20. Backend Security: Everything You Need to Know

Chapter 20

21 Lecture 21: 21.1. Backend Scaling and Performance Engineering: Part-1

Chapter 21

22 Lecture 22: 21.2. Backend Scaling and Performance Engineering: Part-2

Chapter 22

23 Lecture 23: 22. Concurrency & Parallelism: IO Bound vs CPU Bound

Chapter 23

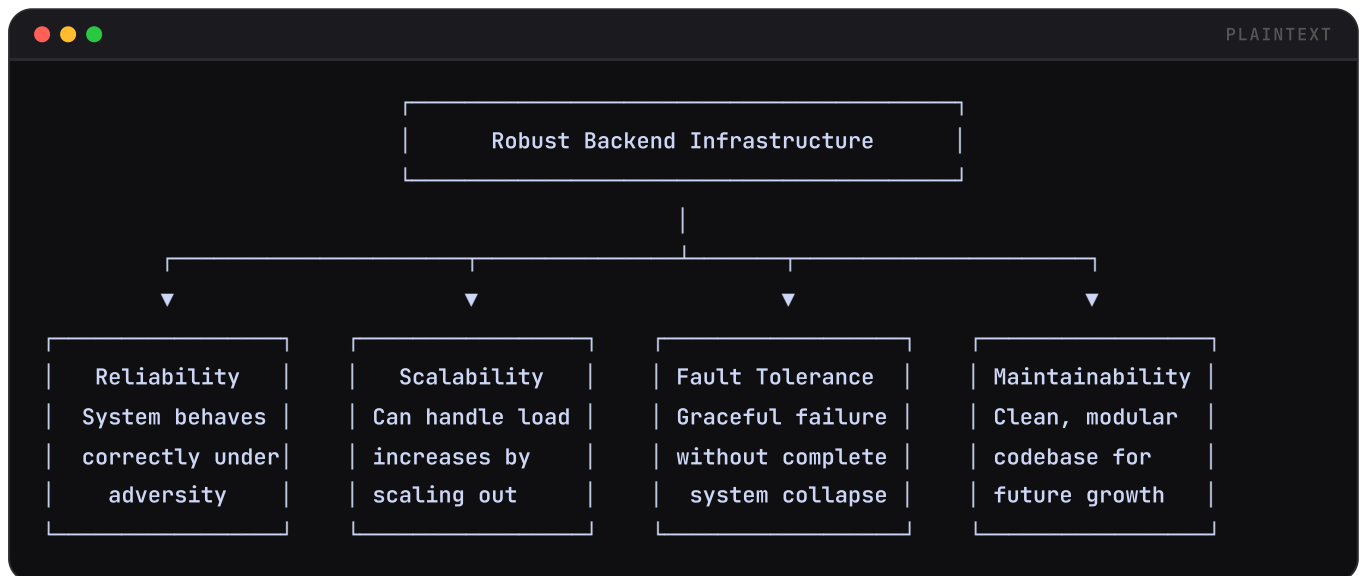
CH
01

Lecture 1: 1. Roadmap for backend from first principles

LECTURE NOTES: ROADMAP FOR BACKEND ENGINEERING FROM FIRST PRINCIPLES

Introduction: The Philosophy of Backend Engineering

Backend engineering is far more than building a collection of CRUD (Create, Read, Update, Delete) APIs. It is the discipline of designing, implementing, and maintaining **reliable, scalable, fault-tolerant, maintainable, and highly efficient systems**.



The Framework Trap vs. First Principles

Many developers enter backend engineering through the lens of a specific language or framework (e.g., Express.js for Node.js, Spring Boot for Java, or Ruby on Rails). This approach limits perspective:

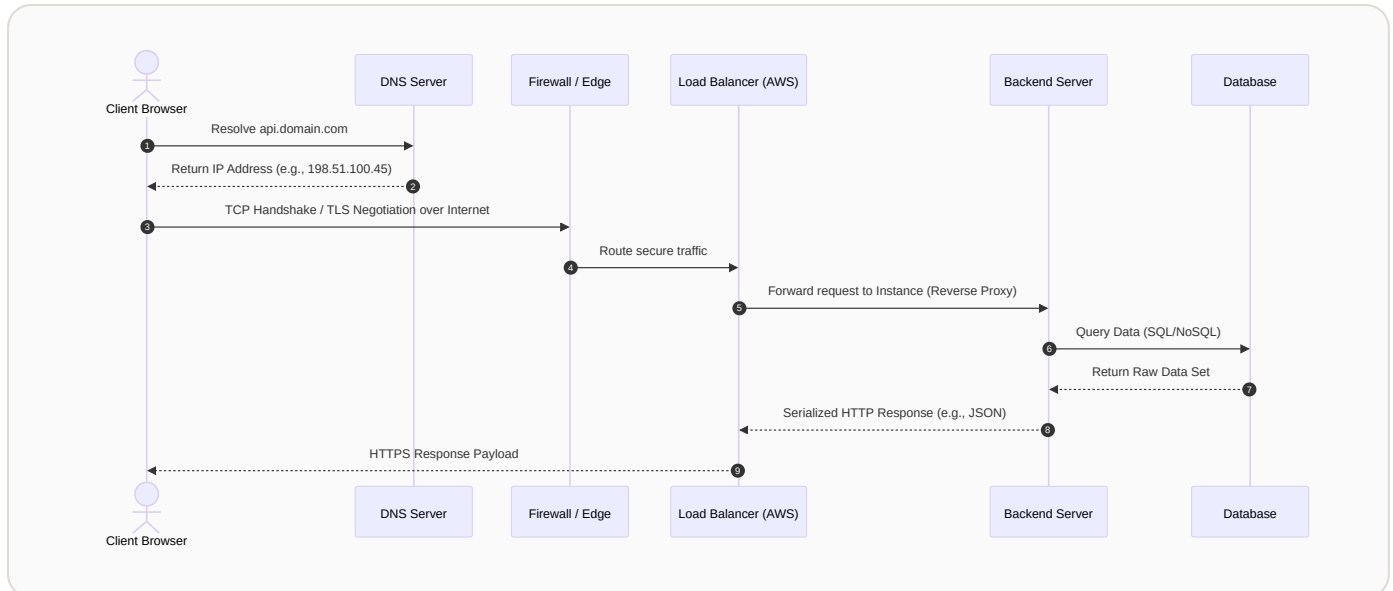
This roadmap covers the foundational concepts necessary to build backend systems from first principles.

Module 1: Network Fundamentals & The Request Lifecycle

Understanding how data traverses the physical and logical boundaries of the internet is the starting point for backend engineering.

1.1 The End-to-End Request Lifecycle

When a user inputs a URL in a browser and hits Enter, a complex series of hops occurs:



1.2 The HTTP Protocol Deep Dive

HTTP (Hypertext Transfer Protocol) is the application-layer protocol that powers web communication.

RAW HTTP MESSAGE STRUCTURE

An HTTP request is a plain-text message formatted according to strict RFC standards:

```
HTTP
POST /v1/users HTTP/1.1
Host: api.example.com
User-Agent: Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_7)
Content-Type: application/json
Content-Length: 47
Authorization: Bearer s3cr3t_t0k3n

{
  "username": "alice",
  "email": "alice@example.com"
}
```

HTTP HEADERS CLASSIFICATION

Headers provide crucial metadata about the transaction. They are categorized as:

HTTP METHODS SEMANTICS

1.3 CORS (Cross-Origin Resource Sharing)

CORS is a browser-enforced security mechanism that prevents web applications from making requests to a domain different from the one that served the page.

PLAINTEXT

```
Client Browser                                Server (api.example.com)
|
| — 1. OPTIONS (Pre-flight) —————> |
| <— 2. Allow CORS Headers ————— | (Access-Control-Allow-Methods, etc.)
|
| — 3. Actual Request (e.g., PUT) -> |
| <— 4. Actual Response ————— |
```

1.4 HTTP Caching & Performance

1.5 Evolution of HTTP Versions

PLAINTEXT

```
HTTP/1.1: [Request 1] —> [Response 1] —> [Request 2] —> [Response 2] (Sequential)

HTTP/2.0: ┌ Stream A — [Frame A1] — [Frame A2] ┐
          │ Stream B — [Frame B1] —————┐ —> Multiplexed over 1 TCP Conn
          └ Stream C — [Frame C1] — [Frame C2] ┘

HTTP/3.0: Same as HTTP/2 but uses QUIC (UDP). Packet loss on Stream A does not stall Stream B/C.
```

1.6 Network Optimizations & Security

Module 2: Routing Mechanics & API Design

Routing is the process of mapping incoming HTTP requests (defined by an HTTP method and a URL path) to specific execution handlers.

PLAINTEXT

```
Incoming Request: GET /v1/users/42/posts?limit=10
|
| ▼
| ┌────────────────────────────────────────────────────────────────────────────────┐
| │ Router Engine                                                                 │
| │                                                                              │
| │ 1. Match Path: "/v1/users/:id/posts"                                       │
| │ 2. Extract Param: id = "42"                                                 │
| │ 3. Extract Query: limit = "10"                                             │
| │ 4. Delegate to: UserController.getUserPosts(id, limit)                   │
| │                                                                              │
| └────────────────────────────────────────────────────────────────────────────────┘
```

2.1 Route Parameters vs. Query Parameters

2.2 Route Types

2.3 API Versioning Strategies

To prevent breaking changes for clients, APIs must be versioned. The primary strategies include:

VERSIONING STRATEGY	EXAMPLE	PROS	CONS
URI Versioning	<code>/v1/users</code>	Highly visible, easy to cache.	Pollutes URL namespace.
Header Versioning	<code>Accept-Version: 1.0.0</code>	Keeps URLs clean.	Harder to test in standard browsers.
Query String	<code>/users?version=1</code>	Simple fallback option.	Can complicate query parsing.
Media Type (Accept Header)	<code>Accept: application/vnd.company.v1+json</code>	Strict REST adherence.	High complexity for client/server.

2.4 Route Grouping & Deprecation

Module 3: Serialization, Deserialization, & Data Formats

Data transmitted over a network must be serialized (converted from in-memory objects to a transfer format) and deserialized (reconstructed into in-memory objects upon receipt).



3.1 Serialization Formats: Text vs. Binary

PROTOBUF

```
// Protobuf Schema Definition
message User {
  int32 id = 1;
  string username = 2;
  string email = 3;
}
```

3.2 Parsing Edge Cases & Common Pitfalls

When deserializing JSON payloads into native data structures (e.g., Go structs, Python dictionaries, or JavaScript objects), several edge cases arise:

3.3 Security & Schema Validation

```
{
  "$schema": "http://json-schema.org/draft-07/schema#",
  "type": "object",
  "properties": {
    "username": { "type": "string", "minLength": 3 },
    "email": { "type": "string", "format": "email" }
  },
  "required": ["username", "email"]
}
```

Module 4: Authentication & Authorization

Authentication (AuthN) verifies **who** a user is; Authorization (AuthZ) determines **what** that user is allowed to do.

```
Authentication: "Who are you?" → Verified Credentials (JWT/Session)
Authorization:  "What can you do?" → Role/Permission check (RBAC/ABAC)
```

4.1 Stateful vs. Stateless Authentication

4.2 Modern Auth Protocols

4.3 Cryptographic Storage Fundamentals

Passwords must never be stored in plain text.

4.4 Access Control Models

4.5 Security Vulnerabilities & Mitigations

Module 5: Input Validation & Data Transformation

All input entering a backend system must be treated as untrusted and validated.



5.1 Validation Types

Lecture 2: 2. Walk the path of a true backend engineer

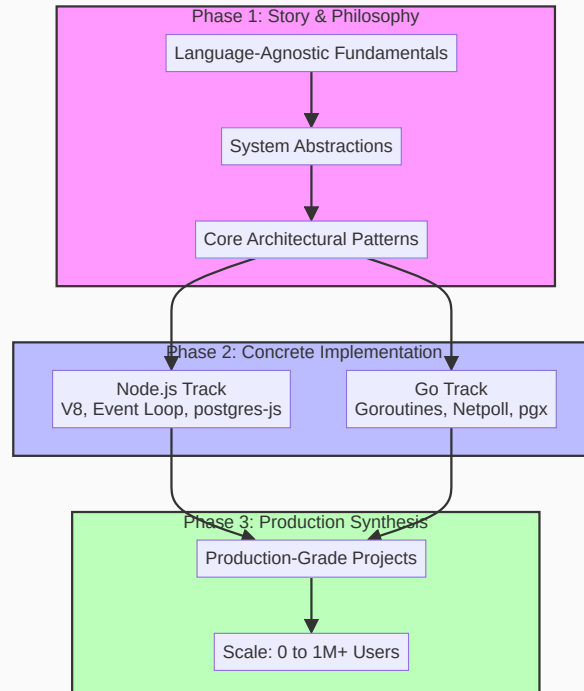
LECTURE STUDY NOTES: WALKING THE PATH OF A TRUE BACKEND ENGINEER

Executive Summary

To transition from a developer who merely writes code to a **true backend engineer**, you must shift your focus from learning specific frameworks to mastering fundamental, language-agnostic architectural principles. This lecture outlines a structured, three-phase pedagogical roadmap designed to build deep conceptual foundations, translate those concepts into highly optimized runtime environments (specifically Node.js and Go), and ultimately synthesize this knowledge to build resilient, production-grade systems capable of scaling from zero to millions of users.

The Three-Phase Pedagogical Roadmap

Mastering backend engineering requires a structured progression. We must first understand architectural concepts in isolation, study how specific runtimes implement them, and finally assemble these patterns into production-grade systems.



Phase I: Story, Philosophy, and Language-Agnostic Foundations

The first phase of the journey focuses entirely on the **core philosophies, conceptual patterns, and mechanics** of backend engineering.

1. The Trap of Framework–First Learning

Many modern developers fall into the trap of learning a framework (e.g., NestJS, Spring Boot, or Django) before understanding the underlying protocols and system behaviors. This creates a fragile knowledge base:

2. Peeling Back the Abstractions

Every tool we use is an abstraction over lower-level concepts. To design scalable systems, an engineer must understand what these tools hide:

High-level Abstraction	What It Abstracts	Under-the-Hood Reality
HTTP Framework (Express/Gin)	Network communication	Raw TCP sockets, buffer allocation, HTTP parsing, and epoll/kqueue system calls.
ORM (Prisma/Gorm)	Data persistence	SQL compilation, connection state management, transaction isolation levels, and wire protocols.
Cloud Hosting (Heroku/Render)	Infrastructure	Virtualization, containerization, network routing, and operating system resource allocation.

By mastering the "story" behind these abstractions, you gain the ability to debug complex performance bottlenecks that high-level abstractions inevitably hide.

Phase II: Implementation & Ecosystem Deep Dives

Once you understand the conceptual "why," you must learn the "how." Phase II bridges theory and execution by mapping abstract principles to two production-proven runtimes: **Node.js** and **Go (Golang)**.



Syntax error in text

mermaid version 10.9.6



Why Node.js and Go?

These two environments represent the modern spectrum of backend architecture:

Deep-Dive Case Study: Database Connectivity

To illustrate the difference between conceptual design and ecosystem implementation, consider how each track handles database connections:

1. THE ABSTRACT CONCEPT

A database driver must serialize queries into a specific wire protocol (e.g., the PostgreSQL frontend/backend protocol), transmit them over a TCP socket, manage a pool of reusable connections to avoid TCP handshake overhead, and handle schema migrations deterministically.

2. THE NODE.JS IMPLEMENTATION

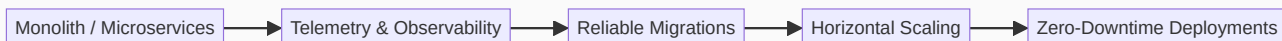
Using a driver like `postgres-js`:

3. THE GO IMPLEMENTATION

Using a driver like `pgx`:

Phase III: Production-Level Synthesis

The final phase brings these concepts and implementations together to build **production-grade systems**.



What Defines "Production-Grade"?

A production-grade application is not defined by its happy-path functionality, but by how it handles failure, scale, and change over time.

Key Takeaways for the Aspiring Engineer

KEY TAKEAWAYS

- **Focus on the "Why" Before the "How":** Never learn a library without understanding the architectural problem it is trying to solve.
- **Be Multi-Paradigm:** A true backend engineer is not a "Node.js Developer" or a "Go Developer." They are system designers who select the right tool based on runtime characteristics and business constraints.
- **Design for Change:** Write code that isolates side-effects (like network calls and database queries) so your application can scale and evolve over its lifecycle.

CH
03

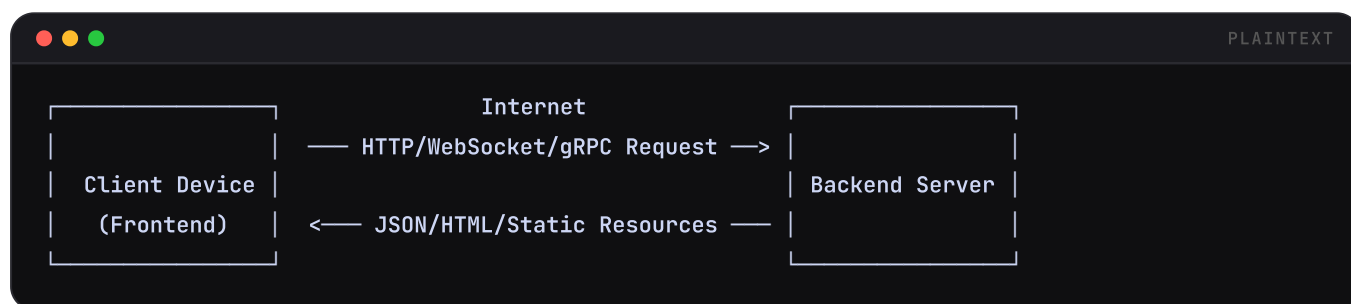
Lecture 3: 3. What is a Backend, how do they work and why do we need them?

LECTURE 3: WHAT IS A BACKEND, HOW DO THEY WORK, AND WHY DO WE NEED THEM?

1. Introduction & High-Level Definition

At its core, a **backend** (or **server**) is a remote computer configured to listen continuously for incoming network requests over specific open ports (such as Port **80** for HTTP or Port **443** for HTTPS).

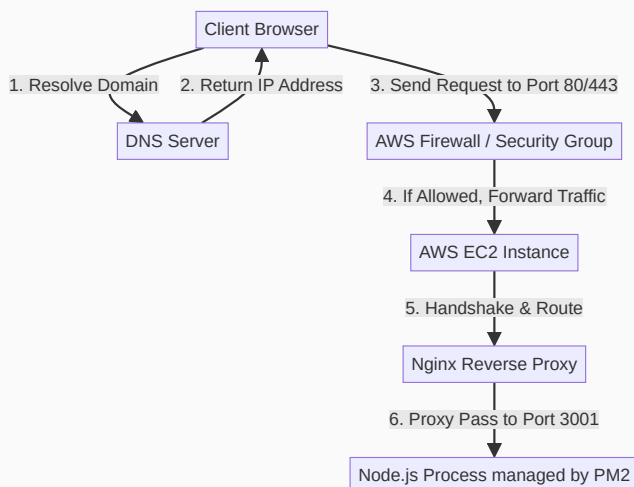
Unlike a client-side application, which runs on a user's device, a backend is designed to process business logic, interact with databases, and return resources to clients over the Internet.



Key Characteristics of a Backend Server

2. The Path of a Request: From Browser to Server

To understand how a backend works in a production environment, we can trace a request from a client's browser to an active server instance deployed on AWS (Amazon Web Services).



Step 1: Domain Resolution via DNS (Domain Name System)

When a user visits a URL like `https://backend-demo.senus.xyz/users`, the browser cannot route the request using the human-readable domain name alone. It must resolve the domain to an **IP (Internet Protocol) Address**.

The browser queries a **DNS Server**, which looks up the registered DNS records for the domain:

`backend-demo.senus.xyz` $\rightarrow$ 54.211.12.34

Step 2: Traversing the Cloud Firewall (AWS Security Groups)

Once the browser obtains the server's public IP address, it dispatches the request. Before the network packets can physically enter the virtual machine (an **AWS EC2 Instance**), they must pass through a cloud firewall—in AWS, this is configured as a **Security Group**.

The Security Group acts as an external gatekeeper. It must be explicitly configured with **Inbound Rules** to allow traffic through specific ports:

***Note:** If Ports `80` and `443` are not whitelisted in the Security Group rules, the cloud provider blocks the packets at the edge. The request times out, never reaching the backend application.*

Step 3: The Reverse Proxy Layer (Nginx)

Once allowed through the firewall, the request arrives at the virtual machine. Instead of exposing the raw application runtime (e.g., Node.js, Python, Go) directly to the public internet, production environments deploy a **Reverse Proxy**—most commonly **Nginx**.

WHY USE A REVERSE PROXY?

NGINX ROUTING CONFIGURATION EXAMPLE

Below is a simplified representation of an Nginx server block configuration. It listens on Port `80`, redirects insecure traffic to Port `443` (HTTPS), and forwards incoming requests for `backend-demo.senus.xyz` to a local process running on Port `3001`:

NGINX

```
# Redirect HTTP to HTTPS
server {
    listen 80;
    server_name backend-demo.senus.xyz;
    return 301 https://$host$request_uri;
}

# Handle Secure HTTPS Traffic
server {
    listen 443 ssl;
    server_name backend-demo.senus.xyz;

    # SSL Certificates managed by Certbot
    ssl_certificate /etc/letsencrypt/live/backend-demo.senus.xyz/fullchain.pem;
    ssl_certificate_key /etc/letsencrypt/live/backend-demo.senus.xyz/privkey.pem;

    location / {
        # Forward public traffic to the local Node.js server
        proxy_pass http://localhost:3001;
        proxy_http_version 1.1;
        proxy_set_header Upgrade $http_upgrade;
        proxy_set_header Connection 'upgrade';
        proxy_set_header Host $host;
        proxy_cache_bypass $http_upgrade;
    }
}
```

Step 4: Process Management (PM2) & Runtime Execution

The final hop occurs inside the server environment. Nginx forwards the traffic to `localhost:3001`.

In production, backend processes are managed by utility tools like **PM2** (Process Manager 2) to ensure high availability.

BASH

```
$ pm2 list
```

id	name	mode	status	cpu	memory	watch
0	frontend-app	fork	online	0.1%	45.2 MB	disable
1	backend-api	fork	online	0%	32.1 MB	disable

WHY USE PM2?

If you access the server's command line and run a local network request using `curl`:

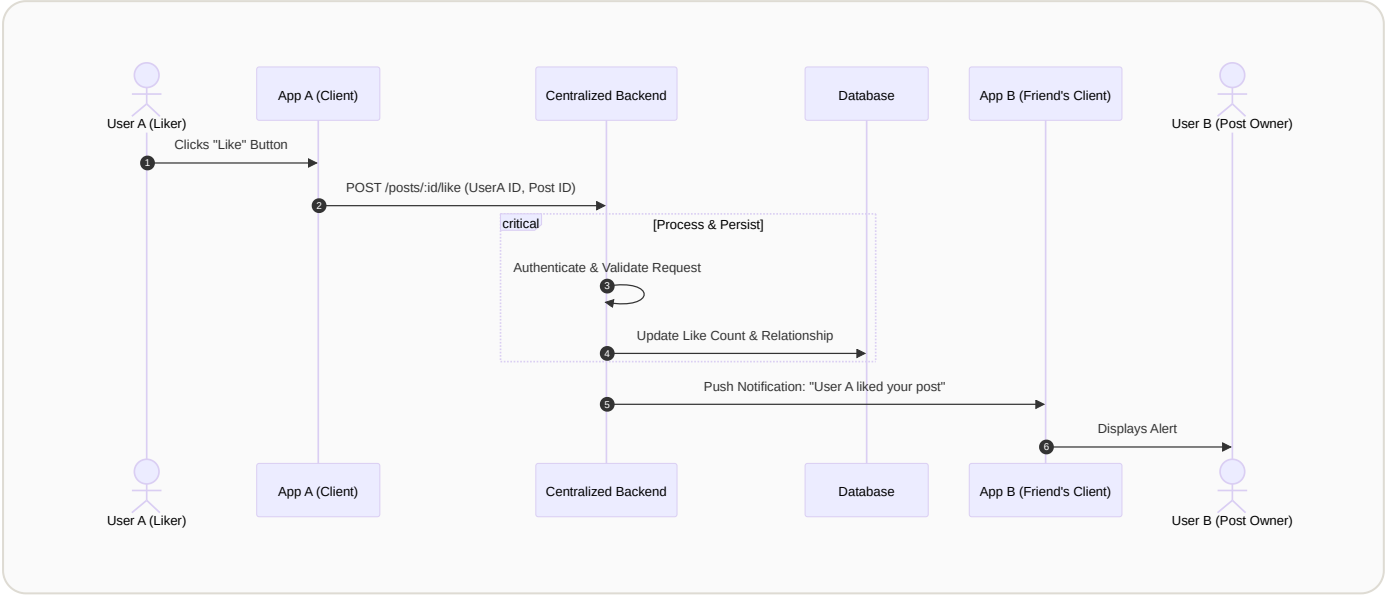
BASH

```
curl http://localhost:3001/users
```

It returns the raw JSON payload directly from the active Node.js memory space, bypassing DNS, the cloud firewall, and Nginx.

3. Why Do We Need a Backend?

To understand why a backend is indispensable, consider a real-world scenario: **Liking a post on Instagram.**



This interaction demonstrates why client applications cannot operate in isolation:

The Defining Core Responsibility: Data

If we condense the entire purpose of a backend into a single concept, it is **Data**:

4. Frontend vs. Backend Execution Models

A common question among developers is: *"If modern client devices (smartphones, laptops) are powerful computers, why can't we execute all our application logic, database queries, and integrations directly on the client side?"*

To understand why this is impossible, we must look at how frontends and backends execute code.

FEATURE	FRONTEND (CLIENT-SIDE)	BACKEND (SERVER-SIDE)
Primary Goal	Render user interface, capture inputs, handle UI interactions.	Process business logic, authenticate users, manage databases.
Delivery Model	Server sends complete source code (HTML, CSS, JS) to the client.	Server runs code internally; sends only structured <i>results</i> (JSON) to client.
Runtime Environment	Browser Sandbox (V8, WebKit, etc.) on end-user hardware.	Controlled Cloud OS (Linux/Windows) on dedicated hardware.
Physical Location	Distributed across millions of uncontrolled client devices.	Centralized cloud instances (AWS, GCP, Azure, bare-metal).

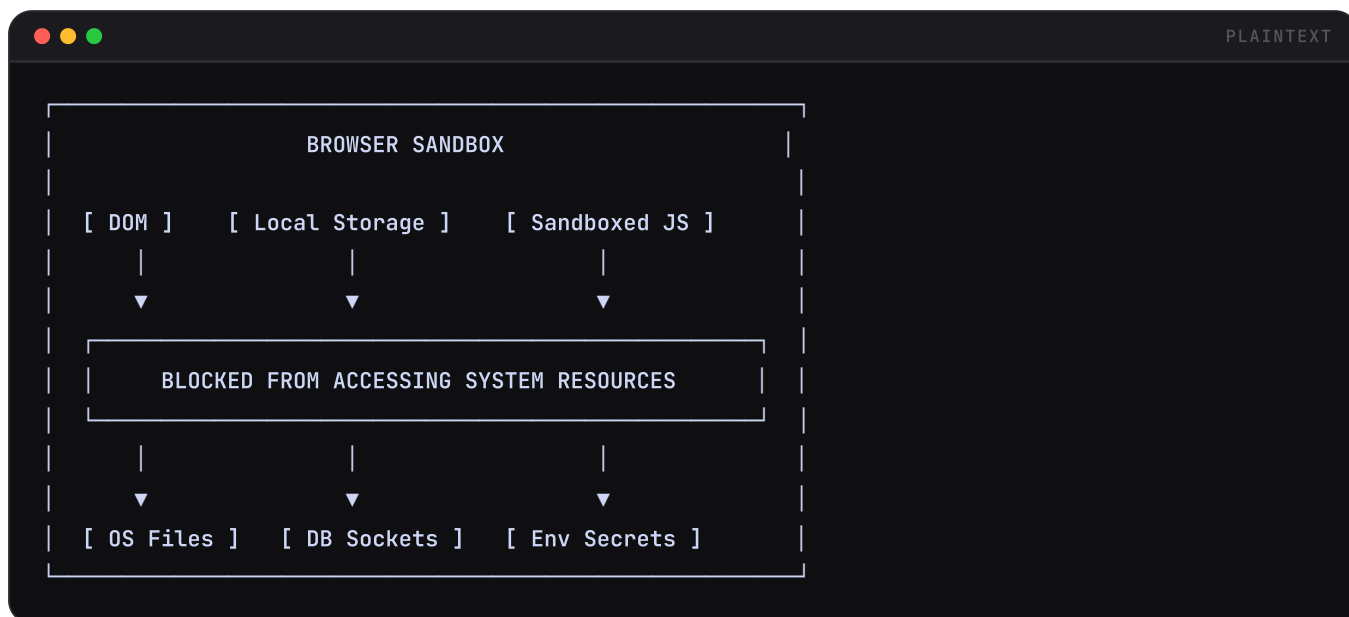
The Frontend Delivery & Hydration Process

In a modern web app (such as a Next.js or React application):

The Key Shift: In this model, the server merely *delivers* the code. The **client's browser is the runtime execution environment**.

5. The Four Pillars: Why Client-Side Engines Cannot Replace Backends

Browsers operate inside a strict **Sandbox**—an isolated environment designed to protect end-users from malicious web scripts. This sandboxed architecture creates four insurmountable barriers that require a dedicated backend.



Pillar 1: Security & OS Isolation

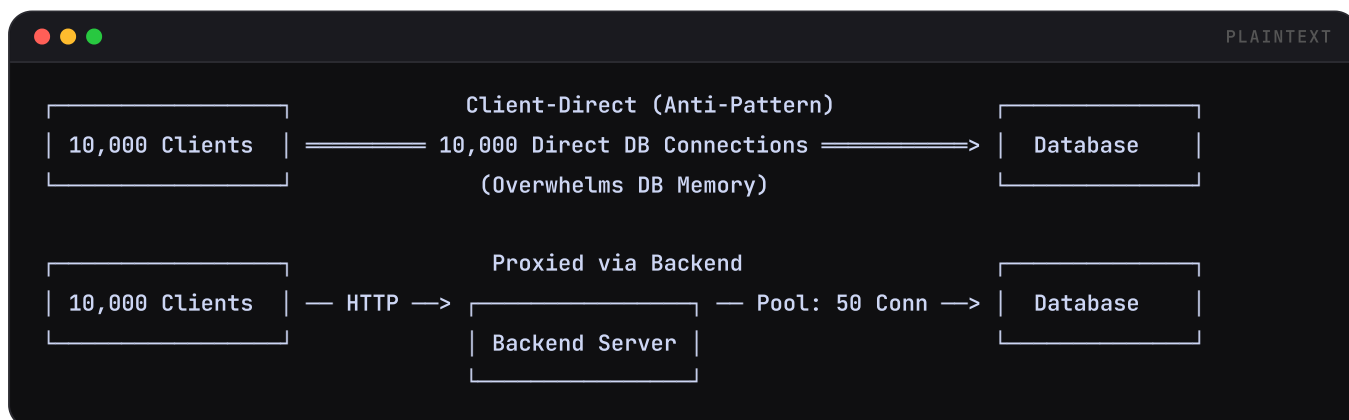
Because browsers download and execute arbitrary code from the internet, they are blocked from accessing the client machine's underlying operating system.

Pillar 2: CORS & Network Restrictions

Browsers enforce a security standard called **CORS (Cross-Origin Resource Sharing)**.

Pillar 3: Database Drivers & Connection Management

Relational databases (like PostgreSQL and MySQL) and NoSQL databases (like MongoDB) require persistent, highly optimized connection channels to operate efficiently.



A backend solves this by acting as a mediator. It maintains a **Connection Pool** (e.g., keeping 20 to 50 persistent connections open). It multiplexes thousands of incoming client HTTP requests across this small, highly efficient pool of database connections:

$\text{\$}\text{\$}\text{\{10,000 Client Requests\}} \rightarrow \text{\{Backend Server\}} \rightarrow \text{\{Multiplexed Pool (50 Connections)\}} \rightarrow \text{\{Database\}}\text{\$}\text{\$}$

Pillar 4: Asymmetric Computing Power

Client devices are highly unpredictable. Your application might run on a high-end desktop, but it is just as likely to run on a low-spec smartphone with a single-core processor and limited RAM.

CH
04

Lecture 4: 4. Benefits of learning backend engineering from first principles

LECTURE 4: BENEFITS OF LEARNING BACKEND ENGINEERING FROM FIRST PRINCIPLES

Executive Summary

In modern software development, engineers frequently face the challenge of jumping into unfamiliar codebases, adopting new programming languages, or selecting the right tool from an overwhelming ecosystem of frameworks and libraries.

This lecture explores why **learning backend engineering from first principles**—rather than focusing on framework-specific syntax—is the most effective way to build a resilient, adaptable, and highly successful career. By understanding the foundational, universal components of backend systems (such as request routing, middleware execution, database connections, and error boundary handling), developers can transition between any technology stack (e.g., moving from Node.js to Rust or Go) with minimal friction. This approach elevates a developer from a "framework-specific coder" to a "true software engineer" who can design scalable, maintainable systems in any environment.

1. The Dilemma of Modern Backend Engineering

A common point of friction for software engineers is the sudden need to operate in an unfamiliar backend environment. Consider these three common scenarios:

PLAINTEXT

[Scenario A: The Frontend/Full-Stack Jump]

Frontend Dev —(Assigned Backend Bug)—> Unfamiliar Codebase —> "Where do I start?"

[Scenario B: The Polyglot Transition]

Go/TS Engineer —(Assigned Rust/Python Project)—> "How do I map my mental models?"

[Scenario C: The Green-field Challenge]

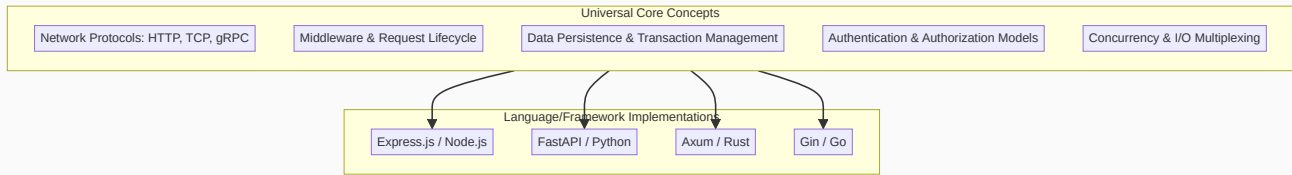
Engineer —(Create API from scratch)—> Boilerplate Tutorials —> "Is this production-grade?"

These challenges stem from a common root cause: **learning systems from the top down (framework-first) rather than from the bottom up (principles-first).**

2. Defining "First Principles" in Backend Engineering

In physics and philosophy, **first principles thinking** involves breaking a complex system down to its most basic, fundamental truths that cannot be deduced any further, and building up a solution from there.

When applied to backend engineering, first principles are the universal mechanisms that remain constant regardless of the language, framework, or cloud provider.



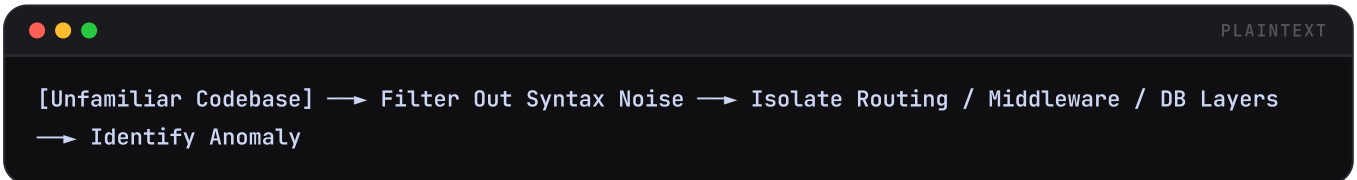
Whether you are using Node.js (Express), Python (FastAPI), Go (Gin), or Rust (Axum), every backend application must solve the exact same set of core problems:

If you master these universal concepts, syntax becomes a secondary detail.

3. The 6 Core Benefits of First-Principles Thinking

Benefit 1: Seeing the Big Picture (Developing "CTO Intuition")

When senior engineers, software architects, or CTOs look at an unfamiliar codebase, they can quickly pinpoint issues or map system flows within minutes. This is not magic; it is highly developed **pattern recognition**.



By practicing first principles, you deliberately train your brain to filter out syntactic noise and identify the core components of any system:

Instead of waiting 5 to 10 years to develop this intuition subconsciously, you can acquire it in 6 to 12 months of deliberate, principles-focused practice.

Benefit 2: Faster Onboarding

When you understand how HTTP works, how databases interact with APIs, and how requests flow through a middleware pipeline, onboarding to a new team or codebase becomes incredibly fast.

You no longer need to read thousands of pages of framework-specific documentation. Instead, your onboarding questions shift from *"How do I write a route?"* to structural inquiries:

Benefit 3: 10x Faster Execution on New Projects

When starting greenfield projects, developers without first-principles knowledge often copy-paste boilerplate templates they do not fully understand. This introduces security vulnerabilities, performance bottlenecks, and structural debt early on.

With a first-principles foundation, you can build production-quality Minimum Viable Products (MVPs) rapidly. You know exactly how to structure your routes, establish safe database connections, and implement robust error boundaries and logging from day one without constantly searching the web for basic tutorials.

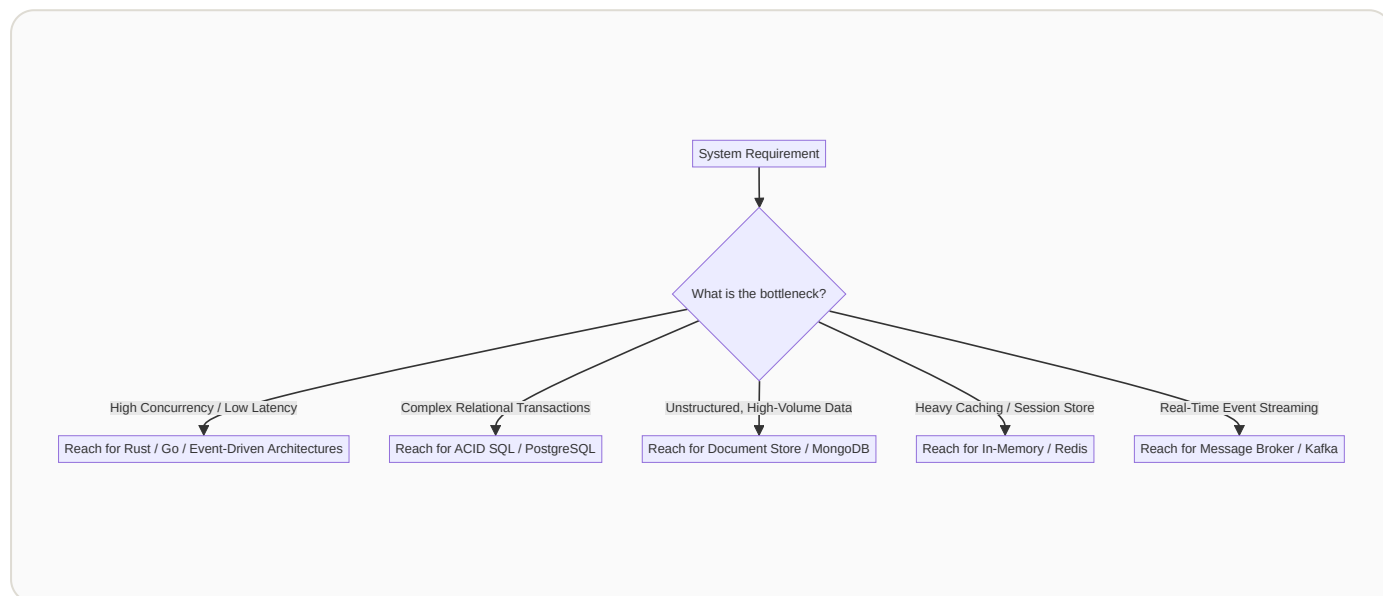
Benefit 4: Eliminating "Syntax Fatigue"

Learning a new programming language is mentally taxing. If you are trying to learn both a new language *and* how to design a backend simultaneously, you will quickly experience burnout.

First principles reduce this syntax fatigue. When you know exactly *what* problem you are trying to solve, learning the new language simply becomes a search for the correct syntax to express your existing mental model.

Benefit 5: Choosing the Right Tool for the Job

Many developers get stuck in "identity silos" (e.g., "*I am a Node.js developer*" or "*I am a Ruby on Rails engineer*"). When faced with unique system requirements, such as ultra-low latency or massive write concurrency, they try to force-fit their familiar stack, even if it is highly sub-optimal.



By understanding the underlying trade-offs of different systems, you gain the confidence to select the best technology objectively:

Benefit 6: Exponentially Higher Employability

In a volatile technology landscape, framework-specific developers are highly vulnerable to market shifts. A developer who only knows React or Django will struggle if their stack's popularity declines.

Employers want **adaptable problem-solvers** who can join any team, quickly grasp any codebase, and deliver value.

Mastering first principles makes you highly versatile and language-agnostic, which is the hallmark of a senior software engineer.

4. Deconstructing the Transition: A Case Study (Node.js to Rust)

To illustrate how first principles eliminate syntax fatigue and accelerate development, let us look at how a developer goes about transitioning from a Node.js environment to Rust.

The Problem

The developer wants to build a high-performance, production-grade REST API in Rust. However, they cannot find a single 5-hour tutorial that explains how to structure a production-ready Rust codebase from end-to-end.

The First-Principles Solution

Instead of searching for a monolithic tutorial, the developer breaks the backend down into its core, universal building blocks. They then implement each block incrementally using Rust.



Syntax error in text

mermaid version 10.9.6

By mapping out these universal steps, the developer can tackle each block individually:

- STEP 1: ROUTING LAYER
- STEP 2: AUTHENTICATION MIDDLEWARE
- STEP 3: INPUT VALIDATION
- STEP 4: DATABASE INTERACTION & CONNECTION POOLING

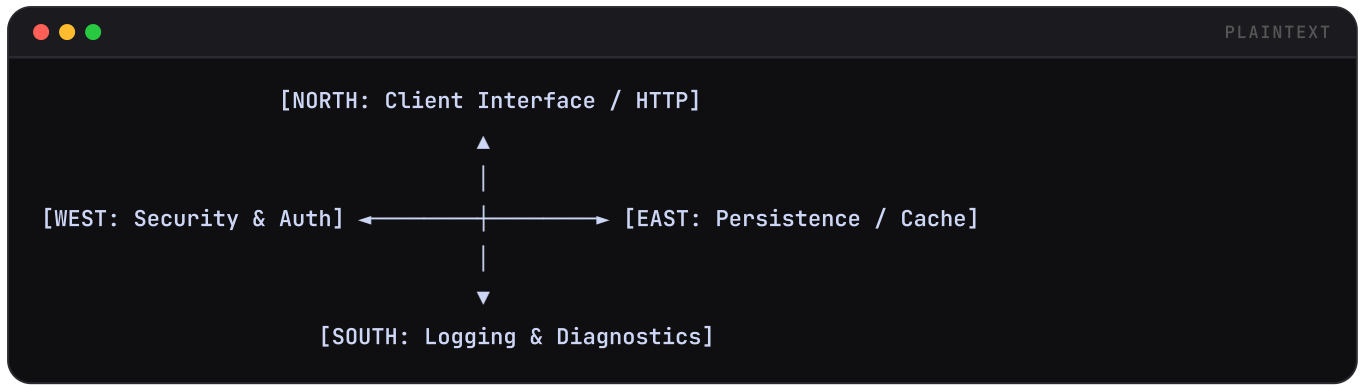
Using this modular approach, the developer can build a production-quality backend in Rust in just a few days, even if they have never written a full Rust application before.

5. Comparative Paradigm: Framework-Centric vs. First-Principles

Metric	Framework-Centric Developer	First-Principles Engineer
Primary Mental Model	"How does my framework (e.g., NestJS) do this?"	"How does the underlying system (e.g., HTTP/TCP) handle this?"
Onboarding Speed	Slow; requires learning specific folder structures, CLI tools, and magic decorators.	Fast; immediately maps routing, middleware, and database layers.
Debugging Strategy	Searches StackOverflow for framework-specific error strings.	Traces the network stack, examines system logs, and inspects database transactions.
Adaptability	High friction when switching stacks; prone to "syntax fatigue" and tech-stack identity lock-in.	Fluid; transitions between Go, Rust, Node.js, or Python seamlessly.
Architectural Decisions	Uses default framework configurations, often leading to over-engineering.	Selects optimal architectures, database schemas, and caching strategies based on actual constraints.

6. Building Your "Internal Compass"

Mastering backend engineering from first principles is about building an **internal compass**. When you step onto unfamiliar ground, this compass ensures you are never truly lost.



As we move forward, we will map out this backend territory in detail. We will deconstruct:

By focusing on these enduring concepts, you will build a deep understanding of backend systems that will serve you throughout your career, regardless of how the technology landscape evolves.

CH
05

Lecture 5: 5. Understanding HTTP for backend engineers, where it all starts

LECTURE 5: UNDERSTANDING HTTP FOR BACKEND ENGINEERS

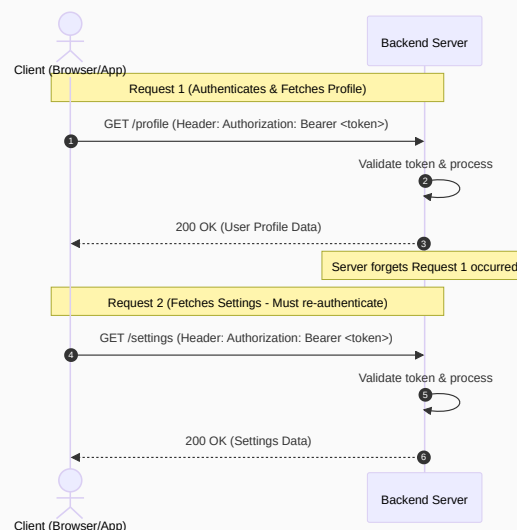
1. Introduction to the Backend Architecture Scope

The backend ecosystem is incredibly vast. Attempting to master every single tool, protocol, and configuration parameter upfront can lead to information overload. To build a robust foundation, this series adopts the **90/10 Rule**: focusing intensively on the core 10% of concepts that power 90% of real-world production codebases.

At the absolute center of this core lies the **HTTP (Hypertext Transfer Protocol)** protocol. Whether you are building microservices, RESTful APIs, server-rendered applications, or integrating third-party integrations, HTTP is the primary medium through which clients (browsers, mobile apps, other servers) and backend servers communicate.

2. Core Pillars of the HTTP Protocol

HTTP is designed around two foundational architectural paradigms: **Statelessness** and the **Client-Server Model**.



A. Statelessness

Statelessness means that the protocol has **no memory of past interactions**. Each HTTP request is treated as an entirely isolated, independent event.

BENEFIT	DESCRIPTION
Simplicity	Minimizes server-side complexity. The backend does not need to allocate memory or synchronize session states across complex clusters.
Scalability	Requests can be seamlessly routed to any active server instance behind a load balancer. If Server A crashes, Server B can handle the next incoming request because the request contains all necessary state context.
Fault Tolerance	No active sessions are lost during server deployments, crashes, or rolling restarts.

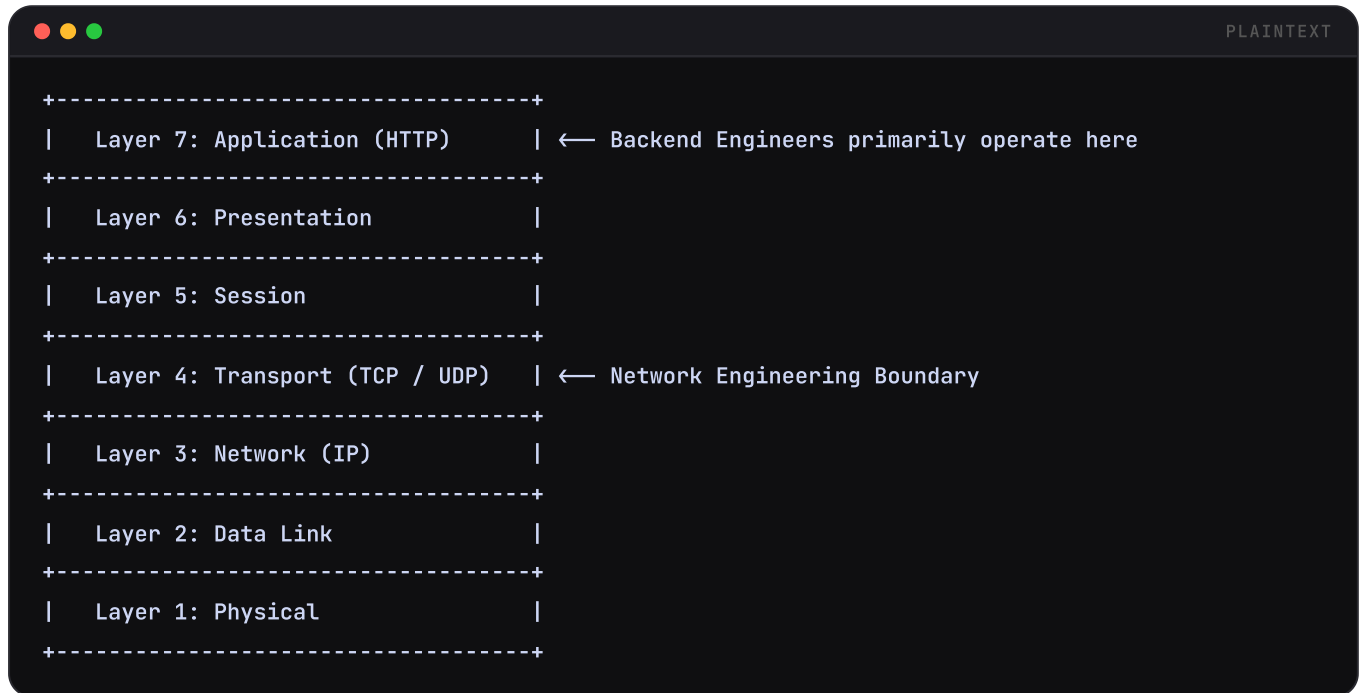
B. The Client–Server Model

HTTP strictly dictates a unidirectional, pull-based architecture.

Note: For real-time, bidirectional communication, modern applications use protocols like WebSockets or HTTP-based patterns like Server-Sent Events (SSE).

3. The Transport Layer & The OSI Context

To understand how HTTP operates, we must look at the **OSI (Open Systems Interconnection) Model**, which conceptualizes how data travels across networks in structured layers.



Layer 7 (Application) vs. Layer 4 (Transport)

Backend application developers primarily work within **Layer 7 (Application Layer)**. However, Layer 7 protocols rely on Layer 4 protocols to manage the raw transmission of data across the network.

HTTP does not dictate *how* packets are sent over the wire; it simply requires that the underlying transport layer be **reliable** (i.e., packets must not be lost, and they must arrive in the correct order).

TCP vs. UDP

4. Evolution of HTTP (1.0 to 3.0)

As web applications grew more complex, the limitations of older HTTP versions became apparent. The protocol evolved to optimize network utilization and reduce latency:

VERSION	UNDERLYING TRANSPORT	KEY FEATURES	PERFORMANCE IMPACT
HTTP/1.0	TCP	Single request per connection.	Inefficient. Opening/closing a new TCP connection for every image, stylesheet, and script introduced massive latency.
HTTP/1.1	TCP	Persistent Connections (Keep-Alive by default), chunked transfers, caching.	Significant improvement. Reuses a single TCP connection for multiple sequential requests.
HTTP/2.0	TCP	Multiplexing , Binary Framing, Header Compression (HPACK), Server Push.	High optimization. Allows sending multiple requests and responses concurrently over a single TCP connection, eliminating the head-of-line (HoL) blocking at the application level.
HTTP/3.0	QUIC (UDP-based)	Multiplexing without transport-level HoL blocking, faster connection times.	Solves TCP-level HoL blocking (where a single lost packet stalls all streams). Dramatically improves performance on unstable mobile networks.

5. Anatomy of HTTP Messages

HTTP communication consists of plain-text messages (in HTTP/1.1) structured into highly predictable parts.

A. The HTTP Request Message

HTTP

```
POST /api/v1/users HTTP/1.1
Host: api.example.com
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64)
Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cGE6ICJ0eS...
Content-Type: application/json
Content-Length: 47

{"username": "johndoe", "email": "john@doe.com"}
```

B. The HTTP Response Message

HTTP

```
HTTP/1.1 201 Created
Date: Wed, 23 Oct 2024 10:00:00 GMT
Server: Nginx/1.18.0
Content-Type: application/json; charset=utf-8
Content-Length: 58

{"id": 101, "username": "johndoe", "status": "active"}
```

6. HTTP Headers: Extensibility and Control

HTTP headers act as metadata wrappers. Think of them like a **shipping label** on a physical parcel: courier systems read the label on the outside of the box to route and handle the package without opening the box to inspect the contents.

PLAINTEXT

```
+-----+
| SHIPPING LABEL (HTTP Headers) |
| - Destination: api.example.com |
| - Fragile? (Content-Type: JSON) |
| - Access Permit (Auth Token) |
+-----+
|
| PARCEL CONTENTS (HTTP Body Payload) |
| {"username": "johndoe"} |
|
+-----+
```

A. Core Categories of Headers

1. REQUEST HEADERS

Sent by the client to convey its configuration environment, authentication state, and formatting preferences.

2. GENERAL HEADERS

Applicable to both requests and responses, providing context about the message itself rather than the payload content.

3. REPRESENTATION HEADERS

Describe the physical format, size, encoding, and validation states of the message body.

4. SECURITY HEADERS

Instruct the browser client to enforce strict security policies, preventing various injection and clickjacking attacks.

B. The Dual Power of Headers: Extensibility and Remote Control

7. HTTP Methods and Semantics

HTTP methods define the **intent** of a request. Using them correctly ensures that your API behaves predictably and conforms to standard RESTful patterns.

PLAINTEXT

HTTP METHOD CHARACTERISTICS

+-----+		
	GET	
	- Safe (Read-only)	
	- Idempotent	
+-----+		
	PUT	
	- Unsafe (Modifies state)	
	- Idempotent (Full replacement)	
+-----+		
	PATCH	
	- Unsafe (Modifies state)	
	- Non-Idempotent (Selective update)	
+-----+		
	POST	
	- Unsafe (Modifies state)	
	- Non-Idempotent (Creates resource)	
+-----+		

A. The Primary Methods

B. Idempotency vs. Non-Idempotency

A critical concept for backend engineers is **Idempotency**: whether executing an operation multiple times yields the same system state as executing it once.

\$\$\text{Idempotent Operation: } f(f(x)) = f(x)\$\$\$

8. Same-Origin Policy (SOP) & CORS (Cross-Origin Resource Sharing)

A. The Same-Origin Policy (SOP)

The Same-Origin Policy is a fundamental **browser-enforced** security mechanism. It prevents a malicious script running on one origin (e.g., `evil.com`) from reading or interacting with sensitive data hosted on another origin (e.g., `yourbank.com`).

An **Origin** is defined by the unique combination of three components:

\$\$\text{Origin} = \text{Protocol (Scheme)} + \text{Domain (Host)} + \text{Port}\$\$\$

If any of these three elements differ, the browser treats the request as **Cross-Origin**.

PLAINTEXT

```
Client Origin: http://localhost:5173
Target Origin: http://localhost:3000
Result: CROSS-ORIGIN (Port mismatch)
```

B. What is CORS?

Cross-Origin Resource Sharing (CORS) is a mechanism that allows servers to selectively bypass SOP restrictions. It lets a server declare which external origins are allowed to read its responses.

Crucial Backend Concept: CORS is not a mechanism that prevents a server from executing a request. The backend server will receive, process, and respond to the request. However, if the server's response lacks the correct CORS headers, the **browser** will block the client-side JavaScript from reading the returned data.

C. Simple vs. Preflighted Requests

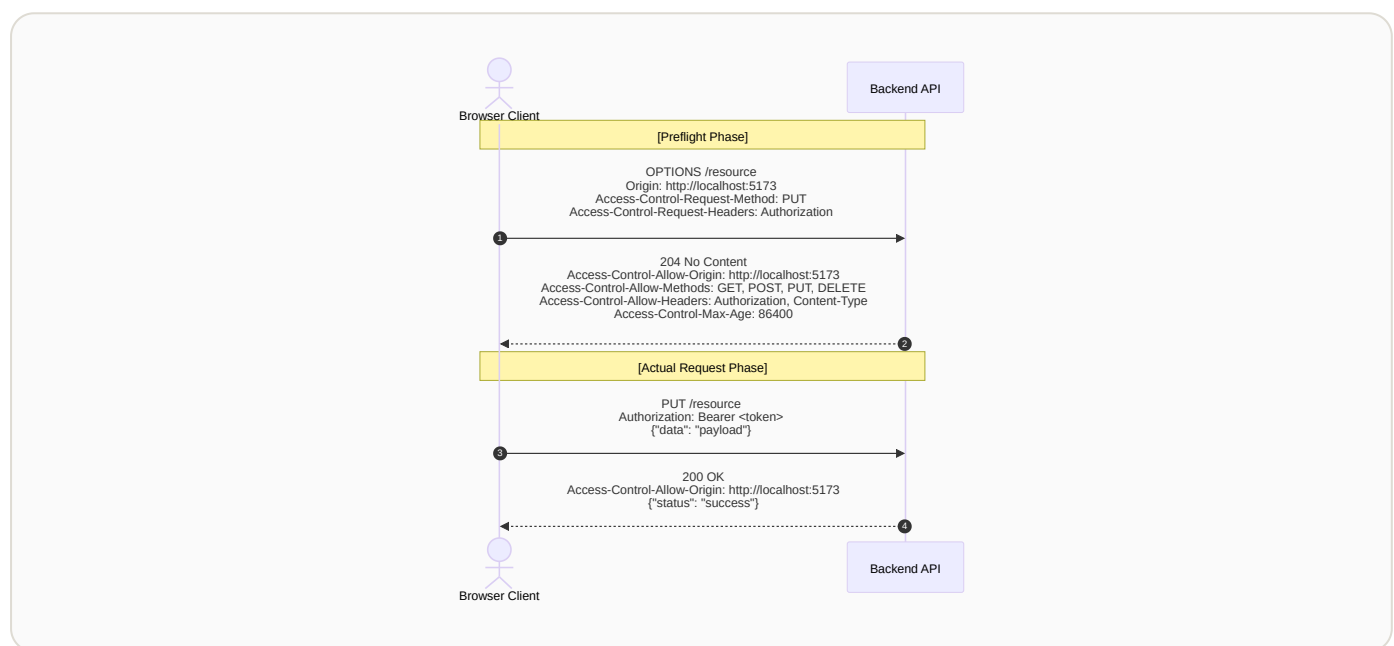
Browsers categorize cross-origin requests into two categories:

1. SIMPLE REQUESTS

These requests do not trigger an initial handshake. They must meet strict criteria:

2. PREFLIGHTED REQUESTS

If a request uses complex methods (like **PUT** or **DELETE**), custom headers (like **Authorization**), or transmits JSON data (**application/json**), the browser will automatically send a **Preflight Request** using the **OPTIONS** method before sending the actual request.



D. Key CORS Headers Explained

9. Standardized HTTP Status Codes

Status codes are structured into five distinct numeric classes, providing a universal language for client-server communication:

PLAINTEXT

```
+-----+
|           HTTP STATUS CLASSES           |
|                                           |
| 1xx: Informational (e.g., 101 Switching Protocols) |
| 2xx: Success (e.g., 200 OK, 201 Created, 204 No Content) |
| 3xx: Redirection (e.g., 301 Permanent, 304 Not Modified) |
| 4xx: Client Errors (e.g., 401 Unauthorized, 403 Forbidden) |
| 5xx: Server Errors (e.g., 500 Internal, 502 Bad Gateway) |
+-----+
```

1xx: Informational Responses

2xx: Success Responses

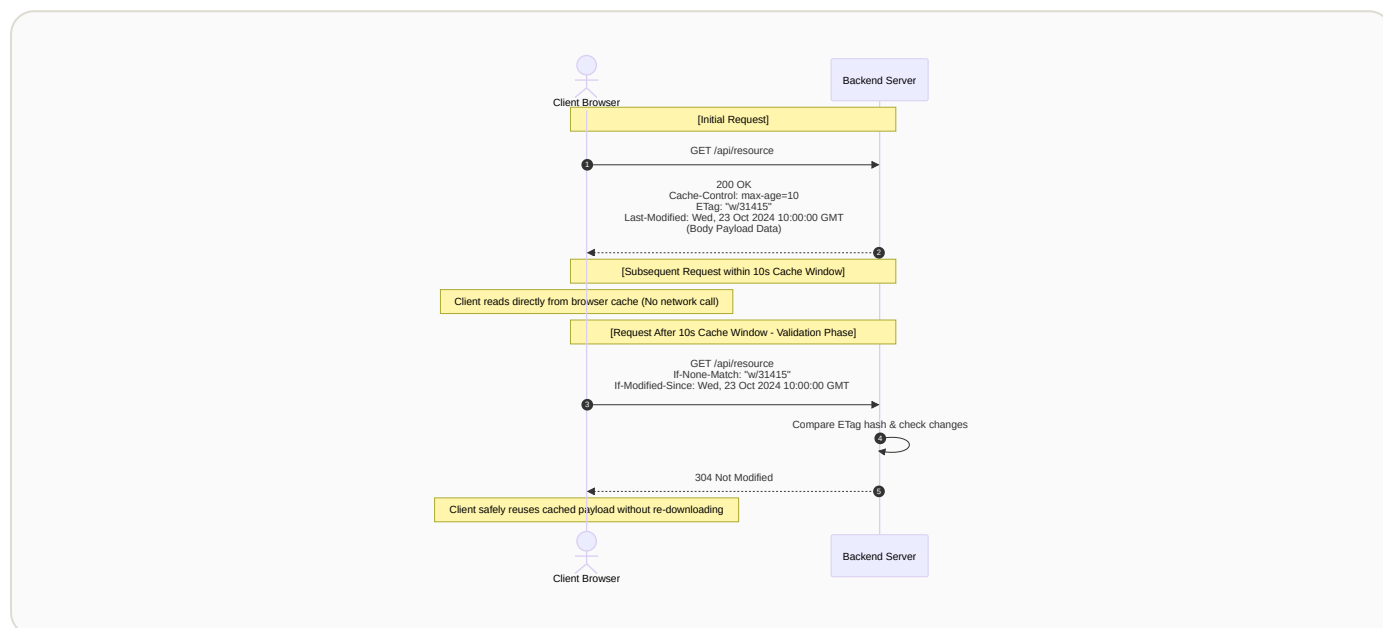
3xx: Redirection Responses

4xx: Client Errors

5xx: Server Errors

10. HTTP Caching and Cache Validation

HTTP caching reduces network traffic, lowers server load, and speeds up page load times by reusing previously fetched resources.



A. Expiration Control

CH
06

Lecture 6: 6. What is Routing in Backend? How Requests Find Their Way Home

LECTURE STUDY NOTES – LECTURE 6: WHAT IS ROUTING IN BACKEND? HOW REQUESTS FIND THEIR WAY HOME

1. Introduction to Routing: The "Where" of a Request

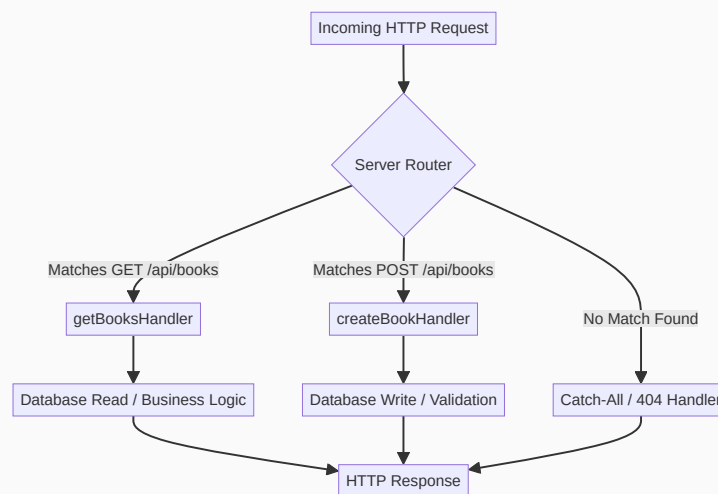
In backend engineering, every client-server interaction relies on a precise coordination of intent and destination.

$$\text{HTTP Method (What)} + \text{Route Path (Where)} \rightarrow \text{Target Server-Side Handler (Action)}$$

The Mapping Mechanism

At its core, **routing** is the process of mapping incoming URL paths (and metadata) to specific server-side execution handlers.

When an HTTP request hits a web server, the server does not execute code at random. It parses the request, extracts the HTTP method and the URI path, and matches them against an internal lookup table known as the **Routing Table**.



2. Static Routing

Static routes are constant, invariable path strings that map directly to a dedicated controller. They contain no dynamic variables or wildcards.

Key Characteristics

Example: Demultiplexing Identical Paths with HTTP Methods

Consider two requests targeting the exact same static path, `/api/books`:

REQUEST A (FETCH COLLECTION)

```
GET /api/books HTTP/1.1
Host: api.library.local
```

REQUEST B (CREATE RESOURCE)

```
POST /api/books HTTP/1.1
Host: api.library.local
Content-Type: application/json

{
  "title": "Designing Data-Intensive Applications",
  "author": "Martin Kleppmann"
}
```

SERVER-SIDE RESOLUTION

Even though the path `/api/books` is identical, the router combines the **HTTP Method** and the **Route Path** to form a unique composite key. This ensures that Request A and Request B never collide, routing them to entirely different logical processes:

`Key_1 = (GET, /api/books) → fetchBooks()`

`Key_2 = (POST, /api/books) → insertBook()`

3. Dynamic Routing (Path Parameters)

When building real-world applications, you cannot hardcode a static route for every single resource instance (e.g., `/api/users/1`, `/api/users/2`, ..., `/api/users/999999`). Instead, we use **Dynamic Routing** via **Path Parameters** (also called Route Parameters).

Syntax and Conventions

In modern web frameworks (such as Express.js, Spring Boot, FastAPI, or Go Chi), dynamic segments of a path are denoted by a prefix—most commonly a colon (`:`).

```
/api/users/:id
```

Here, `:id` acts as a wildcard slot. When a request matches this pattern, the server extracts the value in that slot and exposes it to the developer as a variable.

Under the Hood: Request Matching and Extraction

When a client sends a request to `/api/users/123`, the router parses the path segment by segment:

```
Segment 1: "api"   = "api"   (Match)
Segment 2: "users" = "users" (Match)
Segment 3: "123"  = ":id"   (Match! "123" is assigned to the variable "id")
```


JAVASCRIPT

```
// Example Server-Side Handler (Express-like Pseudocode)
router.get('/api/users/:id', (req, res) => {
  // The router automatically parses "123" as a string
  const userId = req.params.id;
  const user = db.findUserById(userId);
  res.json({ userId: userId, data: user });
});
```

Note on Data Types: Even if a path parameter looks like an integer (e.g., `123`), web application routers extract route components as **strings**. It is the developer's responsibility to sanitize and cast these variables to the appropriate data types (e.g., integer, UUID) before querying databases.

Semantic Value in REST APIs

Dynamic path parameters are highly semantic. They establish a clear, human-readable hierarchy of resource access, letting both the developer and the client easily understand the relationship between resources:

```
GET /api/users/123 \implies "Fetch the specific User resource whose unique identifier is 123"
```

4. Query Parameters

While path parameters identify a *specific resource*, **Query Parameters** are used to sort, filter, paginate, or provide metadata about the request.

Syntax

Query parameters append to the end of a URL, separated from the path by a question mark (`?`). They are structured as `key=value` pairs, separated by ampersands (`&`).

PLAINTEXT

```
/api/search?query=distributed+systems&limit=10
```

Path Parameters vs. Query Parameters

Choosing between path parameters and query parameters is a key step in designing RESTful APIs:

FEATURE	PATH PARAMETERS (/API/USERS/:ID)	QUERY PARAMETERS (/API/USERS? STATUS=ACTIVE)
Primary Purpose	Resource Identification (locating a specific item).	Resource Manipulation (filtering, sorting, searching, paginating).
REST Semantics	Represents a strict hierarchical relationship.	Represents non-hierarchical modifiers.
Mandatory?	Yes (the request cannot match the route without it).	No (optional; defaults are safely handled on the server).

Real-World Use Case: Pagination

GET requests should not contain a request body according to standard HTTP semantics. Therefore, query parameters are the standard way to implement safe, stateless server-side pagination.

THE REQUEST

```
GET /api/books?page=2&limit=20 HTTP/1.1
Host: api.library.local
```

THE PAGINATED RESPONSE STRUCTURE

When the server receives this request, it fetches the requested slice of data and returns it alongside metadata to help the client handle navigation:

```
{
  "metadata": {
    "totalCount": 100,
    "currentPage": 2,
    "totalPages": 5,
    "limit": 20
  },
  "data": [
    { "id": 21, "title": "Book 21" },
    { "id": 22, "title": "Book 22" }
  ]
}
```

Using this metadata, the client-side application can easily construct the next logical query parameter request:

```
/api/books?page=3&limit=20
```

5. Nested Routing

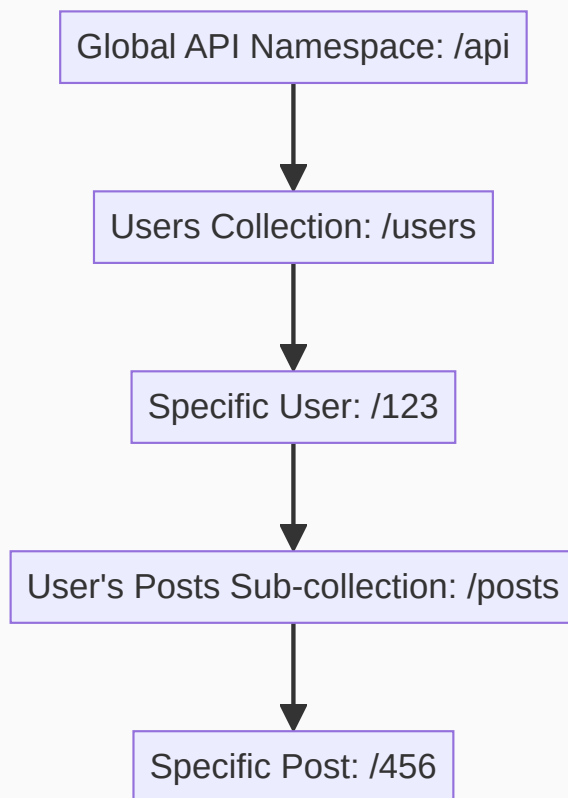
Nested routing is an architectural practice used to represent complex hierarchical relationships where one resource is owned by or nested within another.

Resource Hierarchy Breakdown

Consider the following nested endpoint:

```
GET /api/users/123/posts/456
```

This route contains two dynamic parameters: a user ID (**123**) and a post ID (**456**). The URL structure clearly maps out the underlying database relationships:



Progressive Routing Resolution

A well-designed API router can handle various levels of nesting along this path to serve different data requirements:

6. Route Versioning and Deprecation

As software evolves, API payloads and data schemas change. To prevent updates from breaking existing client installations (e.g., legacy mobile apps, external integrations), backend architectures use **Route Versioning**.

The Problem: Schema Drift & Breaking Changes

Imagine an API that returns a list of products:

```
// V1 Response (Legacy Client Contract)
{
  "data": [
    { "id": 1, "name": "Mechanical Keyboard", "price": 99.99 }
  ]
}
```

The product database changes, and the field `"name"` is renamed to `"title"`. If you push this change directly to the main endpoint, any legacy front-end application expecting `"name"` will break.

The Solution: Route Versioning

By embedding a version marker (e.g., `/v1/` or `/v2/`) directly into the route path, you can run multiple versions of your API in parallel.

JAVASCRIPT

```
// Express-like Implementation of Dual-Version Routing
router.get('/api/v1/products', handleV1Products); // Returns { id, name, price }
router.get('/api/v2/products', handleV2Products); // Returns { id, title, price }
```

Deprecation Workflow

Route versioning gives engineering teams a clean, structured deprecation window:

PLAINTEXT

```
[Release V2 API]
  |
  ▼
[Notify Consumers / Deprecate V1 (Mark as Deprecated in Headers/Docs)]
  |
  ▼
[Migration Window: Front-end / Clients update requests to /v2/]
  |
  ▼
[Sunset V1 API safely without breaking active clients]
```

7. Catch-All Routes (Wildcard Routing)

In any routing table, requests are evaluated sequentially or through a trie-based matching tree. If an incoming request does not match any of the registered routes, the server needs a fallback mechanism to handle it gracefully. This is where **Catch-All Routes** (or Wildcard Routes) come in.

Syntax

Typically declared at the very bottom of the routing file using a wildcard character (usually `*` or `/*`):

JAVASCRIPT

```
// Catch-all route must be defined LAST
router.get('*', (req, res) => {
  res.status(404).json({
    error: "Not Found",
    message: `The requested path [${req.path}] does not exist on this server.`
  });
});
```

Why Catch-All Routes are Crucial

8. Summary & Reference Implementation

Below is a complete reference implementation demonstrating all the routing patterns discussed in this lecture.

JAVASCRIPT

```
const express = require('express');
const app = express();
app.use(express.json());

// =====
// 1. STATIC ROUTE
// =====
app.get('/api/books', (req, res) => {
  res.json({ message: "Fetched all books." });
});

// =====
// 2. DYNAMIC ROUTE (Path Parameter)
// =====
app.get('/api/users/:id', (req, res) => {
  const userId = req.params.id; // Extracted dynamic path parameter
  res.json({ userId: userId, message: `Fetched profile for user ${userId}` });
});

// =====
// 3. QUERY PARAMETERS (Filtering & Pagination)
// =====
app.get('/api/search', (req, res) => {
  const { query, limit, page } = req.query; // Extracted query parameters
  res.json({
    results: [],
    meta: { searchQuery: query, limit: limit || 10, page: page || 1 }
  });
});

// =====
// 4. NESTED ROUTE (Hierarchical Resource Mapping)
// =====
app.get('/api/users/:userId/posts/:postId', (req, res) => {
  const { userId, postId } = req.params;
  res.json({
    userId: userId,
    postId: postId,
    message: `Fetched post ${postId} written by user ${userId}`
  });
});

// =====
// 5. ROUTE VERSIONING
// =====
app.get('/api/v1/products', (req, res) => {
  res.json({ products: [{ id: 1, name: "V1 Legacy Product" }] });
});
```

```
app.get('/api/v2/products', (req, res) => {
  res.json({ products: [{ id: 1, title: "V2 Modern Product Spec" }] });
});

// =====
// 6. CATCH-ALL ROUTE (Must be registered last)
// =====
app.use('*', (req, res) => {
  res.status(404).json({
    status: 404,
    error: "Route Not Found",
    suggestion: "Please check your URL path and HTTP verb."
  });
});

app.listen(3000, () => console.log('Server running on port 3000'));
```

CH
07

Lecture 7: 7. Serialization and Deserialization for backend engineers

LECTURE 7: SERIALIZATION AND DESERIALIZATION FOR BACKEND ENGINEERS

1. Introduction to the Communication Problem

In modern web architecture, systems are rarely monolithic or written in a single programming language. A typical production environment consists of a client-side interface (the Frontend) communicating with one or more server-side systems (the Backend).

Client: Browser
JavaScript / React / Vue

Network Communication
HTTP / gRPC / WebSockets

Server: Backend
Rust / Go / Java

The Heterogeneity Challenge

Consider a client application built with **JavaScript** (e.g., a React or Angular SPA running in Google Chrome) communicating with a backend server written in **Rust**.

When a user submits a form on the frontend, JavaScript represents this data in memory as a runtime object:

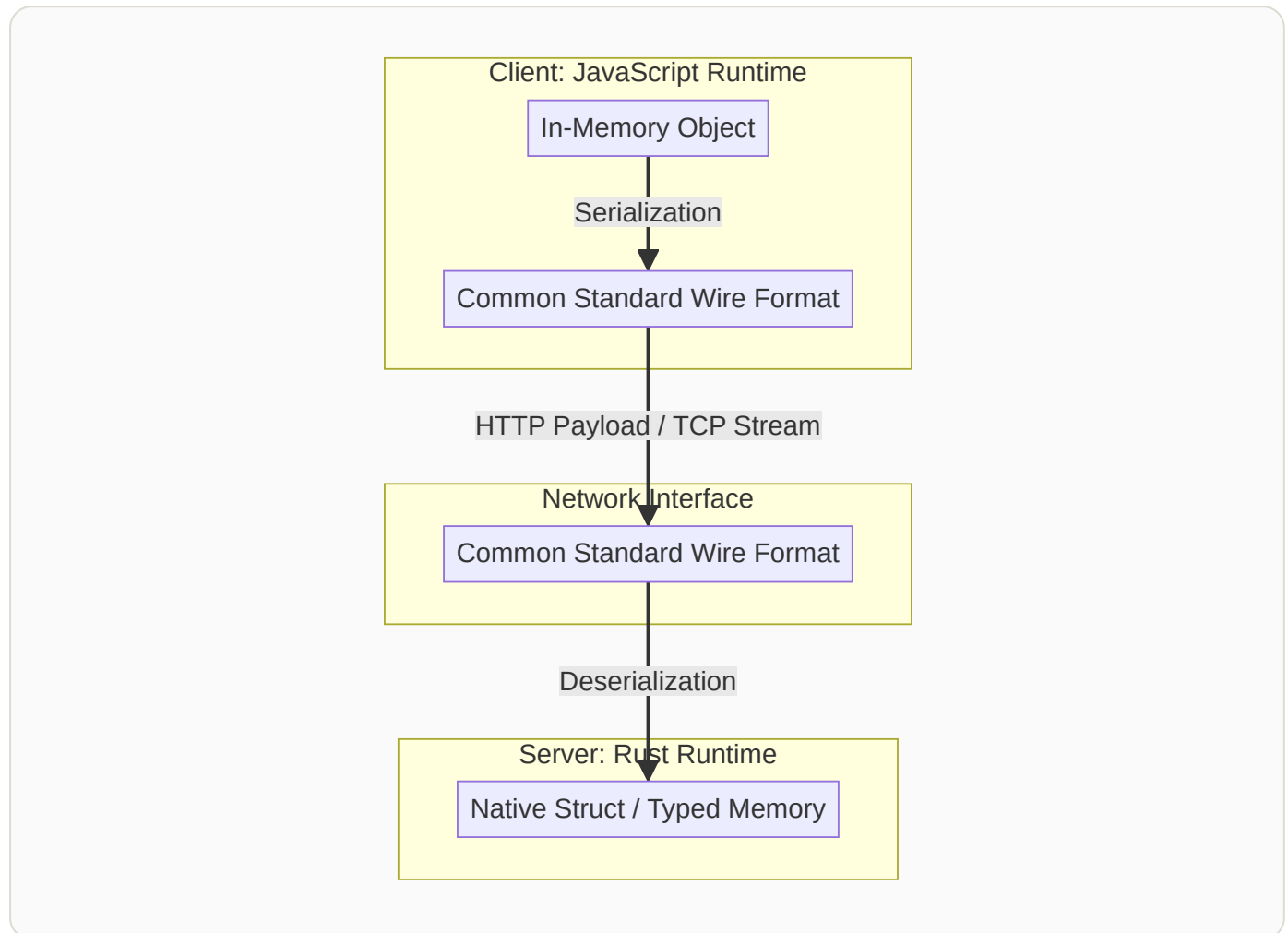
```
// JavaScript In-Memory Object
const user = {
  name: "ALex",
  age: 30
};
```

If the JavaScript engine tried to transmit the raw memory address or binary layout of this object directly to the Rust server, the Rust server would not be able to parse it. Rust's memory layout for structs is completely different from JavaScript's V8 engine object representation.

This mismatch requires a way to translate in-memory runtime objects into a standardized, universally understood format that can travel across a network and be reconstructed by a completely different runtime environment.

2. Defining Serialization and Deserialization

To solve the language barrier between different runtime environments, backend engineering relies on two complementary processes: **Serialization** and **Deserialization**.



Serialization

Serialization is the process of converting an in-memory data structure or object (used by a specific programming language/runtime) into a standardized, flat format suitable for transmission over a network or storage in a persistent database.

Deserialization

Deserialization is the reverse process. It takes the standardized, flat format received from a network stream or storage medium and reconstructs it back into a native, structured in-memory object that the receiving programming language can understand and manipulate.

The Standard Format Solution

Rather than writing custom translators for every possible language pair (e.g., JS-to-Rust, Rust-to-Go, Go-to-Python), the industry uses **Language-Agnostic Common Standards**.

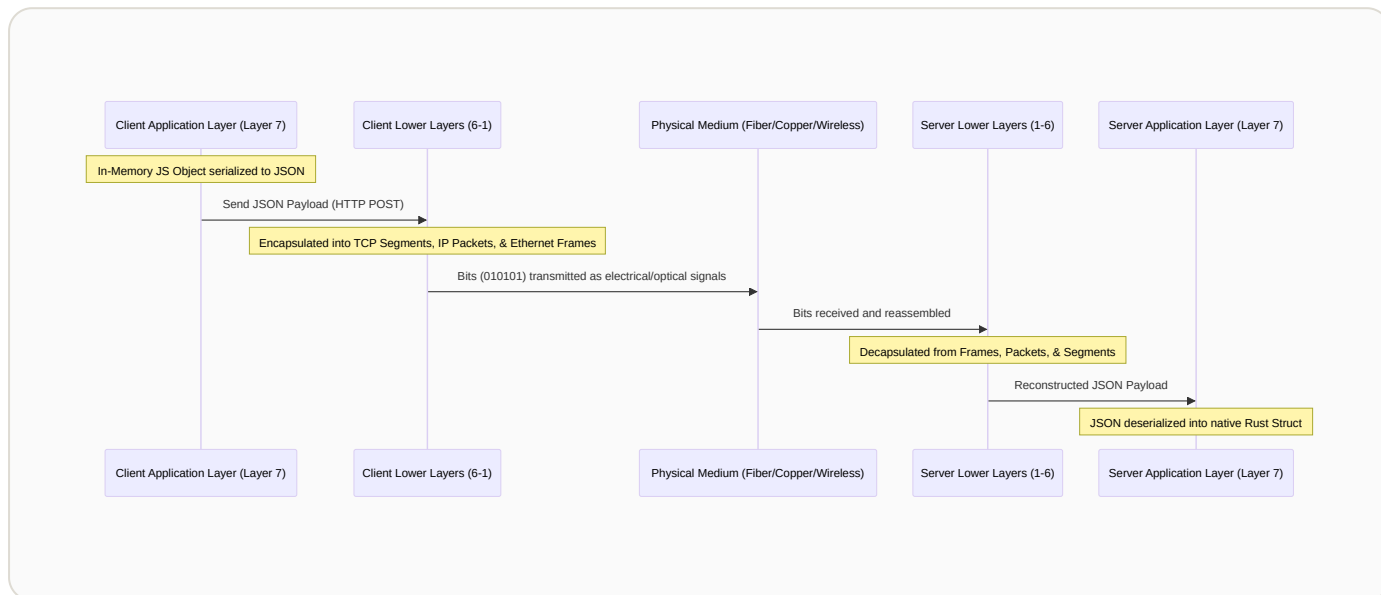
Both the client and the server agree to a predefined protocol. The client translates its language-specific data into this common standard before sending, and the server translates from this common standard back into its own language-

specific structures upon receipt.

3. The Network Perspective: The OSI Model Abstraction

To understand how serialized data travels across the internet, it is helpful to look at it through the lens of the **OSI (Open Systems Interconnection) Model**.

The OSI model divides network communications into seven logical layers, spanning from the physical hardware up to the user-facing application software.



The Backend Engineer's Mental Model

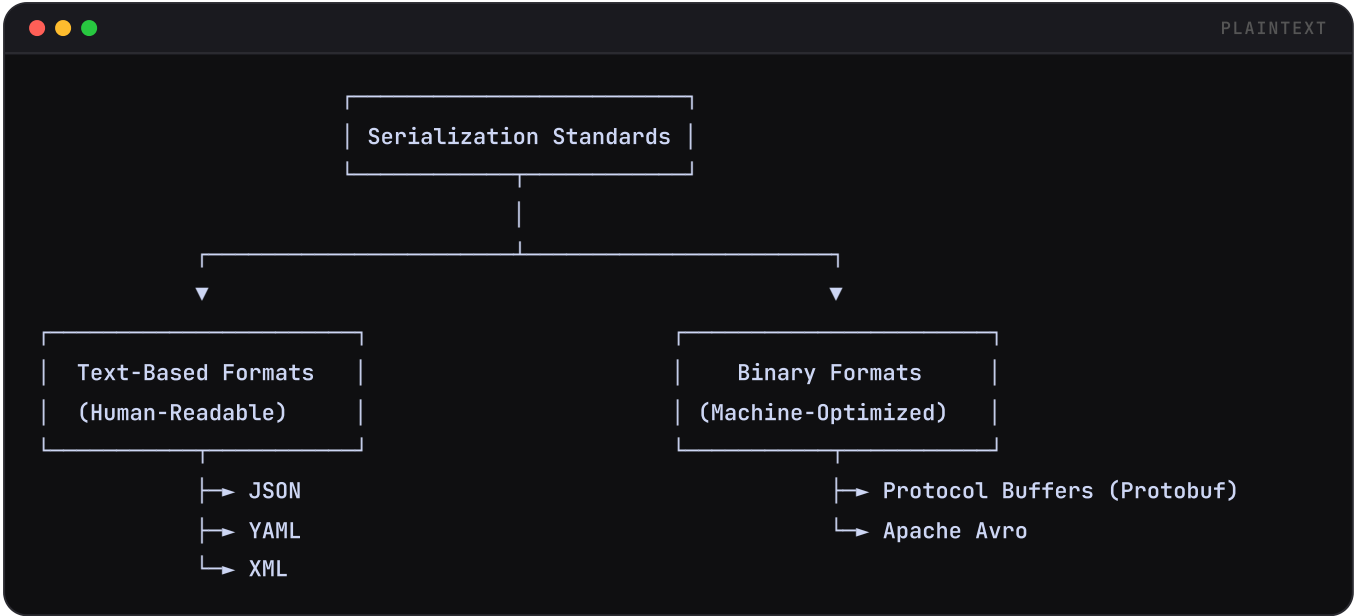
While network engineers focus on IP packets (Layer 3), TCP segments (Layer 4), and data frames (Layer 2), **backend software engineers primarily operate at Layer 7 (the Application Layer)**.

Your primary responsibility is ensuring that the data format leaving your application is standard-compliant, and that your application can parse incoming payloads at the application boundary. The lower-level translations—such as converting text into binary streams, dividing streams into packets, routing packets over the internet, and converting them to electrical voltages or light pulses over optical fibers—are abstracted away by the operating system's network stack and network hardware.

At the application layer:

4. Classification of Serialization Formats

Serialization standards are broadly divided into two categories: **Text-Based Formats** and **Binary Formats**.



Text-Based Formats

These formats serialize data into human-readable text strings. They are highly portable, easy to debug, and widely adopted.

Binary Formats

These formats serialize data into compact, non-human-readable binary streams (sequences of 0s and 1s). They are highly optimized for speed, minimal CPU overhead, and low network bandwidth usage.

Comparative Analysis

FEATURE	JSON	XML	YAML	PROTOBUF
Type	Text	Text	Text	Binary
Human-Readable	Yes	Yes	Yes	No
Payload Size	Medium	Large	Medium	Very Small
Parsing Speed	Fast	Slow	Slow	Extremely Fast
Schema Support	Optional (JSON Schema)	Strict (XSD)	No	Strict (Required)
Primary Use Case	Public APIs, Web Apps	Enterprise Systems	Configurations	Microservices, gRPC

5. Deep Dive: JSON (JavaScript Object Notation)

JSON is the most common serialization standard for HTTP-based web development.

Key Characteristics

Supported JSON Data Types

The values in a JSON structure can only be one of the following basic data types:

Anatomy of a Valid JSON Payload

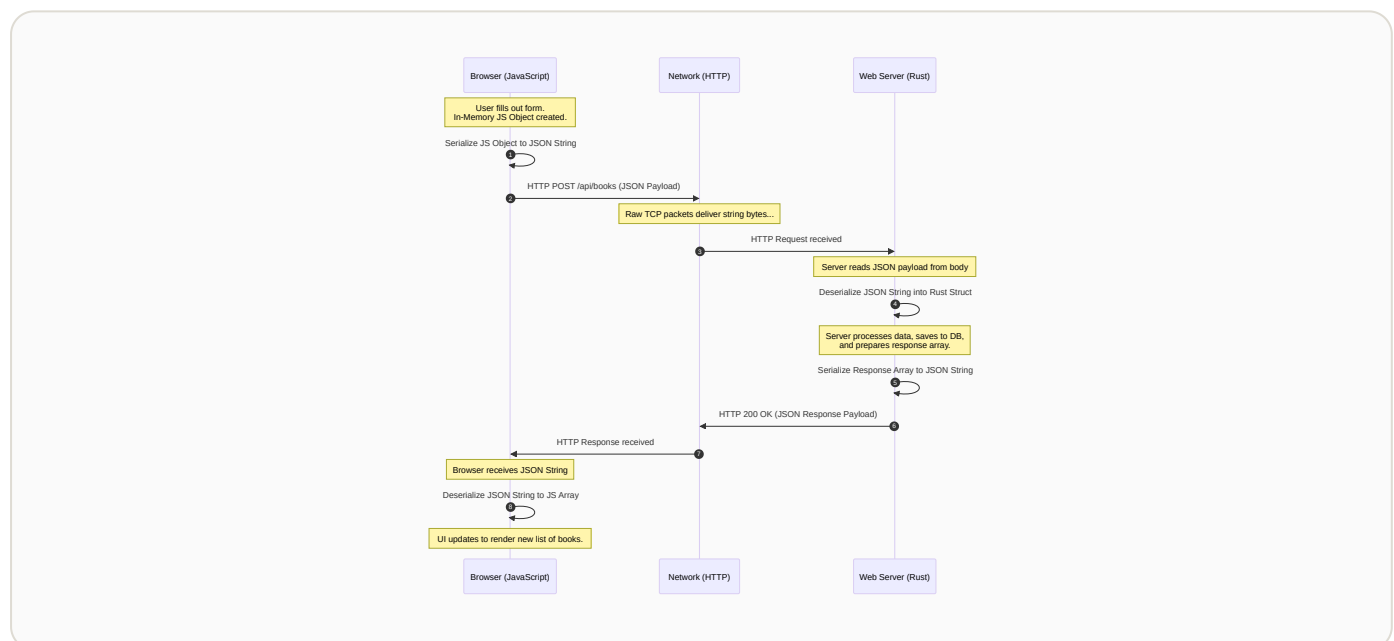
Below is a valid, nested JSON document demonstrating all supported data types:

```
{
  "id": 101,
  "title": "The Rust Programming Language",
  "available": true,
  "tags": ["programming", "systems", "rust"],
  "metadata": {
    "publisher": "No Starch Press",
    "pages": 552
  },
  "discount_code": null
}
```

6. End-to-End Walkthrough: The API Book-Store Example

To see serialization and deserialization in action, let us trace a real-world HTTP transaction where a client adds a new book to a database using a **POST** request.

The System Architecture



Detailed Execution Trace

STEP 1: CLIENT-SIDE SERIALIZATION

A user fills out a web form to add a book. The JavaScript runtime captures this data in an object:

JAVASCRIPT

```
// In-memory JavaScript object (Dynamic Runtime Representation)
const bookPayload = {
  id: 4,
  title: "Designing Data-Intensive Applications",
  author: "Martin Kleppmann"
};

// Serialization: Convert the object into a JSON string
const serializedData = JSON.stringify(bookPayload);
console.log(serializedData);
// Output: '{"id":4,"title":"Designing Data-Intensive Applications","author":"Martin Kleppmann"}'
```

STEP 2: THE NETWORK PAYLOAD (HTTP REQUEST)

The serialized string is placed into the body of an HTTP **POST** request.

HTTP

```
POST /api/books HTTP/1.1
Host: bookstore-api.com
Content-Type: application/json
Content-Length: 87

{"id":4,"title":"Designing Data-Intensive Applications","author":"Martin Kleppmann"}
```

STEP 3: SERVER-SIDE DESERIALIZATION

The Rust server receives the raw bytes of the HTTP request body. It cannot manipulate this raw text directly as a string if it wants to perform business logic. It must deserialize it into a native Rust **struct** using a serialization/deserialization library (such as **serde** in Rust).

RUST

```
// Rust Type Definition
#[derive(serde::Deserialize, serde::Serialize)]
struct Book {
  id: u32,
  title: String,
  author: String,
}

// Deserialization: Raw JSON string is parsed into a type-safe memory structure
let raw_body = r#"{"id":4,"title":"Designing Data-Intensive Applications","author":"Martin Kleppmann"}"#;
let book: Book = serde_json::from_str(raw_body).unwrap();

// The server can now safely interact with the data
println!("Adding book: {} by {}", book.title, book.author);
```

STEP 4: SERVER RESPONSE GENERATION

The server processes the request, saves the record to a database, retrieves the updated list of books, and prepares to send it back. It must serialize the updated list before sending it.

```

// Rust In-Memory Vector (Array) of Books
let response_data = vec![
    Book { id: 1, title: "The Rust Programming Language".to_string(), author: "Steve Klabnik".to_string() },
    Book { id: 4, title: "Designing Data-Intensive Applications".to_string(), author: "Martin Kleppmann".to_string() }
];

// Serialization
let serialized_response = serde_json::to_string(&response_data).unwrap();

```

STEP 5: THE CLIENT RECEIVES AND DESERIALIZES

The client browser receives the HTTP response containing the serialized JSON string of the book list.

```

HTTP/1.1 200 OK
Content-Type: application/json
Content-Length: 198

[{"id":1,"title":"The Rust Programming Language","author":"Steve Klabnik"},
{"id":4,"title":"Designing Data-Intensive Applications","author":"Martin Kleppmann"}]

```

The browser deserializes this response to update its internal state and render the new UI:

```

// Browser receives the response payload
const rawResponse = '[{"id":1,"title":"The Rust Programming Language","author":"Steve Klabnik"},
{"id":4,"title":"Designing Data-Intensive Applications","author":"Martin Kleppmann"}]';

// Deserialization
const booksArray = JSON.parse(rawResponse);

// The dynamic array is now ready to be mapped to DOM elements
booksArray.forEach(book => {
    console.log(`Rendered: ${book.title}`);
});

```

7. Summary and Key Takeaways

KEY TAKEAWAYS

- Serialization and Deserialization allow applications running on different platforms, operating systems, and programming languages to exchange structured data over a network or save it to a storage medium.
- These processes rely on an agreed-upon, common standard (such as JSON or Protobuf) to bridge the gap between different runtime type systems (e.g., dynamic JavaScript engines and strict Rust compilers).
- JSON is the most widely used serialization standard for web APIs due to its human-readability, simplicity, and native compatibility with JavaScript.
- While low-level network layers (OSI Layers 1–6) translate payloads into packets and bits, backend engineers primarily work at the Application Layer (Layer 7), focusing on the serialization and deserialization boundaries of their application.

CH
08

Lecture 8: 8. Authentication and authorization for backend engineers

LECTURE 8: AUTHENTICATION AND AUTHORIZATION FOR BACKEND ENGINEERS

Module 1: Foundational Paradigms of Identity

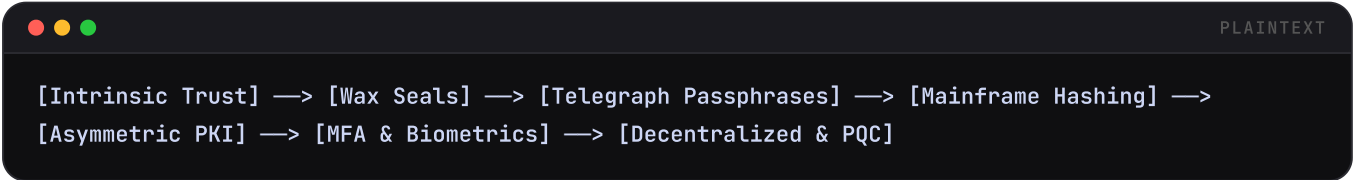
At the core of any secure application architecture lie two distinct, yet deeply interconnected, security processes: **Authentication (AuthN)** and **Authorization (AuthZ)**. For backend engineers, conflating these concepts is a critical architectural error. They address entirely different questions within an application's execution context.



- Authentication (AuthN)
- Authorization (AuthZ)

Module 2: The Historical Evolution of Authentication

The mechanisms we use to verify identity have evolved alongside societal complexity and technological capability. Understanding this trajectory reveals the trade-offs built into modern protocols.



1. Pre-Industrial Era: Intrinsic Trust
2. Medieval Era: Explicit Proofs & Wax Seals
3. Industrial Revolution: Telegraphy & Shared Secrets
4. Early Computing: Mainframes & Project MAC (1961)

5. 1970s: Asymmetric Cryptography & Kerberos

6. 1990s: Multi-Factor Authentication (MFA)

7. Modern Era & Future Horizons

Module 3: Core Cryptographic & Web Components

To design secure backend systems, engineers must understand the mechanics and trade-offs of three fundamental web mechanisms: **Sessions**, **JSON Web Tokens (JWTs)**, and **Cookies**.

1. Sessions (Stateful Architecture)

Historically, the Hypertext Transfer Protocol (HTTP) was designed to be **stateless**. It treats every request as an isolated transaction, holding no memory of past interactions. This design was ideal for serving static documents but became a bottleneck as the web transitioned to dynamic applications (such as e-commerce shopping carts and user logins). To solve this, developers introduced **Sessions**, establishing a stateful layer on top of stateless HTTP.



Syntax error in text
mermaid version 10.9.6

SESSION STORAGE EVOLUTION

2. JSON Web Tokens (JWTs) (Stateless Architecture)

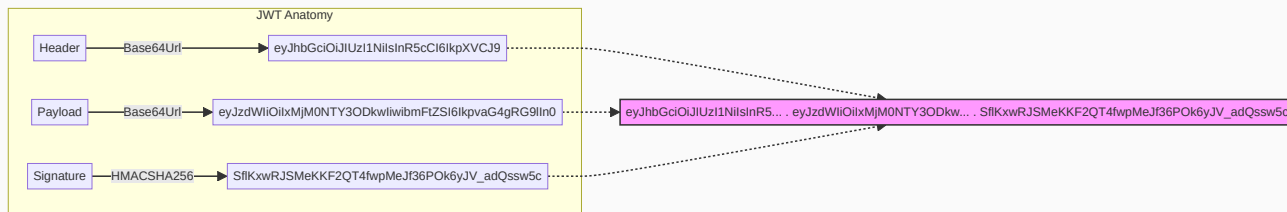
As systems grew into globally distributed microservices, synchronizing session state across geographic regions introduced latency and consistency challenges. Formalized in 2015 (RFC 7519), **JSON Web Tokens** solved this by offloading session state from the server to the client.

A JWT is a self-contained, cryptographically signed, Base64Url-encoded string consisting of three distinct parts separated by dots (.):



PLAINTEXT

```
[Header].[Payload].[Signature]
```

DETAILED BREAKDOWN OF JWT STRUCTURE

A. HEADER (METADATA)

Specifies the token type and the signing algorithm used (e.g., HMAC SHA256 [**HS256**] or RSA [**RS256**]).

```
{
  "alg": "HS256",
  "typ": "JWT"
}
```

B. PAYLOAD (CLAIMS)

Contains the actual statements (claims) about the subject and any supplementary metadata. Standard claims include:

```
{
  "sub": "usr_9082314",
  "iat": 1710000000,
  "exp": 1710000900,
  "role": "editor",
  "name": "Jane Doe"
}
```

C. SIGNATURE

Used to verify that the sender of the JWT is who it claims to be and to ensure that the message was not tampered with along the way. The signature is calculated by taking the encoded header, the encoded payload, a secret key, and signing them using the algorithm specified in the header.

JAVASCRIPT

```
// Conceptual signature generation
const signature = HMAC_SHA256(
  base64UrlEncode(header) + "." + base64UrlEncode(payload),
  server_secret_key
);
```

THE STATELESS ADVANTAGE

When a service receives a JWT, it does not query a database. It decodes the token, verifies the signature using its copy of the secret key (or the public key of the identity provider), and checks the `exp` claim. If the signature is valid and the token is not expired, the backend trusts the claims inside the payload.

THE REVOCATION PROBLEM & THE HYBRID MITIGATION

Because JWT verification is stateless, **revoking a token before its natural expiration is difficult**. If an attacker steals a user's JWT, they can impersonate that user until the token expires.

To mitigate this while maintaining performance, backends often implement a **Hybrid Approach**:

3. Cookies

A **Cookie** is not an alternative to sessions or JWTs; it is a **storage and transmission mechanism** built into the HTTP specification and managed by web browsers.

PLAINTEXT

```
HTTP REQUEST
Browser =====> Server
<=====
Set-Cookie: token=xyz
(Cookie stored by browser)

HTTP REQUEST
Browser =====> Server
(Automatically appends: Cookie: token=xyz)
```

KEY SECURITY DIRECTIVES FOR COOKIES

When transmitting sensitive tokens (session IDs or JWTs) via cookies, you must set these security flags in the `Set-Cookie` header:

Module 4: Authentication Paradigms & Architectures

Backend engineers must choose the appropriate authentication paradigm based on security requirements, system complexity, and client types.

Authentication Type	State Location	Verification Mechanism	Primary Use Case
Stateful (Session)	Server-side (RAM/Redis)	Database/Cache Lookup	Traditional Web Apps, Monoliths
Stateless (JWT)	Client-side (Cookie/Storage)	Cryptographic Verification	Microservices, Mobile Apps, APIs
API Keys	Server-side Database	Database Lookup / Prefix Hashing	Machine-to-Machine, Developer APIs
OAuth 2.0 / OIDC	Decentralized / Federated	Cryptographic Token Verification	Third-Party Auth, Single Sign-On (SSO)

1. Stateful Authentication

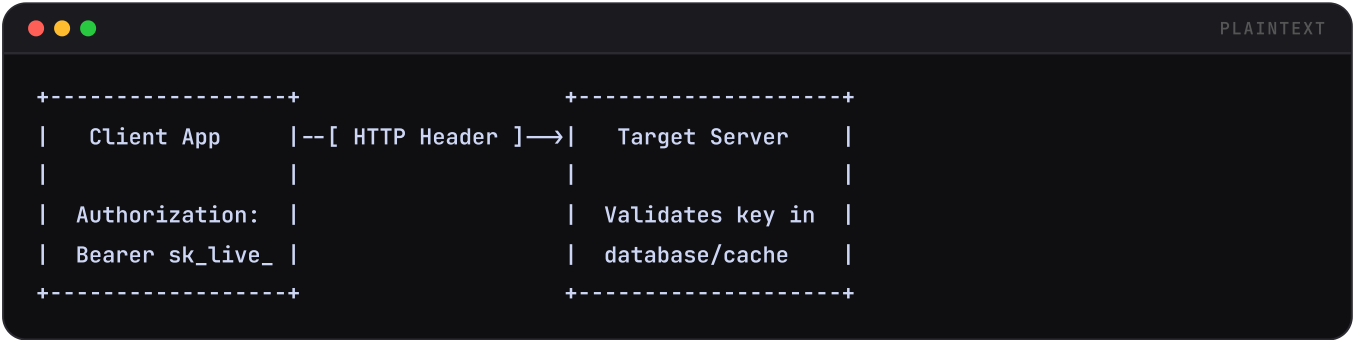
The server maintains the authoritative record of active sessions.

2. Stateless Authentication

The client stores the state in a cryptographically signed token (JWT).

3. API Key-Based Authentication

API keys are designed for **programmatic, machine-to-machine (M2M) communication** (e.g., a backend script integrating with OpenAI's API).

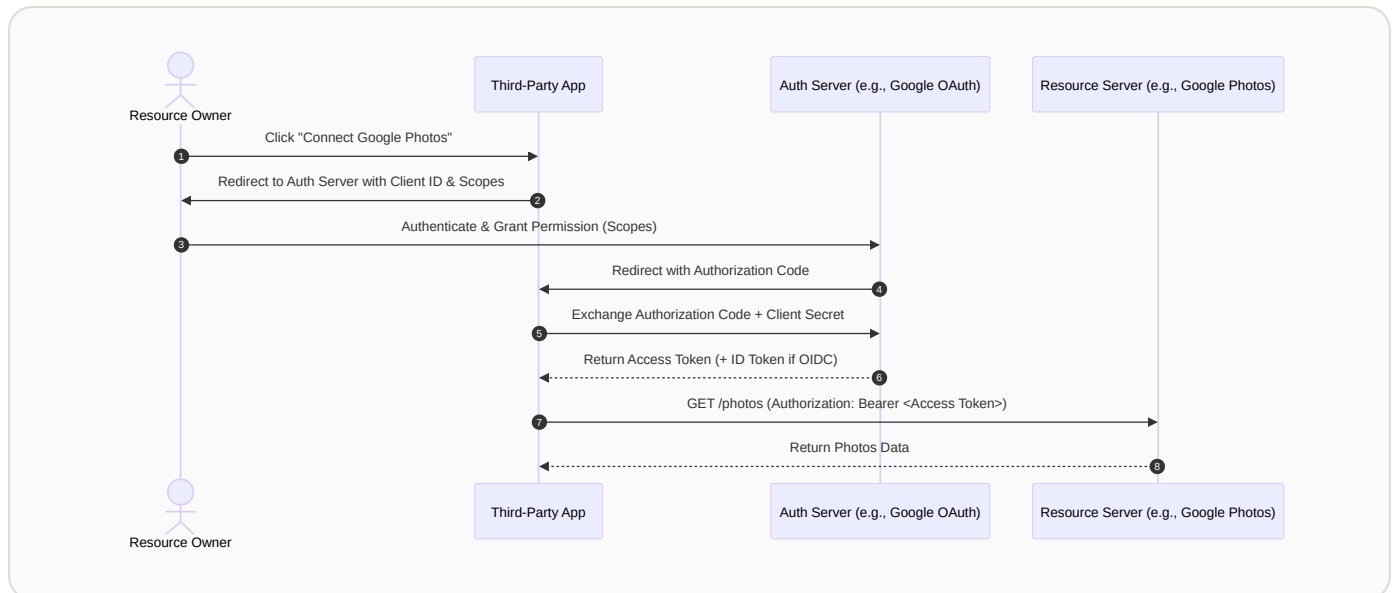


4. OAuth 2.0 & OpenID Connect (OIDC)

THE DELEGATION PROBLEM

Historically, if a travel application needed to scan your emails for flight tickets, you had to share your email password directly with the travel app. This anti-pattern had severe security flaws:

OAuth 2.0 (RFC 6749) solved this delegation problem by introducing **Access Tokens** instead of password sharing.



CORE COMPONENTS OF OAUTH 2.0

THE AUTHORIZATION CODE FLOW

This is the most secure flow for web and mobile applications:

OPENID CONNECT (OIDC): ADDING IDENTITY TO AUTHORIZATION

OAuth 2.0 is strictly an **authorization framework** (it issues access tokens but does not tell the client *who* the user is). To solve this, **OpenID Connect (OIDC)** was built on top of OAuth 2.0 in 2014.

OIDC introduces an **ID Token** (always a JWT) alongside the Access Token. This ID Token contains standardized claims about the user's identity (such as **email**, **name**, and **sub**), enabling federated login flows like *"Sign in with Google"*.

Module 5: Authorization & Access Control Models

Once a user's identity is authenticated, the backend must enforce authorization policies. The industry standard for modern application development is **Role-Based Access Control (RBAC)**.

PLAINTEXT

```
[User: Alice] —> Assigned Role —> [Role: Moderator] —> Granted Permissions —> [Perm:
delete_notes]
                                     —> [Perm:
read_notes]
```

Role-Based Access Control (RBAC)

In RBAC, permissions are not assigned directly to users. Instead, permissions are grouped into **Roles**, and roles are assigned to **Users**. This separation simplifies administration.

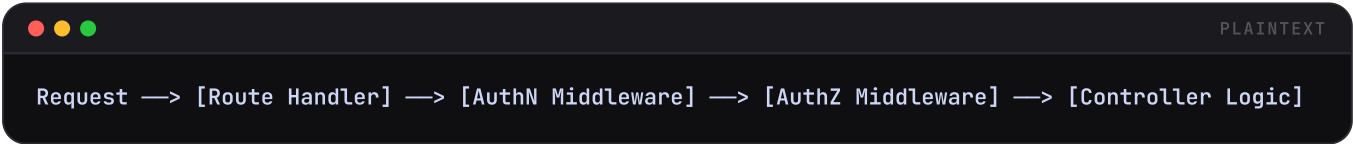
DATABASE SCHEMA DESIGN FOR RBAC

A standard relational schema uses a many-to-many relationship mapping:



Backend Enforcement Patterns: Middleware

In modern web frameworks, authorization is enforced early in the request lifecycle using a **Middleware Chain**.



Here is a conceptual implementation of an RBAC validation middleware in Node.js/Express:

JAVASCRIPT

```
// Authentication Middleware: Verifies Identity
async function authenticate(req, res, next) {
  const authHeader = req.headers.authorization;
  if (!authHeader || !authHeader.startsWith('Bearer ')) {
    return res.status(401).json({ error: 'Authentication required' });
  }

  const token = authHeader.split(' ')[1];
  try {
    // Decode and verify the JWT
    const decoded = jwt.verify(token, process.env.JWT_SECRET);

    // Attach user identity metadata directly to the request object
    req.user = {
      id: decoded.sub,
      role: decoded.role // e.g., 'moderator'
    };
    next();
  } catch (err) {
    return res.status(401).json({ error: 'Invalid or expired token' });
  }
}

// Authorization Middleware Builder: Verifies Permissions
function authorize(allowedRoles = []) {
  return (req, res, next) => {
    // Ensure user is authenticated first
    if (!req.user) {
      return res.status(401).json({ error: 'Security context missing' });
    }

    // Evaluate if the user's role meets the requirement
    if (!allowedRoles.includes(req.user.role)) {
      // Return 403 Forbidden to indicate insufficient privileges
      return res.status(403).json({ error: 'Access denied: insufficient permissions' });
    }

    next();
  };
}

// Route Application
app.delete('/api/v1/notes/:id', authenticate, authorize(['admin', 'moderator']), async (req, res) => {
  // Controller logic to delete note...
  res.status(200).json({ message: 'Resource successfully deleted' });
});
```

Module 6: Defensive Backend Security

Backend engineers must design systems defensively, assuming attackers will actively probe authentication endpoints for vulnerabilities.

1. Generic Error Message Design

A common vulnerability in authentication endpoints is **Information Leakage** through overly specific error messages.

PLAINTEXT	
VULNERABLE PATHWAY	DEFENSIVE PATHWAY
POST /login { "email": "nonexistent@user.com" }	POST /login { "email": "nonexistent@user.com" }
<— 404 Not Found "User does not exist!" (Attacker learns email is invalid)	<— 401 Unauthorized "Invalid credentials" (Attacker learns nothing)
POST /login { "email": "valid@user.com", "pass": "x" }	POST /login { "email": "valid@user.com", "pass": "x" }
<— 401 Unauthorized "Incorrect password!" (Attacker confirms email is valid)	<— 401 Unauthorized "Invalid credentials" (Attacker learns nothing)

THE VULNERABILITY

If your endpoint returns *"User not found"* or *"Incorrect password"*, an attacker can use **Username Enumeration** to test a list of email addresses and identify registered users on your platform. Once they have a list of valid emails, they can launch targeted credential-stuffing or brute-force attacks.

THE MITIGATION

Always return a generic error message (e.g., *"Invalid email or password"* or *"Invalid credentials"*) with a consistent HTTP status code (typically **401 Unauthorized**), regardless of whether the email was missing or only the password was wrong.

2. Timing Attacks & Mitigations

A **Timing Attack** is a side-channel attack where an attacker analyzes the time a server takes to respond to different inputs to infer internal state or validate usernames.

PLAINTEXT	
[Attacker] → POST /login (non-existent email) → Fast DB lookup fails → [Server] → Response: 40ms	
[Attacker] → POST /login (valid email, bad pass) → DB succeeds → CPU-intensive hash → [Server] → Response: 280ms	

THE MECHANICS OF AN AUTH TIMING ATTACK

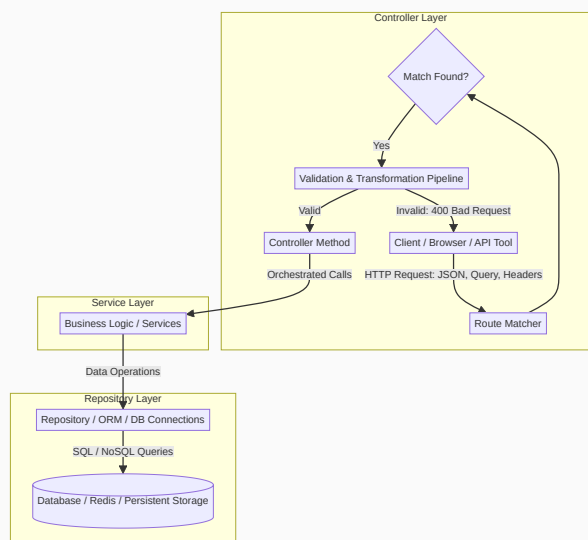
Consider the typical backend logic for a login endpoint:

Lecture 9: 9. Validations and transformations for backend engineers

LECTURE 9: VALIDATIONS AND TRANSFORMATIONS FOR BACKEND ENGINEERS

1. Architectural Placement of Validations and Transformations

To understand why and where validations and transformations occur, we must first examine the multi-layered architecture of modern backend applications. A standard backend system is structured with a clear separation of concerns, divided into distinct layers of execution.



The Layered Architecture

Why Separate the Controller and Service Layers?

Keeping HTTP-specific elements out of the service layer ensures **high testability and reusability**. The service layer should remain transport-agnostic; it should not care whether a business command was triggered by an HTTP POST request, a gRPC call, a CLI command, or a message queue consumer.

The Exact Point of Validation

Validations and transformations occur **at the outermost boundary of the Controller Layer**, immediately after the routing engine matches a request to its respective controller but **before any business logic or service layer code executes**.

This boundary is typically implemented as a global or route-specific **middleware** or execution pipeline.

2. The Core Philosophy of Validation: Fail-Fast & Data Integrity

The primary objective of validation is to protect downstream system layers from dirty, malformed, or malicious data.

The Problem: Propagation of Bad Data

Without an early validation pipeline, invalid data penetrates deep into the application, leading to unpredictable failures.



Syntax error in text mermaid version 10.9.6

In this scenario:

Implications of Database-Level Failures

The Solution: Fail-Fast with `400 Bad Request`

By validating input at the entry point, the server can immediately reject the request with a `400 Bad Request` status code and a structured payload explaining exactly what failed.

```
// Example of a structured 400 Bad Request response payload
{
  "status": "fail",
  "errors": [
    {
      "field": "name",
      "message": "Expected type string, but received number."
    }
  ]
}
```

3. Categories of Validation

Validations are grouped into distinct tactical layers based on the complexity of the rules they enforce.

Validation Type	Focus Area	Common Examples
Type Validation	Primitive data types and structural shapes	Checking if a field is a <code>string</code> , <code>number</code> , <code>boolean</code> , or <code>array</code> .
Syntactic Validation	Pattern matching and format compliance	Checking email patterns, phone formats, or ISO date structures.
Semantic Validation	Logical correctness of values within context	Ensuring date of birth is in the past, or age is within human limits.
Complex / Conditional	Multi-field cross-validation	Matching passwords, or requiring field B only if field A is true.

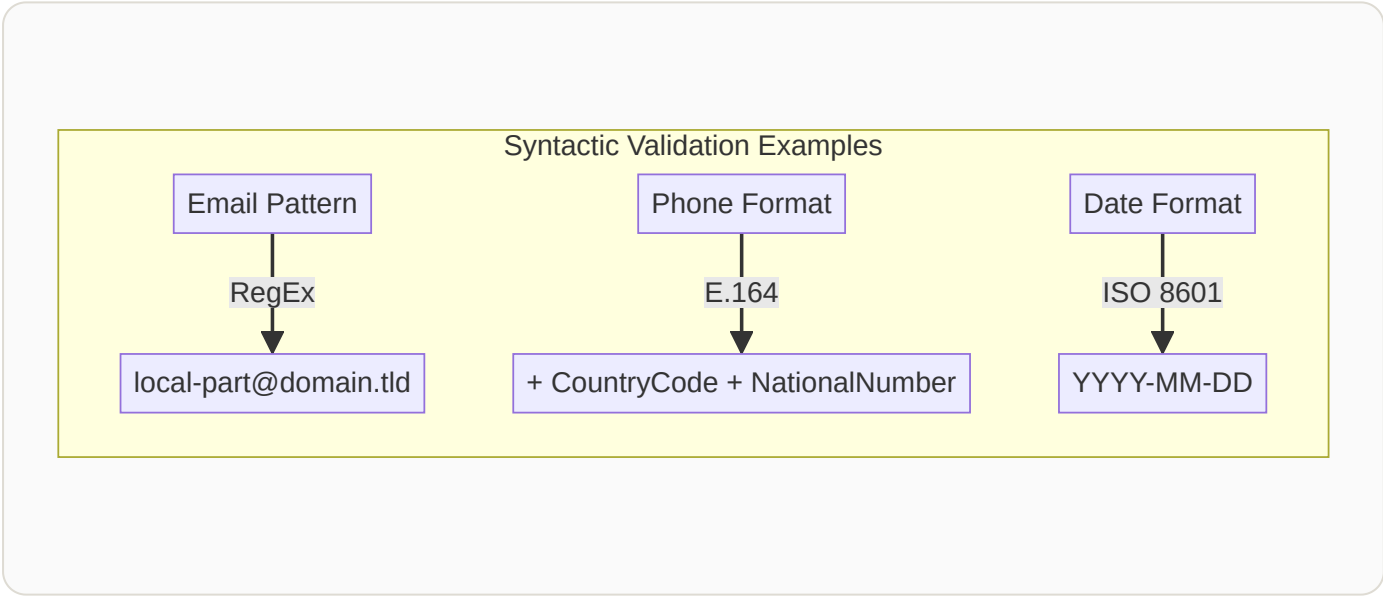
A. Type Validation

Type validation ensures that the incoming data matches the primitive types defined in your API contract. It is the first line of defense against structural mismatches.

```
// Expected:
{ "tags": ["tech", "learning"] }
// Rejected:
{ "tags": "tech, learning" } // Rejected: Expected array, received string.
```

B. Syntactic Validation

Syntactic validation verifies that a value—while matching its primitive type (usually a string)—conforms to a strict structural pattern.



C. Semantic Validation

Semantic validation evaluates whether the values make logical sense in a real-world context.

D. Complex and Conditional Validation

Complex validations require analyzing relationships between multiple fields in the payload.

Ensuring `password` matches `password_confirmation`.

Suppose we have a field `married` (boolean). If `married` is `true`, then a secondary field `partner_name` (string) becomes strictly required. If `married` is `false`, `partner_name` must be absent or ignored.

4. Data Transformations and Type Casting

Validation and transformation form a unified pipeline because data often needs modification before its validity can be assessed.



A. Type Casting (The Query Parameter Problem)

By default, all query parameters and URL path variables are parsed as strings when they reach an HTTP server.

Consider a paginated API route: `GET /api/bookmarks?page=2&limit=20`

When parsed by the server engine:

If your validation schema expects strict integers, validating these values directly will fail. The transformation pipeline must intercept these strings and cast them into numeric primitives before validation runs:

`String: "2" → Casting / Transformation Integer: 2`

Once cast, the semantic validation step can safely check if `page > 0` and `limit ≤ 100`.

B. Data Normalization and Sanitization

Transformation also modifies valid input data into standard internal formats to ensure data consistency across your database.

5. Frontend vs. Backend Validation: The Security Boundary

A common architectural mistake is relying on frontend validation to ensure backend data integrity.



Comparing Frontend and Backend Validation

DIMENSION	FRONTEND VALIDATION	BACKEND VALIDATION
Primary Purpose	User Experience (UX)	Security & Data Integrity
Trust Level	Untrusted (Fully under client control)	Trusted (Fully under server control)
Bypassability	Extremely easy to bypass	Cannot be bypassed if correctly configured
Execution Context	Browser / Client Device	Server
Feedback Loop	Instant, real-time	Network round-trip

Why Frontend Validation Is Not a Security Boundary

Any code executing on the client device can be modified, disabled, or bypassed entirely. A user can easily bypass frontend constraints by:

Therefore, backend validation must treat every incoming request as **potentially malicious**. While client-side validation is essential for providing real-time feedback, backend validation is your system's ultimate security boundary.

6. Summary: Key Guidelines for API Design

To build robust, secure, and developer-friendly APIs, keep these rules in mind when designing your validation pipelines:

Lecture 10: 10. What are controllers, services, repositories, middlewares and request context?

LECTURE 10: ARCHITECTURAL LAYERS (CONTROLLERS, SERVICES, REPOSITORIES), MIDDLEWARES, AND REQUEST CONTEXT

1. Introduction: The Internal Request Life Cycle

When a client initiates an HTTP request to a server, the interaction spans a well-defined **Request Life Cycle**. While external life cycles deal with DNS resolution, TCP handshakes, and network transit, the **internal life cycle** begins the moment the operating system's network stack receives the raw TCP packets on a designated port (e.g., 3000, 4000, 80) and forwards them to the running server application.

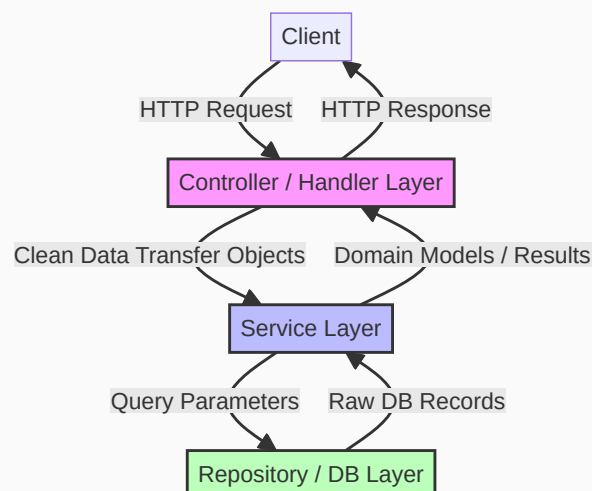
Inside the server, the raw request must be routed, authenticated, parsed, validated, processed, and responded to. To build highly scalable, maintainable, and testable backend systems, engineers use structured design patterns to divide these tasks.

This document explores the core pillars of modern backend architecture:

2. The Three-Tier Architectural Pattern

While it is technically possible to write all database queries, validation logic, and HTTP response formatting inside a single handler function, doing so creates a highly coupled codebase (often referred to as "spaghetti code") that is difficult to scale, test, or debug.

The **Three-Tier Pattern** separates concerns into three distinct layers:



Layer 1: The Controller (Handler) Layer

The **Controller** (or **Handler**) acts as the entry point for a routed HTTP request. It is directly aware of the HTTP protocol and is responsible for managing the communication boundary between the client and the server.

RESPONSIBILITIES

Layer 2: The Service Layer

The **Service Layer** is the core of your application. It contains the **business logic** and domain rules.

KEY PRINCIPLE: HTTP AGNOSTICISM

An essential design pattern of a clean Service layer is that **it must have no knowledge of HTTP**.

If you look at a Service method, you should not be able to tell if it is being triggered by an HTTP REST API, a gRPC call, a CLI command, or a background message queue consumer. This isolation makes your business logic highly reusable and easy to unit test.

RESPONSIBILITIES

Layer 3: The Repository Layer

The **Repository Layer** (or **Data Access Layer**) acts as an abstraction over the database or persistent storage engine.

RESPONSIBILITIES

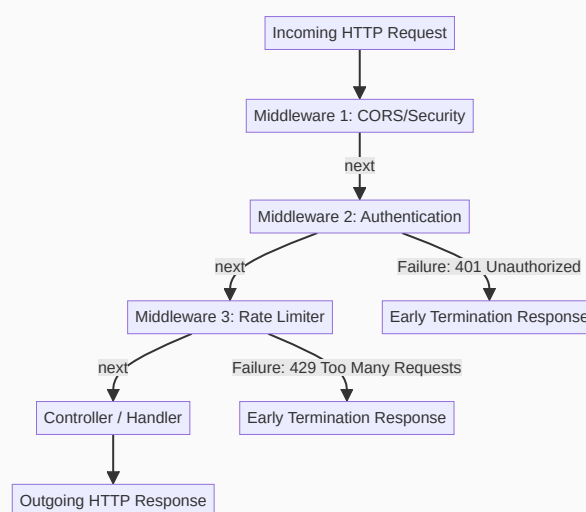
KEY PRINCIPLE: SINGLE RESPONSIBILITY PER METHOD

Repository methods should be highly granular and perform exactly one database operation.

3. Deep Dive into Middlewares

What is a Middleware?

A **Middleware** is a specialized function that sits in the middle of the request-response pipeline. It intercept incoming requests before they reach the controller, and can also intercept outgoing responses before they are sent back to the client.



The Middleware Signature

In almost all modern frameworks (such as Express in Node.js, Gin in Go, or FastAPI dependencies in Python), a middleware function receives three parameters:

If a middleware does not call `next()`, the request-response cycle is halted, and the request is prevented from reaching downstream middlewares or controllers. This is known as **early termination**.

Why Use Middlewares? (The DRY Principle)

Middlewares prevent code duplication by centralizing cross-cutting concerns. Instead of copying and pasting logging, security, and authentication checks into hundreds of different controllers, these operations are delegated to reusable middleware components.

EARLY RESOURCE PROTECTION

By executing checks (like rate limiting and authentication) at the very beginning of the internal request life cycle, the server can reject bad requests immediately. This saves CPU cycles, memory, and database connection pools from being wasted on unauthorized or malicious traffic.

Critical Production Middleware Patterns

MIDDLEWARE TYPE	PRIMARY RESPONSIBILITY	FAILURE OUTCOME
CORS (Cross-Origin Resource Sharing)	Inspects the <code>Origin</code> header and appends appropriate headers (<code>Access-Control-Allow-Origin</code>) to inform browsers whether to permit the cross-origin request.	Browser blocks the response on the client side.
Security Headers	Injects HTTP security headers like Content Security Policy (<code>CSP</code>), <code>X-Frame-Options</code> (prevents clickjacking), and <code>Strict-Transport-Security</code> (<code>HSTS</code>).	N/A (Enhances client-side security).
Authentication & Authorization	Extracts tokens (e.g., Bearer JWT or Session Cookie), validates them, resolves user identity, and registers user metadata into the request context.	Returns <code>401 Unauthorized</code> or <code>403 Forbidden</code> and terminates the request.
Rate Limiting	Tracks incoming requests per IP or User ID using fast in-memory caches (like Redis) to prevent abuse and brute-force attacks.	Returns <code>429 Too Many Requests</code> and terminates.
Logging & Monitoring	Records incoming request metrics (HTTP method, path, query parameters) and tracks execution time for performance audits.	N/A (Purely observational).
Global Error Handling	Catches unhandled exceptions or thrown errors from downstream layers, structures them cleanly, and prevents system crash stack traces from leaking to clients.	Captures internal failures and returns a structured JSON payload with a <code>500 Internal Server Error</code> code.
Compression	Compresses outgoing response bodies using algorithms like Gzip or Brotli to reduce payload transit times over the network.	N/A (Optimizes bandwidth).

The Importance of Middleware Ordering

Because middlewares execute sequentially, **their order of registration is critical**.



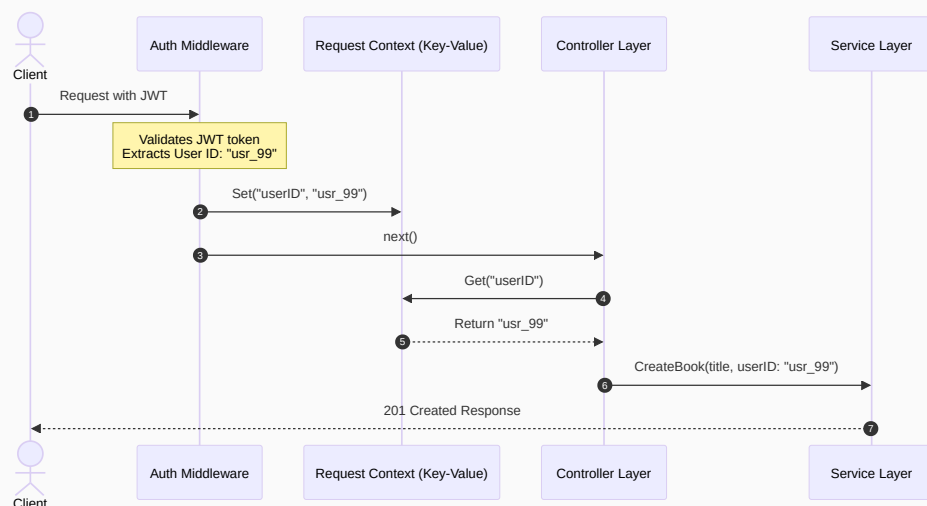
Syntax error in text mermaid version 10.9.6

4. Request Context: Scoped State Management

What is Request Context?

A **Request Context** is a thread-safe data store or state container that is created at the start of an HTTP request and destroyed when the response is sent back to the client.

Because a backend server processes thousands of requests concurrently, global variables cannot be used to store request-specific information. The request context acts as an isolated, request-scoped dictionary (a key-value store) that travels alongside the request across all middleware, controller, and service boundaries.



Real-World Production Patterns

1. DECOUPLED AUTHENTICATION METADATA (RBAC)

When an authentication middleware successfully decodes a JWT, it extracts metadata such as the `userID`, `role` (e.g., `admin`, `user`), and permission scopes.

```
$$\text{Context.Set("userID", decodedToken.sub)}$$$
```

```
$$\text{Context.Set("role", decodedToken.role)}$$$
```


2. REQUEST TRACING AND OBSERVABILITY (CORRELATION IDS)

In distributed microservice architectures, debugging a single request that spans multiple services can be extremely difficult.

3. CONTEXT CANCELLATION, DEADLINES, AND TIMEOUTS

In languages like Go, the request context (`context.Context`) is also used to manage resource lifecycles.

Avoiding Security Vulnerabilities (Spoofing Prevention)

Request context is vital for preventing ID spoofing vulnerabilities.

Consider a request to create a resource: `POST /books` with the payload `{"title": "Clean Code", "userID": "123"}`.

5. Architectural Summary Reference

COMPONENT	HTTP AWARE?	PRIMARY INPUTS	PRIMARY OUTPUTS	KEY DESIGN PRINCIPLE / PATTERN
Controller (Handler)	Yes	<code>Request</code> , <code>Response</code> objects	Formatted HTTP response (JSON, HTML)	Single Responsibility: Focus on serialization, validation, and HTTP mapping.
Service	No	Clean, native data types	Native data structures, domain errors	Business Logic Isolation: Must remain completely agnostic of transport protocols (HTTP, gRPC).
Repository	No	Query parameters, native models	Database entities, query errors	Single Responsibility: One query per method; abstracts database technology.
Middleware	Yes	<code>Request</code> , <code>Response</code> , <code>Next()</code>	Modified request/response, or early exit	DRY (Don't Repeat Yourself): Centralizes cross-cutting concerns; execution order matters.
Request Context	No	Key-Value pairs, deadlines	Scoped request-specific state	Thread-Safety & Isolation: Manages request-scoped data securely without global variables.

Lecture 11: 11. Complete REST API Design

LECTURE 11: COMPLETE REST API DESIGN

1. Introduction: The Evolution of API Design

In modern backend engineering, API design represents the bridge between raw data storage and client consumption. Historically, the internet relied heavily on **Multi-Page Applications (MPAs)**, where the server was responsible for both database interaction and rendering the user interface (HTML).

With the advent of **Single-Page Applications (SPAs)** (using frameworks like React, Vue, or Svelte) and native mobile applications, the backend's role shifted. Today, servers primarily expose raw data and business logic through structured API endpoints, leaving presentation and routing to client-side runtimes.



Despite decades of industry evolution, backend developers still struggle with fundamental API design questions:

The goal of REST (Representational State Transfer) is not to introduce rigid, arbitrary constraints, but to **standardize communication interfaces**. A standardized API reduces developer friction, eliminates guesswork, and allows teams to focus on core business logic rather than debating design patterns.

2. History of the Web and the Scalability Crisis

To understand why REST is structured the way it is, we must look at the history of the internet.

The Genesis (1990)

In 1990, **Sir Tim Berners-Lee** initiated the Worldwide Web project at CERN to facilitate global information sharing. Within a brief window, he invented the foundational pillars of our current digital infrastructure:

The Scalability Crisis (1993–1996)

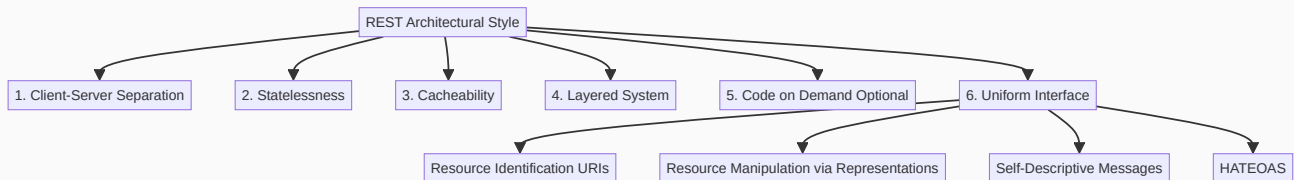
As the web's user base grew exponentially, the early architecture began to buckle. The original design did not account for millions of concurrent users, massive data transfer, or persistent network connections.

In 1993, **Roy Fielding** (co-founder of the Apache HTTP Server project) and other web pioneers became deeply concerned with the web's scalability. To prevent a complete architectural breakdown, Fielding proposed a set of architectural constraints designed to preserve the web's decentralized, scalable nature.

These constraints formed the basis of **HTTP 1.1**, which Fielding co-authored alongside Tim Berners-Lee. In 2000, Fielding formally named and described this architectural style in his seminal PhD dissertation: **Representational State Transfer (REST)**.

3. The Six Constraints of REST

An API is not "RESTful" simply because it runs over HTTP and returns JSON. To be truly RESTful, an architecture must adhere to the six core constraints defined by Roy Fielding:



1. Client-Server Separation

This constraint enforces a strict separation of concerns. The client handles the user interface and user experience, while the server manages data storage, security, and business logic. This decoupling allows both systems to evolve independently.

2. Statelessness

Every request from a client to a server must contain all the information necessary to understand and process the request. The server must not store any session state about the client on its side.

3. Cacheability

Responses from the server must explicitly label themselves as cacheable or non-cacheable. When a response is cacheable, the client or intermediate proxies can reuse that response data for subsequent equivalent requests, reducing latency and server load.

4. Layered System

A client cannot ordinarily tell whether it is connected directly to the end server or to an intermediary (such as a load balancer, reverse proxy, or CDN). Intermediate layers can improve security, coordinate load balancing, and provide shared caches without requiring any code changes on the client.

5. Code on Demand (Optional)

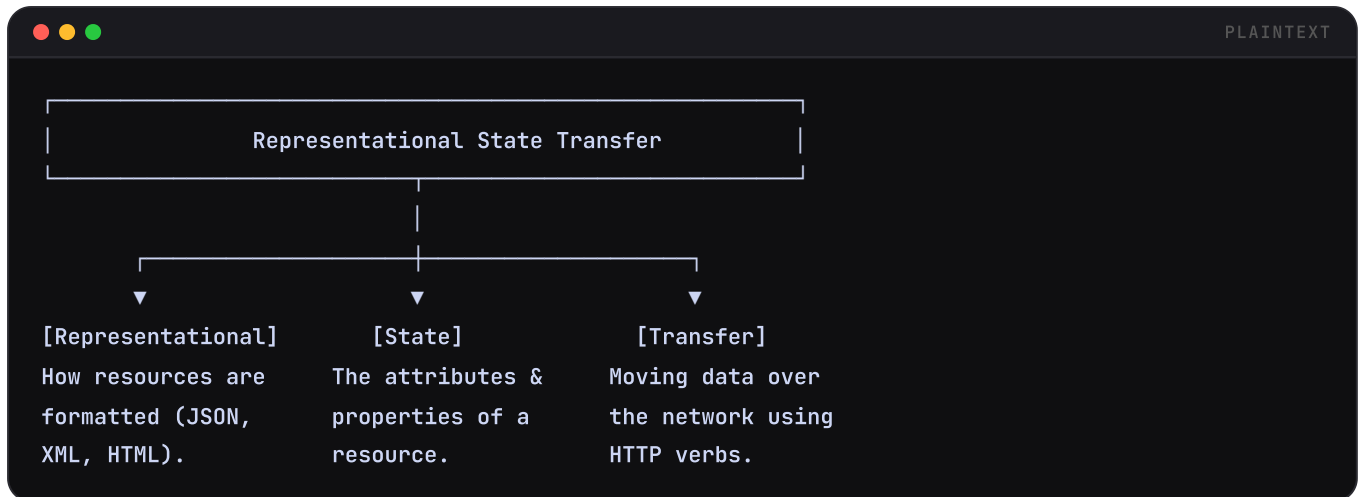
Servers can temporarily extend or customize client functionality by transferring executable code (such as compiled Java applets or client-side JavaScript). This is the only optional constraint in the REST specification.

6. Uniform Interface

The uniform interface is the defining characteristic of a RESTful system. It simplifies the system architecture by enforcing a single, standardized way for clients and servers to communicate. It contains four sub-constraints:

4. Deconstructing the Term: Representational State Transfer

To understand the mechanics of REST, we can break down its name:



1. Representational

Resources (data entities or objects) are abstract concepts stored in a database. To interact with them, they must be translated into a specific format. A single resource can have multiple representations based on client preferences (negotiated via the `Accept` header):

2. State

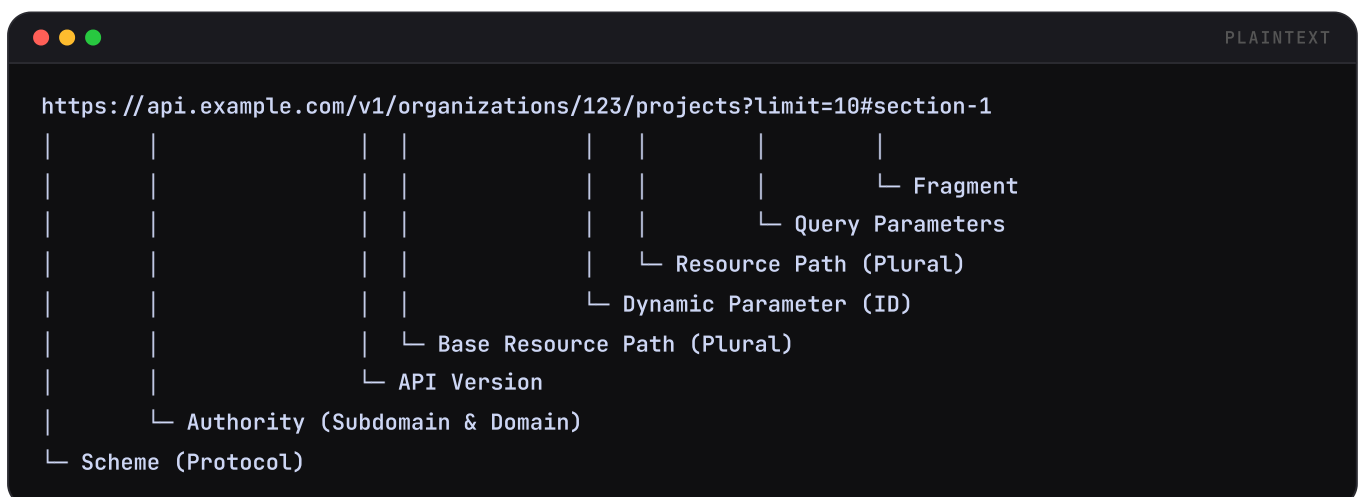
State refers to the current configuration, properties, and values of a resource at a given moment. For example, a shopping cart resource's state includes its unique identifier, the list of items, quantities, and the calculated total price.

3. Transfer

Transfer refers to the movement of these resource representations over the network. The client and server exchange these states using standardized HTTP methods (e.g., `GET`, `POST`, `PUT`, `PATCH`, `DELETE`).

5. Anatomy of a RESTful URL

A Uniform Resource Locator (URL) is a specific type of URI that identifies a resource and specifies the protocol used to access it.



API-Specific Path Design Rules

RULE 1: ALWAYS USE PLURAL NOUNS FOR RESOURCES

Path segments should represent collections of resources. Use plural nouns, never singular nouns.

RULE 2: EXPRESS HIERARCHY WITH FORWARD SLASHES

The forward slash (`/`) indicates a hierarchical relationship between resources.

This URL points to the collection of projects nested under organization number `5`.

RULE 3: CASE AND CHARACTER CONVENTIONS

6. HTTP Methods, Safety, and Idempotency

Understanding the semantic meaning, safety, and idempotency of HTTP methods is critical for correct API behavior.

HTTP METHOD	CRUD EQUIVALENCE	SAFE?	IDEMPOTENT?	TYPICAL SUCCESS STATUS CODE
GET	Read	Yes	Yes	200 OK
POST	Create / Custom Action	No	No	201 Created / 200 OK
PUT	Update (Replace)	No	Yes	200 OK
PATCH	Update (Modify)	No	Yes (Generally)	200 OK
DELETE	Delete	No	Yes	204 No Content

Defining Safety and Idempotency

PLAINTEXT

```
Idempotent Operation: f(x) = f(f(x)) = f(f(f(x)))
```

WHY IS `DELETE` IDEMPOTENT?

The first `DELETE /v1/projects/9` request deletes the project and returns a success status (such as `204 No Content`). If the client sends the same request again, the server will return a `404 Not Found` because the resource no longer exists.

However, **the state of the server has not changed further**. The project remains deleted. Because the side-effect on the server is identical, `DELETE` is idempotent.

WHY IS `POST` NON-IDEMPOTENT?

If you send a `POST /v1/projects` request with a payload to create a project, the server creates a new row in the database. If you send the exact same request ten times, the server will create ten distinct project entries with different IDs. Because the side-effects multiply with each call, `POST` is non-idempotent.

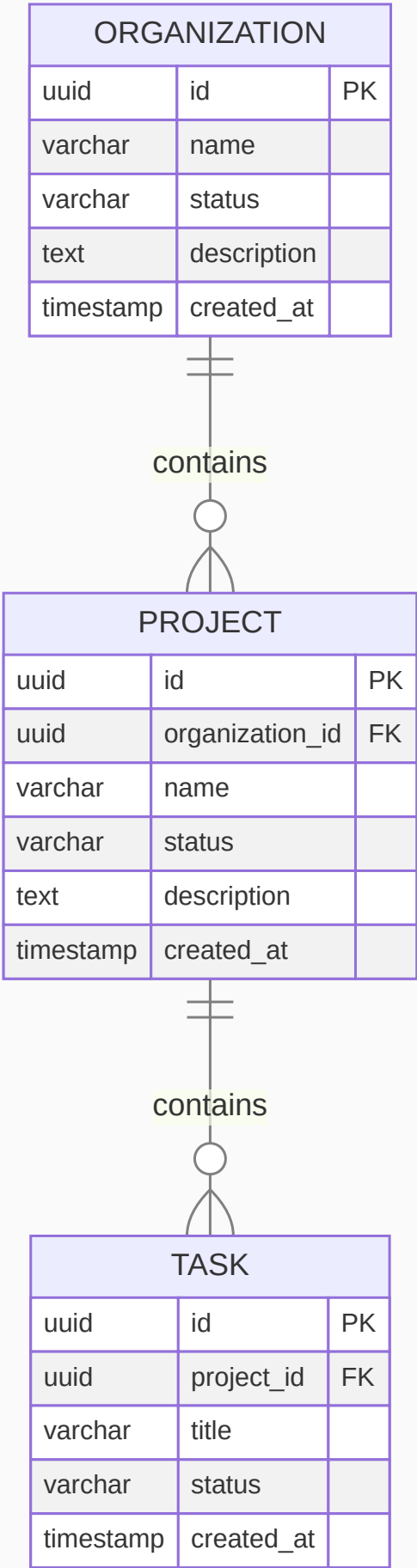
`PUT` VS. `PATCH`

7. API Interface Design: Case Study

To see these principles in action, let's design the API interface for a project management platform like Jira or Linear.

The Domain Schema

Before coding, we must identify our entities (nouns) and their database relationships:



Phase 1: Designing Organization CRUD Endpoints

1. CREATE ORGANIZATION

```
{
  "name": "Acme Corp",
  "status": "active",
  "description": "Enterprise solutions provider"
}

{
  "id": "e98b7a42-706f-427c-8647-73d82a122e2b",
  "name": "Acme Corp",
  "status": "active",
  "description": "Enterprise solutions provider",
  "createdAt": "2023-10-27T10:14:00Z",
  "updatedAt": "2023-10-27T10:14:00Z"
}
```

2. GET SINGLE ORGANIZATION

```
{
  "id": "e98b7a42-706f-427c-8647-73d82a122e2b",
  "name": "Acme Corp",
  "status": "active",
  "description": "Enterprise solutions provider",
  "createdAt": "2023-10-27T10:14:00Z",
  "updatedAt": "2023-10-27T10:14:00Z"
}
```

3. LIST ORGANIZATIONS (WITH PAGINATION, SORTING, AND FILTERING)

When designing endpoints that return lists, returning all database records at once is a major anti-pattern. Large datasets consume excessive server memory, increase JSON serialization latency, and overwhelm client applications. Instead, we use **pagination**, **sorting**, and **filtering**.

```
{
  "data": [
    {
      "id": "e98b7a42-706f-427c-8647-73d82a122e2b",
      "name": "Acme Corp",
      "status": "active",
      "description": "Enterprise solutions provider",
      "createdAt": "2023-10-27T10:14:00Z"
    },
    {
      "id": "b12c8a93-807f-537d-9748-84e93b233f3c",
      "name": "Stark Industries",
      "status": "active",
      "description": "Defense and energy solutions",
      "createdAt": "2023-10-25T08:12:00Z"
    }
  ],
  "total": 5,
  "page": 1,
  "totalPages": 3
}
```

Sane Defaults for List APIs

To ensure a smooth developer experience, the server should apply default values if the client does not provide query parameters:

Empty List Responses

If a filter is applied and no records match (e.g., `?status=archived` when there are no archived organizations), the server **must not** return a `404 Not Found` status. A 404 indicates that the resource path itself is invalid. Instead, return

a **200 OK** status with an empty data array:

```
{
  "data": [],
  "total": 0,
  "page": 1,
  "totalPages": 0
}
```

4. UPDATE ORGANIZATION (PARTIAL UPDATE)

```
{
  "status": "archived"
}
```

```
{
  "id": "e98b7a42-706f-427c-8647-73d82a122e2b",
  "name": "Acme Corp",
  "status": "archived",
  "description": "Enterprise solutions provider",
  "createdAt": "2023-10-27T10:14:00Z",
  "updatedAt": "2023-10-27T11:00:00Z"
}
```

5. DELETE ORGANIZATION

Phase 2: Custom Actions (Non-CRUD Operations)

Real-world applications often require actions that do not fit neatly into standard CRUD operations. For example, archiving an organization is more than just a status update; it may require cascading database updates, disabling user access, and sending notification emails.

To handle these scenarios, REST design uses **Custom Actions**.

```
Standard CRUD Pattern:  PATCH  /v1/organizations/:id
Custom Action Pattern:  POST   /v1/organizations/:id/archive
                        └─ Action Verb
```

HOW TO DESIGN CUSTOM ACTIONS

EXAMPLE: ARCHIVE ORGANIZATION

```
{
  "id": "e98b7a42-706f-427c-8647-73d82a122e2b",
  "status": "archived",
  "archivedAt": "2023-10-27T11:05:00Z"
}
```

EXAMPLE: CLONE PROJECT

Cloning a project is another operation that cannot be modeled as a simple update. It involves reading an existing project, copying its configurations and tasks, generating new IDs, and saving the copies to the database.

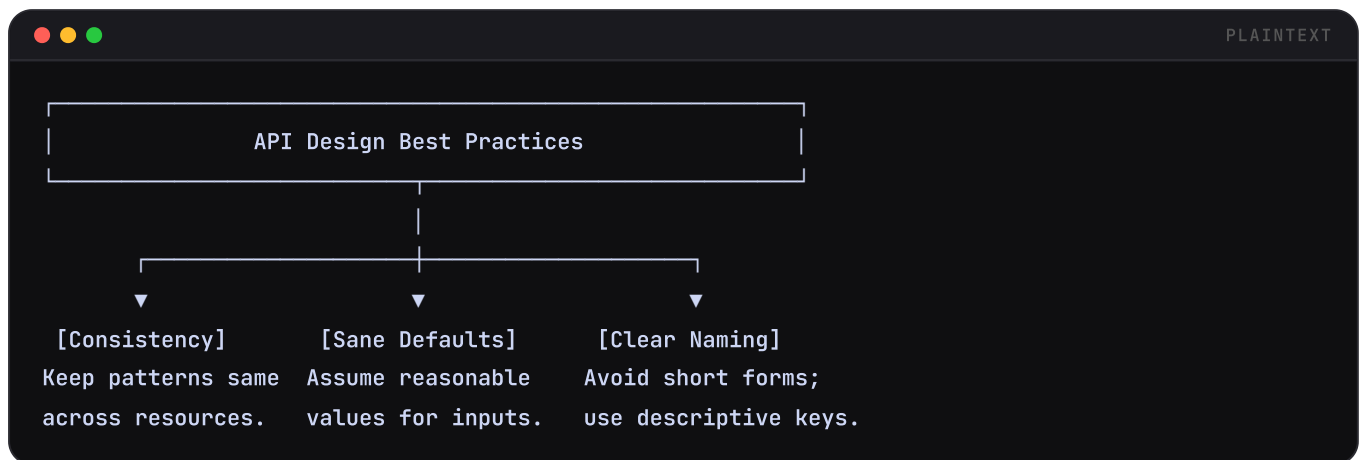
```
{
  "id": "a11c2b34-907f-637d-8748-94e93b233f4d",
  "name": "Project Alpha (Clone)",
  "status": "planned",
  "clonedFrom": "f38b7a42-806f-527c-9647-83d82a122e2c",
}
```



```
"createdAt": "2023-10-27T11:10:00Z"  
}
```

8. Principles of High-Quality API Design

Building high-quality APIs requires attention to detail, consistency, and a focus on developer experience (DX).



1. Maintain Strict Consistency

Consistency is the most important element of a great API. If you establish a pattern for one resource, you must apply it to every other resource in your system.

2. Provide Sane Server-Side Defaults

Do not force clients to provide values for fields that have logical defaults.

3. Avoid Abbreviations

Abbreviations can lead to confusion and integration errors. Always write out full words.

4. Interactive Documentation

Always maintain an accurate **OpenAPI (Swagger)** specification for your APIs. Tools like Swagger UI or Redoc provide an interactive playground where frontend engineers and API consumers can test endpoints and inspect payloads without reading backend code.

9. Richardson Maturity Model: The Path to RESTful Perfection

To understand where your API design stands, you can measure it against the **Richardson Maturity Model**. This model breaks down the steps of a RESTful API into four distinct levels of maturity:

Lecture 12: 12. Mastering Databases with Postgres

LECTURE 12: MASTERING DATABASES WITH POSTGRES – COMPREHENSIVE STUDY NOTES

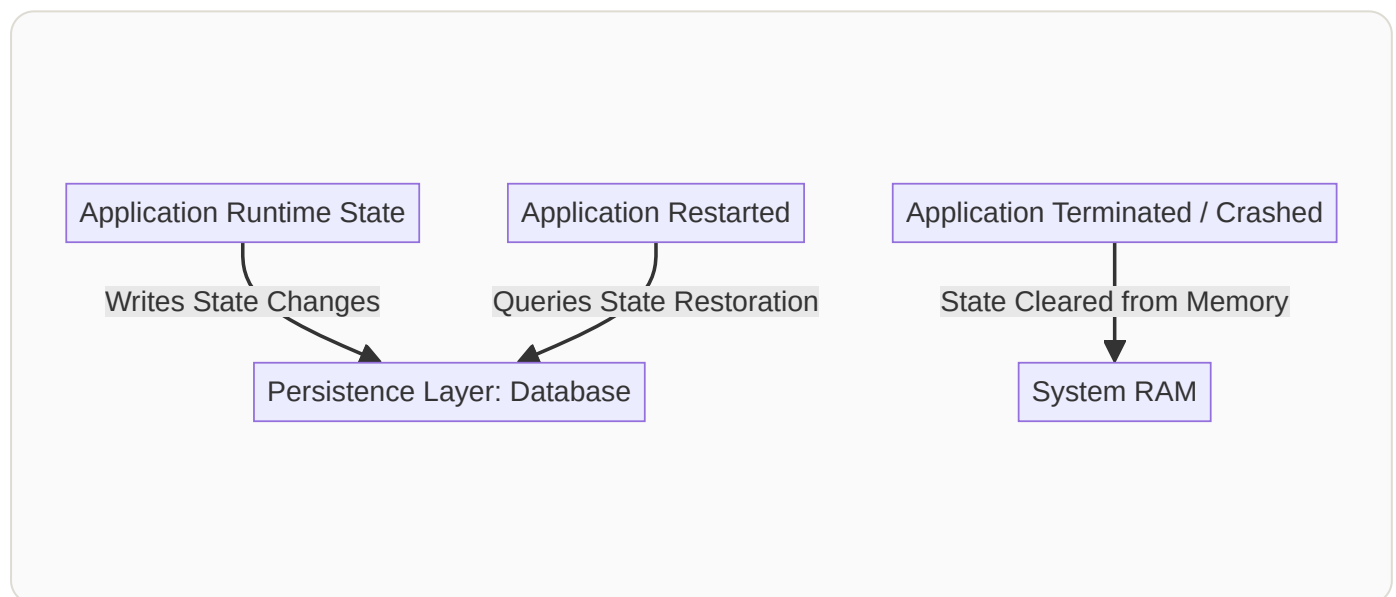
1. Foundations of Database Systems & Persistence

At its core, a database is a structured system designed to **persist** information across different runtime sessions. Understanding database systems requires a firm grasp of what persistence means, how memory hierarchies dictate database design, and why modern backend architectures decouple temporary state from permanent storage.

The Concept of Persistence

In software engineering, **persistence** refers to storing data in a non-volatile medium so that it survives even after the application process that created it terminates.

Consider a standard To-Do application:



Broad vs. Developer Context of "Database"

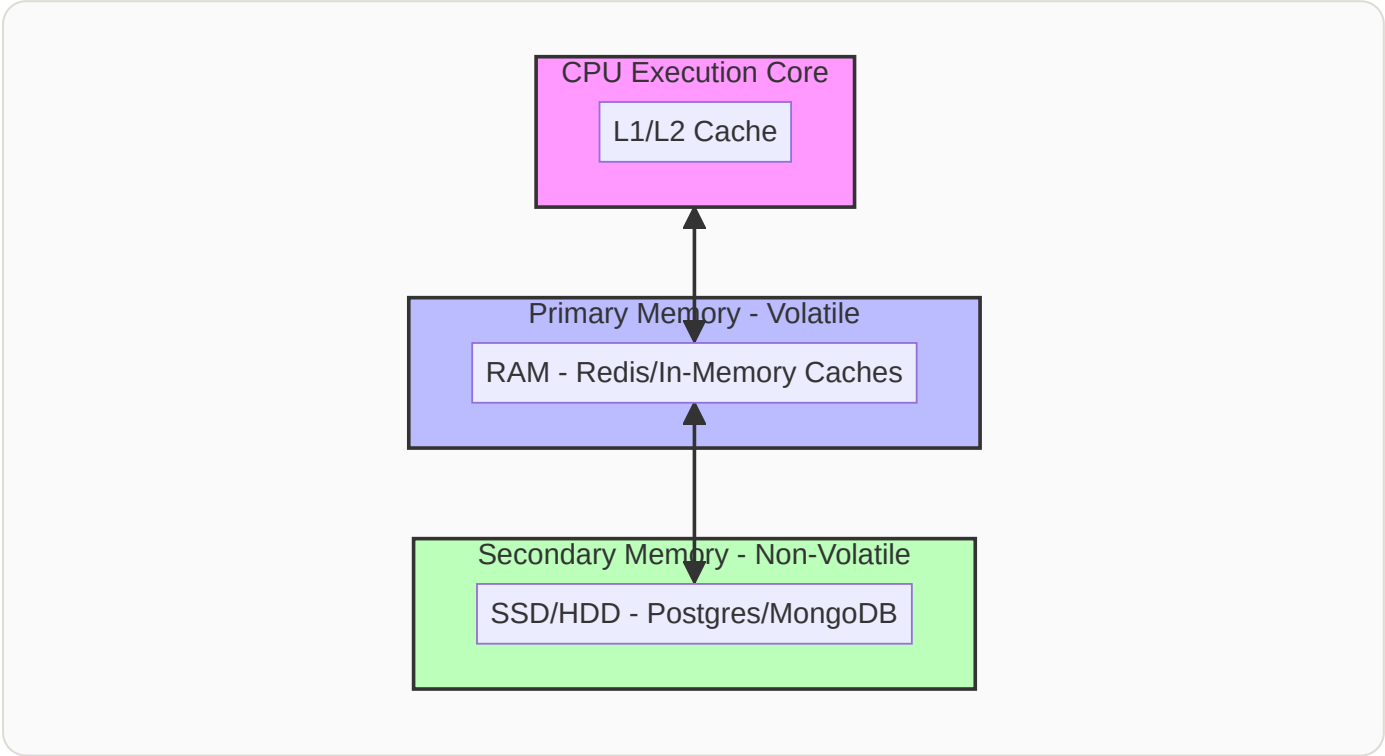
The term "database" is highly generic. In its broadest sense, any structured mechanism for storing data can be classified as a database:

However, in the context of **backend engineering and enterprise server architectures**, a "database" refers specifically to a **disk-based, highly concurrent Database Management System (DBMS)**.

Memory Hierarchy: Primary vs. Secondary Storage

To understand why backend systems rely on disk-based databases, we must analyze the hardware tradeoffs between Primary Memory (RAM) and Secondary Memory (Solid-State Drives/Hard Disk Drives).

Attribute	Primary Memory (RAM)	Secondary Memory (SSD / HDD)
Speed	Extremely fast (nanosecond latency).	Relatively slow (microsecond/millisecond latency).
Cost	High cost per gigabyte.	Low cost per gigabyte.
Capacity	Constrained (typically 8 GB – 128 GB in standard nodes).	Massive (typically 512 GB – 2 TB+ per drive).
Volatility	Volatile (data is lost upon power loss).	Non-volatile (data persists indefinitely without power).
Role	Used for execution state and caching (e.g., Redis).	Used for persistent databases (e.g., PostgreSQL).



Because of these trade-offs, caching systems like **Redis** operate within Primary Memory to provide sub-millisecond response times, while traditional databases like **PostgreSQL** or **MongoDB** write data to Secondary Memory to balance storage capacity, cost-efficiency, and permanent safety.

2. DBMS vs. Flat Files & Architectural Paradigms

Before modern Database Management Systems (DBMS) emerged, applications persisted data using flat text files. This approach exhibits fundamental flaws when scaled.

The Limitations of Flat-File Storage

PLAINTEXT

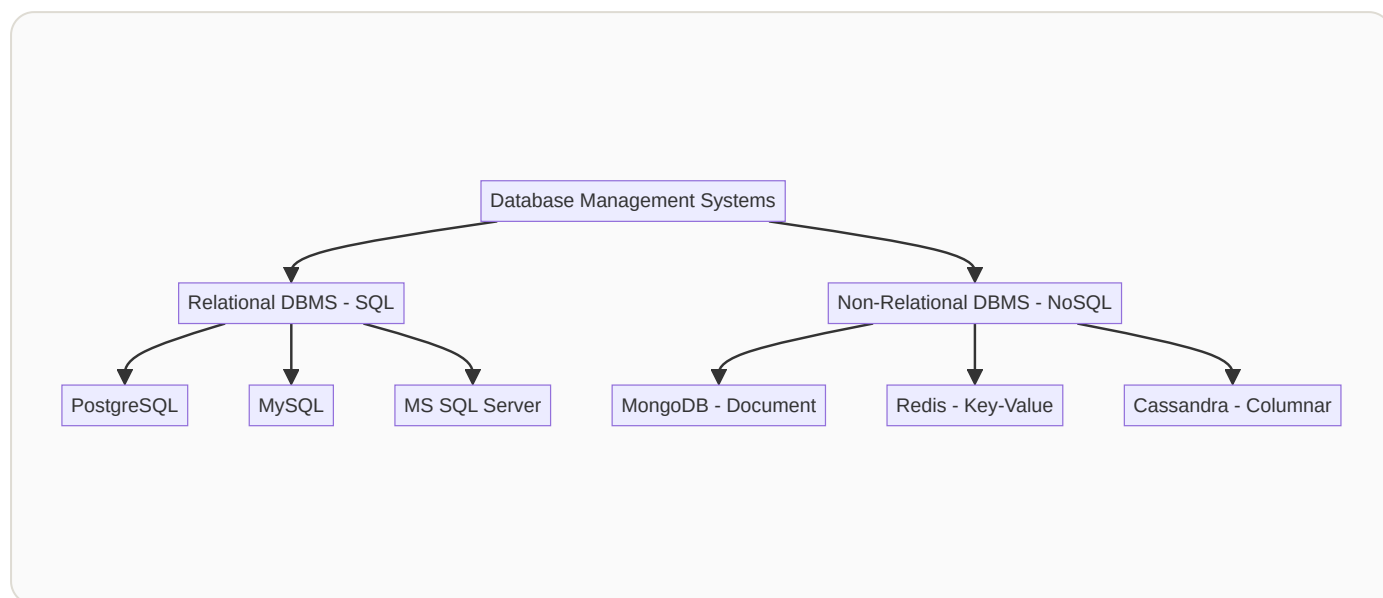
```
Thread A:  [Read: 40] -----> [Write: 60]
Thread B:  [Read: 40] -----> [Write: 20]
File State: (40) -----> (20) -----> (60)  ← State Inconsistency!
```

Core Responsibilities of a DBMS

A Database Management System (DBMS) is a specialized software system designed to eliminate these exact pain points. Its core responsibilities include:

3. Relational vs. Non-Relational (NoSQL) Paradigms

Modern database engines generally fall into one of two major design paradigms:



Relational Databases (RDBMS)

Relational databases organize data into rigid, tabular formats consisting of **tables, rows (tuples), and columns**.

Non-Relational Databases (NoSQL – Document Store Focus)

NoSQL databases, such as MongoDB, bypass tabular structures in favor of flexible formats like JSON-like documents.

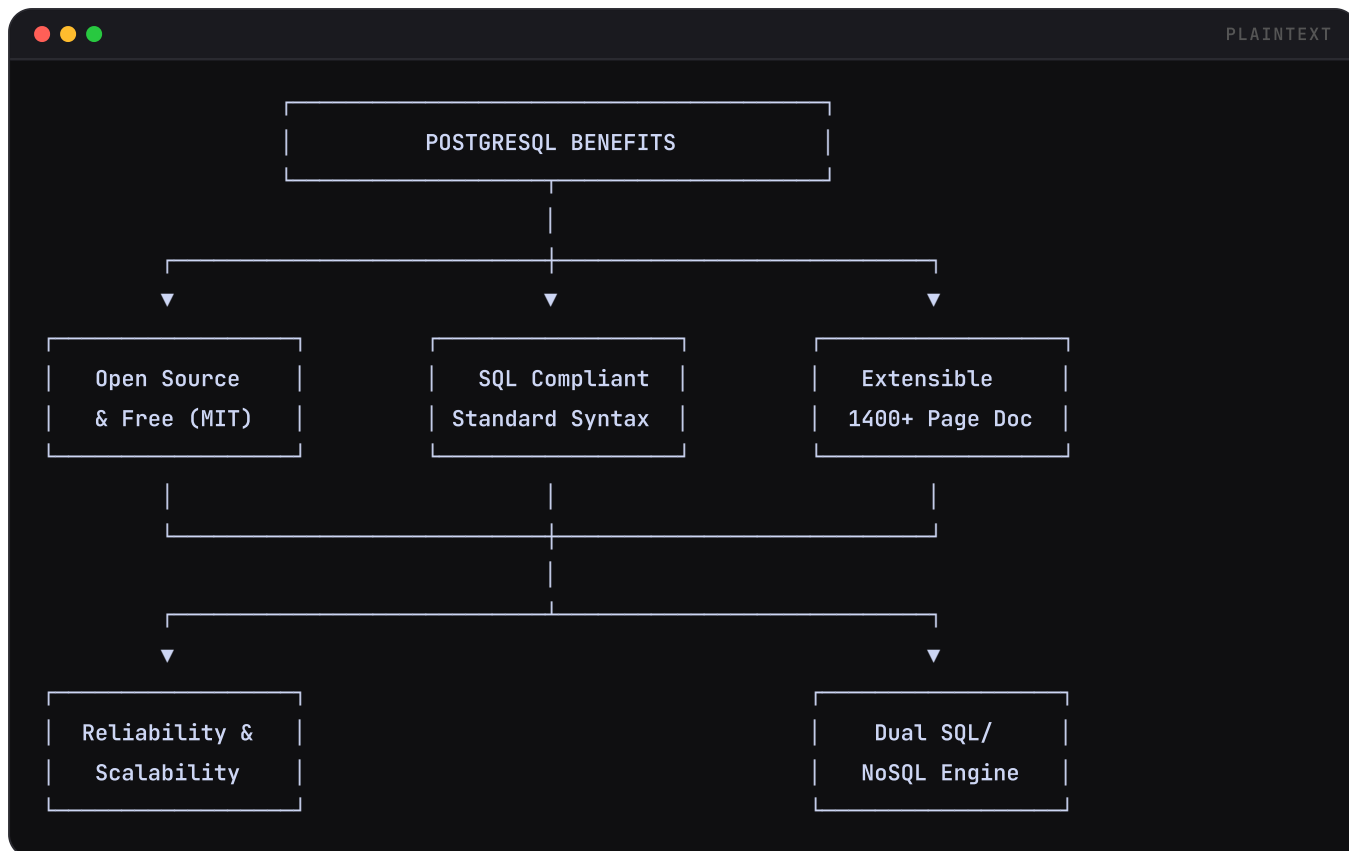
Comparative Architectural Fit

The NoSQL Trade-off: Schema Flexibility vs. Data Integrity

While NoSQL databases allow rapid prototyping due to their lack of schema constraints, they transfer the responsibility of data validation entirely to the application code. This increases application complexity and heightens the risk of data corruption when multiple services access the same collection using different versions of validation logic.

4. Why PostgreSQL?

When selecting a relational database for backend applications, PostgreSQL stands out as the industry standard for a wide range of use cases.



5. PostgreSQL Data Types: A Comprehensive Guide

PostgreSQL enforces strict typing. Choosing the correct type for each column is vital for performance, storage efficiency, and data integrity.

Numeric Types

String Types

Temporal Types

Advanced Types

6. Database Migrations & Version Control

In a professional development environment, you should never make manual structural changes to a production database using a graphical tool like TablePlus or PgAdmin. This approach lacks auditability, cannot be automated, and risks putting environments (local, staging, production) out of sync.

What are Database Migrations?

Database migrations are version-controlled, sequential files containing raw SQL statements that define structural changes over time. They act as "Git for your database schema."

A migration tool (such as `dbmate`, `golang-migrate`, or `Flyway`) reads these files in order, executing them on the target database and tracking which migrations have already been run.

PLAINTEXT

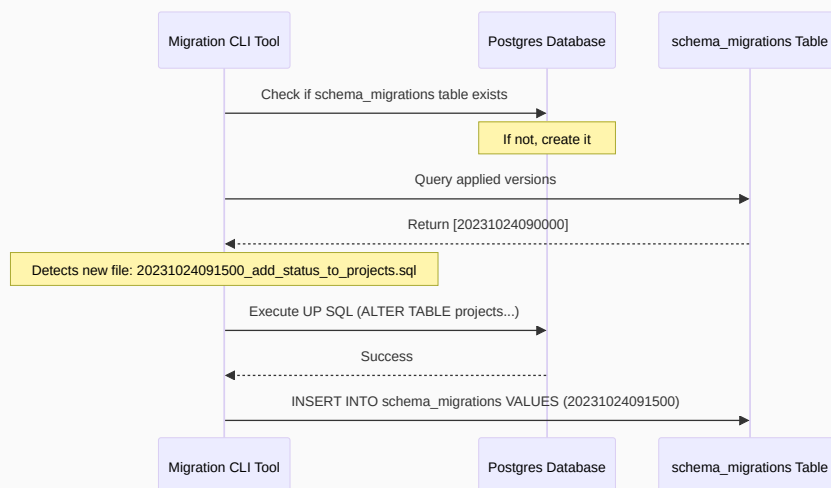
```
migrations_folder/
├── 20231024090000_create_users_table.sql
├── 20231024091500_add_status_to_projects.sql
└── 20231024093000_create_tasks_table.sql
```

Up vs. Down Migrations

Most migration workflows support two directions:

Schema Tracking Mechanics

To prevent re-running the same SQL files, migration tools maintain a dedicated table inside your database, typically named `schema_migrations`. This table contains a single column (`version`) that records the timestamps or sequence numbers of all successfully applied migrations.



7. Schema Design: Project Management Platform

To apply these concepts, we will design a relational schema for a project management system.



Syntax error in text
mermaid version 10.9.6

Step 1: Up Migration Schema Setup

Below is the complete, production-ready SQL migration script (`up` section) to build this schema. It includes custom domain enums, strict constraints, and referential integrity rules.

```
SQL

-- migrate:up

-- 1. Create Custom Enum Types
CREATE TYPE project_status AS ENUM ('active', 'completed', 'archived');
CREATE TYPE task_status AS ENUM ('pending', 'in_progress', 'completed', 'cancelled');
CREATE TYPE member_role AS ENUM ('owner', 'admin', 'member');

-- 2. Create Users Table
CREATE TABLE users (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  email TEXT NOT NULL CONSTRAINT unique_user_email UNIQUE,
  full_name TEXT NOT NULL,
  password_hash TEXT NOT NULL,
  created_at TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP,
  updated_at TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP
);

-- 3. Create User Profiles Table (One-to-One with Users)
CREATE TABLE user_profiles (
  user_id UUID PRIMARY KEY REFERENCES users(id) ON DELETE CASCADE,
  avatar_url TEXT,
  bio TEXT,
  phone TEXT,
  created_at TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP,
  updated_at TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP
);

-- 4. Create Projects Table (One-to-Many with Users as Owner)
CREATE TABLE projects (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  name TEXT NOT NULL,
  description TEXT,
  status project_status NOT NULL DEFAULT 'active',
  owner_id UUID NOT NULL REFERENCES users(id) ON DELETE RESTRICT,
  created_at TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP,
  updated_at TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP
);

-- 5. Create Tasks Table (One-to-Many with Projects & Many-to-One with Users as Assignee)
CREATE TABLE tasks (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  project_id UUID NOT NULL REFERENCES projects(id) ON DELETE CASCADE,
  title TEXT NOT NULL,
  description TEXT,
  priority INT NOT NULL DEFAULT 1 CONSTRAINT check_priority_range CHECK (priority BETWEEN 1 AND 5),
  status task_status NOT NULL DEFAULT 'pending',
  due_date DATE,
  assigned_to UUID REFERENCES users(id) ON DELETE SET NULL,
```



```

        created_at TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP,
        updated_at TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP
    );

-- 6. Create Project Members Table (Many-to-Many Link Table with Composite Primary Key)
CREATE TABLE project_members (
    project_id UUID REFERENCES projects(id) ON DELETE CASCADE,
    user_id UUID REFERENCES users(id) ON DELETE CASCADE,
    role member_role NOT NULL DEFAULT 'member',
    created_at TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP,
    PRIMARY KEY (project_id, user_id)
);

```

Architectural Analysis of Schema Design Choices

1. CASE SENSITIVITY & NAMING CONVENTIONS

PostgreSQL is **case-insensitive** by default. If you create a table using CamelCase (`CREATE TABLE "UserProfiles"`), Postgres converts it to lowercase (`userprofiles`) unless you wrap the name in double quotes every time you reference it.

2. ENUMS VS. PLAIN TEXT FOR STATUS FIELDS

Instead of storing state as plain text (`"active"` , `"completed"`), we created explicit `ENUM` types. This provides two key benefits:

3. ONE-TO-ONE RELATIONSHIP MECHANICS

In `user_profiles` , we omitted a separate `id` primary key column. Instead, the `user_id` foreign key serves as the table's **PRIMARY KEY** . This guarantees that a user can have at most *one* profile, enforcing a strict 1:1 relationship while saving index space.

4. MANY-TO-MANY LINK TABLES & COMPOSITE KEYS

The `project_members` table handles the many-to-many relationship between projects and users.

5. REFERENTIAL INTEGRITY & DELETION RULES

When a row referenced by a foreign key is deleted, we must define how Postgres handles the dependent rows:

Step 2: Down Migration Script

To support rollbacks, we must define the exact inverse of our schema setup in reverse order.

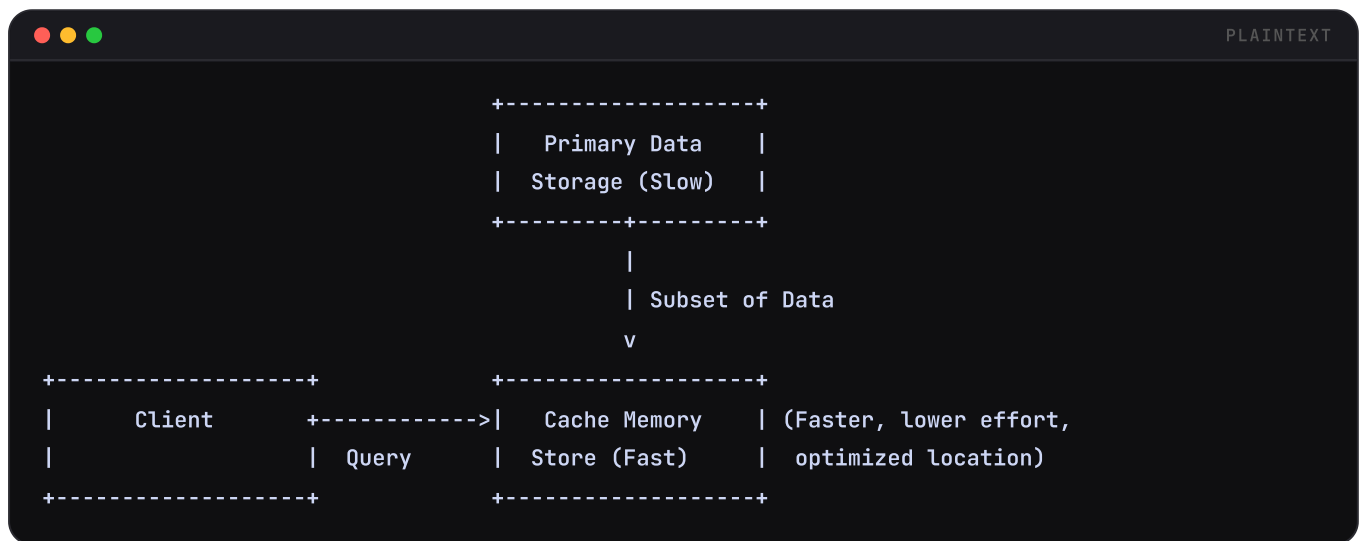
CH
13

Lecture 13: 13. Caching, the secret behind it all

LECTURE 13: CACHING, THE SECRET BEHIND IT ALL

1. Introduction to Caching

At its core, caching is a mechanism designed to **decrease the time and computational effort** required to perform a specific task.



From a technical perspective, **caching is the practice of storing a subset of primary data in a temporary, high-speed access location**. The determination of which subset to cache depends on several critical parameters:

In modern high-performance system designs, caching is a fundamental pillar. For applications targeting ultra-low latencies—measured in **single- or double-digit milliseconds (\$ms\$)** or even **microseconds (\$\mu s\$)**—proper cache implementation is mandatory.

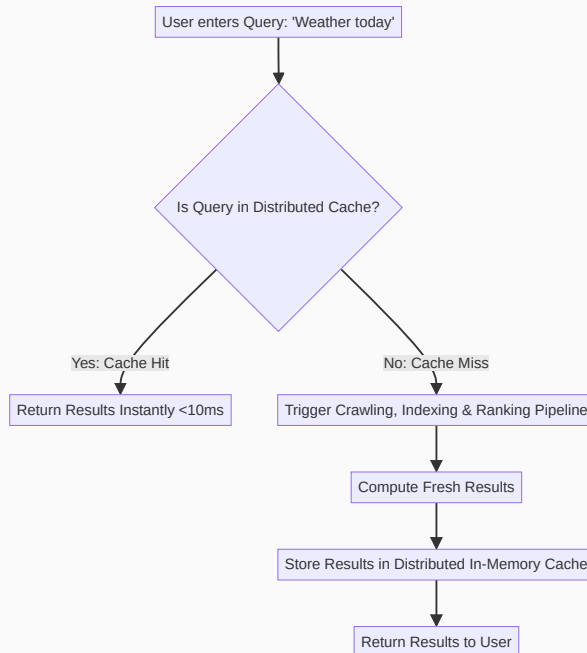
2. Architectural Case Studies

To understand the impact of caching on global-scale infrastructures, let us examine how industry leaders leverage this pattern to prevent system degradation and catastrophic resource exhaustion.

Case Study 1: Google Search

Every query entered into the Google search bar initiates a highly complex processing pipeline. This pipeline involves:

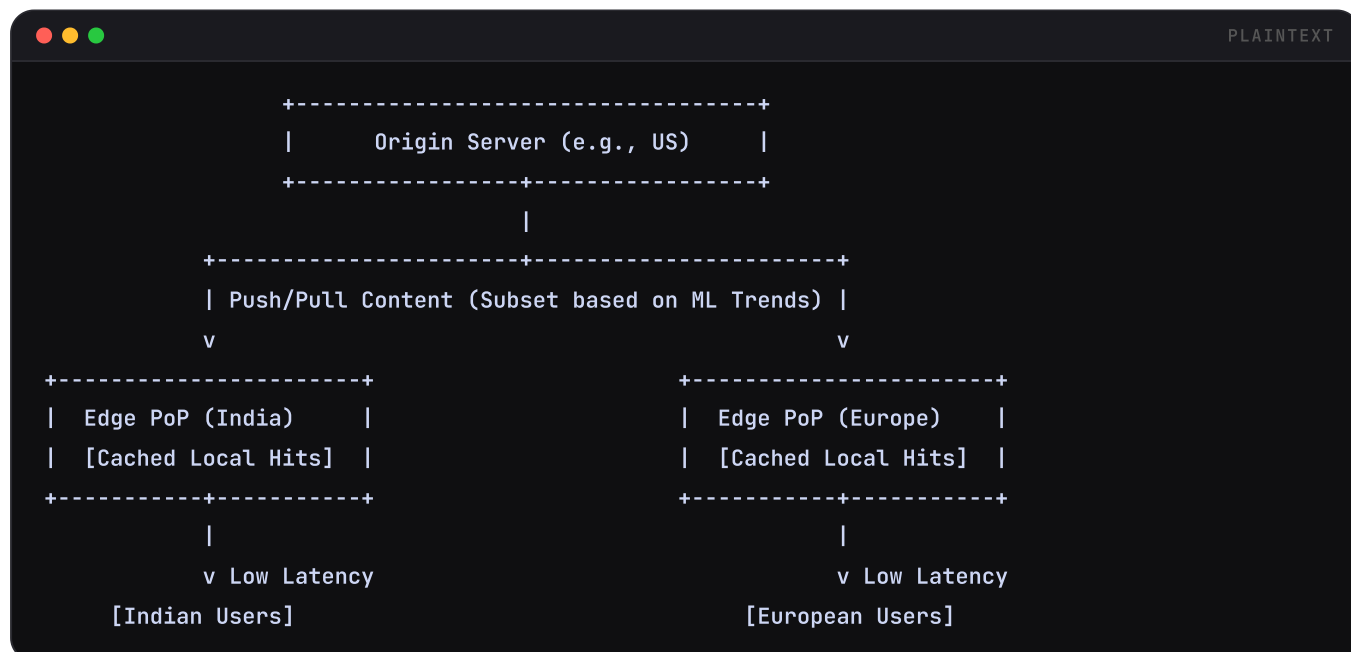
This pipeline is exceptionally **computationally expensive**, consuming significant CPU cycles and memory bandwidth. Without caching, popular queries like *"what is the weather today"* would trigger this resource-heavy ranking pipeline millions of times per day. The resulting server load would lead to cascading delays and high latency.



To resolve this, Google utilizes a **globally distributed, in-memory caching system**.

Case Study 2: Netflix (Global Content Delivery)

Netflix streams massive volumes of video data to millions of concurrent global users. A single movie file is not stored as a single static asset; it undergoes **encoding** to generate multiple resolutions (e.g., 1080p, 720p, 480p) tailored for various device types and fluctuating network bandwidths. This process produces terabytes of raw data per title. To deliver this content with minimal buffering and avoid crashing central data centers, Netflix uses a custom **Content Delivery Network (CDN)**.



Case Study 3: X (Formerly Twitter) Trending Topics

Calculating trending topics requires processing millions of incoming tweets per second to detect patterns, extract keywords, and apply natural language processing (NLP) models.

This computation is highly resource-intensive, requiring specialized GPU clusters and distributed streaming pipelines. If this calculation were executed every time a user refreshed their "Trending" tab, the infrastructure would experience immediate CPU starvation and crash.

Because trending topics do not change second-by-second—they typically evolve over hours—they are highly suitable for caching:

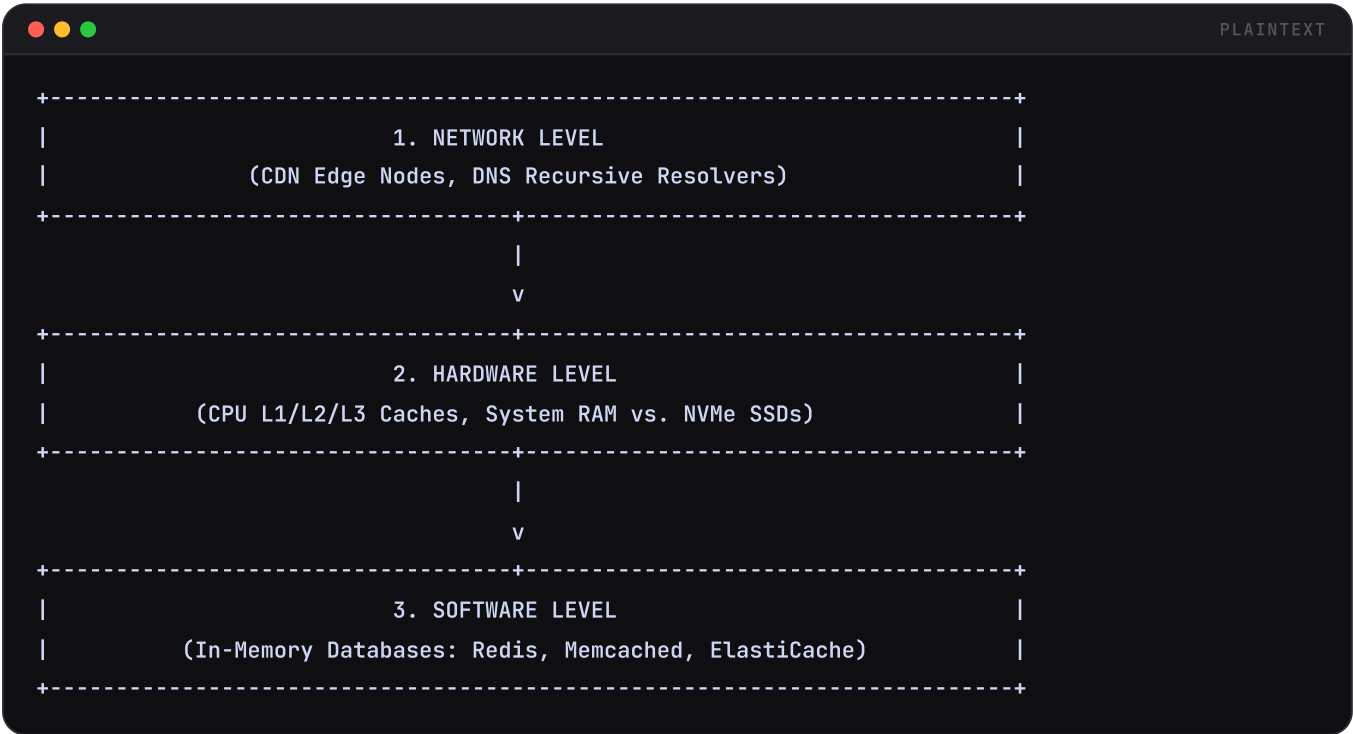
3. The Core Dual Mandate of Caching

Analyzing these case studies reveals a consistent pattern. Caching is applied to address two primary system bottlenecks:

BOTTLENECK CATEGORY	SYSTEM OBJECTIVE	REAL-WORLD EXAMPLE
Heavy Computation	Avoid re-executing complex, CPU/GPU-intensive algorithms for static or slowly changing results.	X Trending Topics, Google Search Ranking.
Heavy Data Transit	Avoid moving massive volumes of data over long physical distances or narrow network pipes.	Netflix Video Streaming, Static Asset Delivery (HTML/JS/CSS).

4. The Three Levels of Caching in Backend Architectures

As a backend engineer, your interaction with caching spans three distinct conceptual layers of system architecture:



5. Network-Level Caching

Network-level caching intercepts requests before they reach the application server, reducing both network transit times and backend system load.

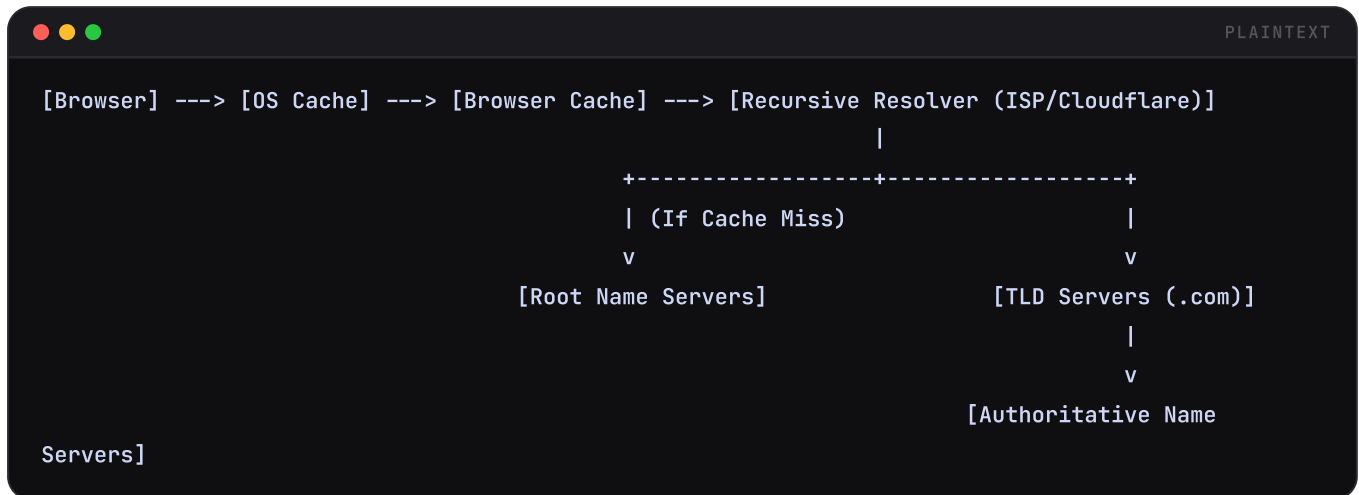
Content Delivery Networks (CDNs)

A CDN caches static and semi-dynamic assets (images, video segments, HTML, CSS, JavaScript files) across geographically distributed Edge servers.

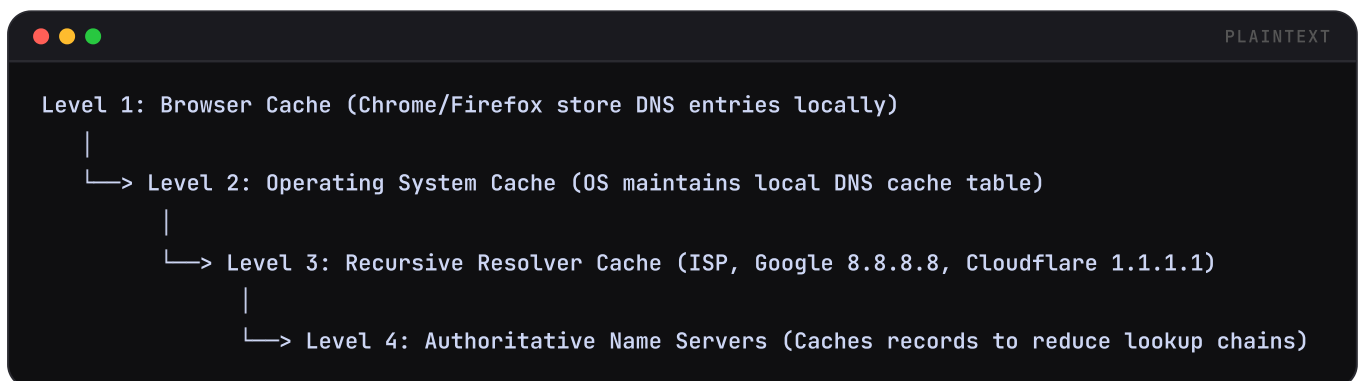
THE CDN REQUEST LIFECYCLE:

DNS Caching

The Domain Name System (DNS) maps human-readable domain names (e.g., `example.com`) to machine-readable IP addresses (e.g., `192.0.2.1`). Resolving a domain name through recursive queries is a multi-step process that would introduce latency if performed on every request.



To prevent network congestion, DNS resolution relies on aggressive multi-level caching:



If a cache miss occurs at the OS and Browser levels, the query is dispatched to the **Recursive Resolver**. If the resolver has the mapping cached, it returns it instantly. Otherwise, it must recursively query:

6. Hardware-Level Caching & RAM Mechanics

To understand why software-level caches are fast, we must look at how physical hardware manages memory.

CPU Cache Hierarchy and Locality

CPUs operate at speeds far exceeding those of system RAM. To prevent the CPU from idling during memory retrievals, hardware designers integrate small, ultra-fast caches directly onto the CPU die:



SPATIAL LOCALITY IN ACTION (ARRAYS VS. LINKED LISTS)

This hardware design explains why traversing sequential data structures, like arrays, is highly optimized. When a CPU reads an element of an array, its predictive prefetching algorithms assume the subsequent elements will be needed next. It loads a contiguous block of memory (a **cache line**, typically 64 bytes) directly into the L1/L2 cache. Consequently, subsequent array accesses result in immediate cache hits ($O(1)$ at the hardware layer). Conversely, traversing a linked list—where nodes are scattered non-contiguously throughout memory addresses—frequently results in hardware cache misses, forcing slower retrievals from system RAM.

Physical RAM vs. Secondary Storage (Solid-State Disks / HDDs)

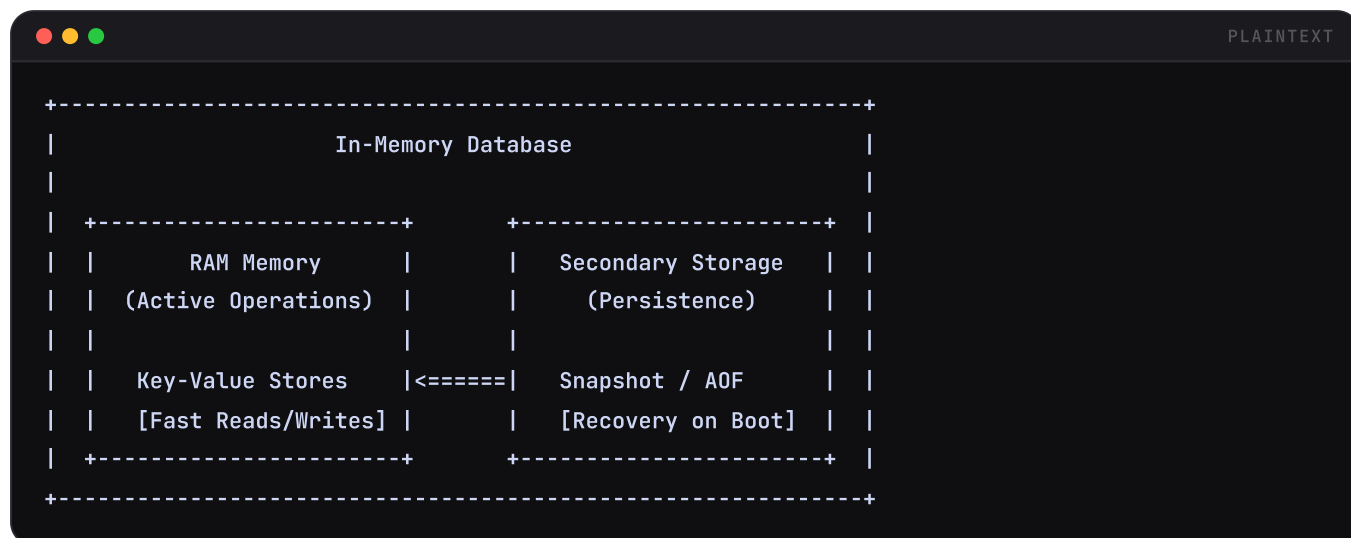
System RAM (Random Access Memory) and secondary storage (SSDs/HDDs) use fundamentally different physical designs, which shapes how they are used in software caching.

PLAINTEXT

System RAM (DRAM)	Solid-State Disk (SSD)
Speed: Extremely Fast (~ns)	Speed: Fast (~μs to ms)
Architecture: Capacitors	Architecture: NAND Flash
Volatility: Volatile	Volatility: Non-Volatile
Cost: Expensive per GB	Cost: Affordable per GB

7. Software-Level Caching (In-Memory Key-Value Stores)

In-memory databases like **Redis**, **Memcached**, and **AWS ElastiCache** leverage the speed of system RAM to provide high-performance caching at the application layer.



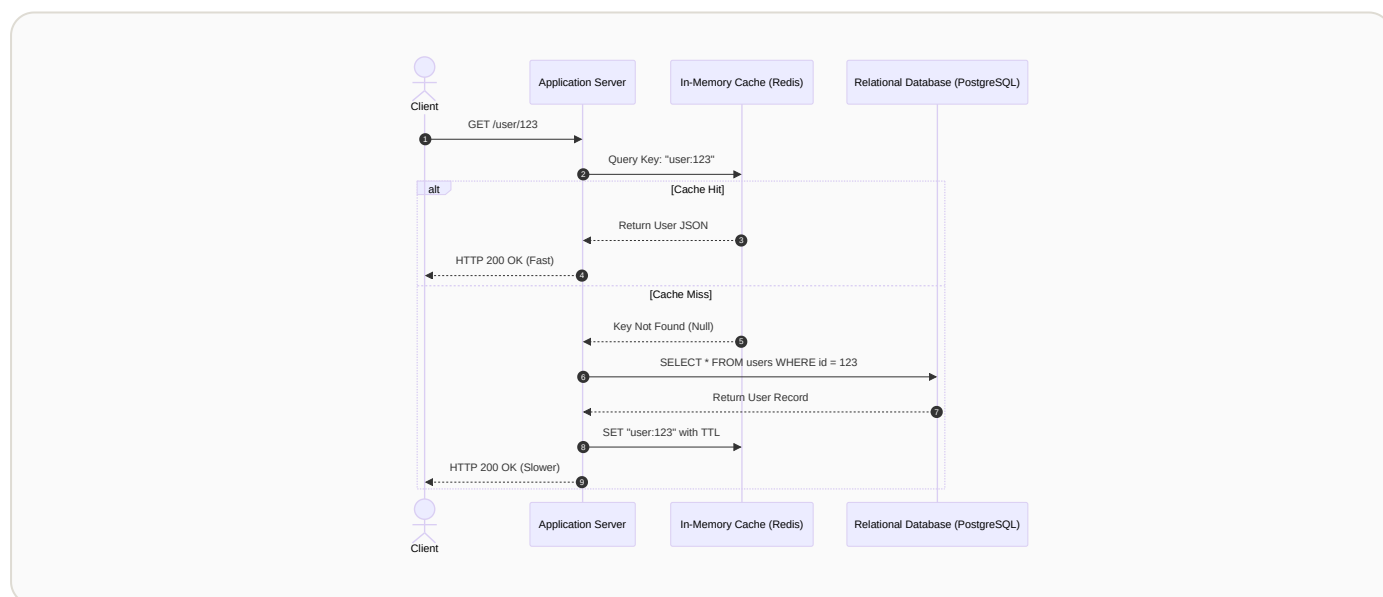
These technologies operate directly in RAM to achieve sub-millisecond read and write times. To address the volatility of RAM, databases like Redis implement background persistence mechanisms:

Caching Strategies

How an application coordinates data flow between the client, the cache, and the database depends on the chosen caching strategy.

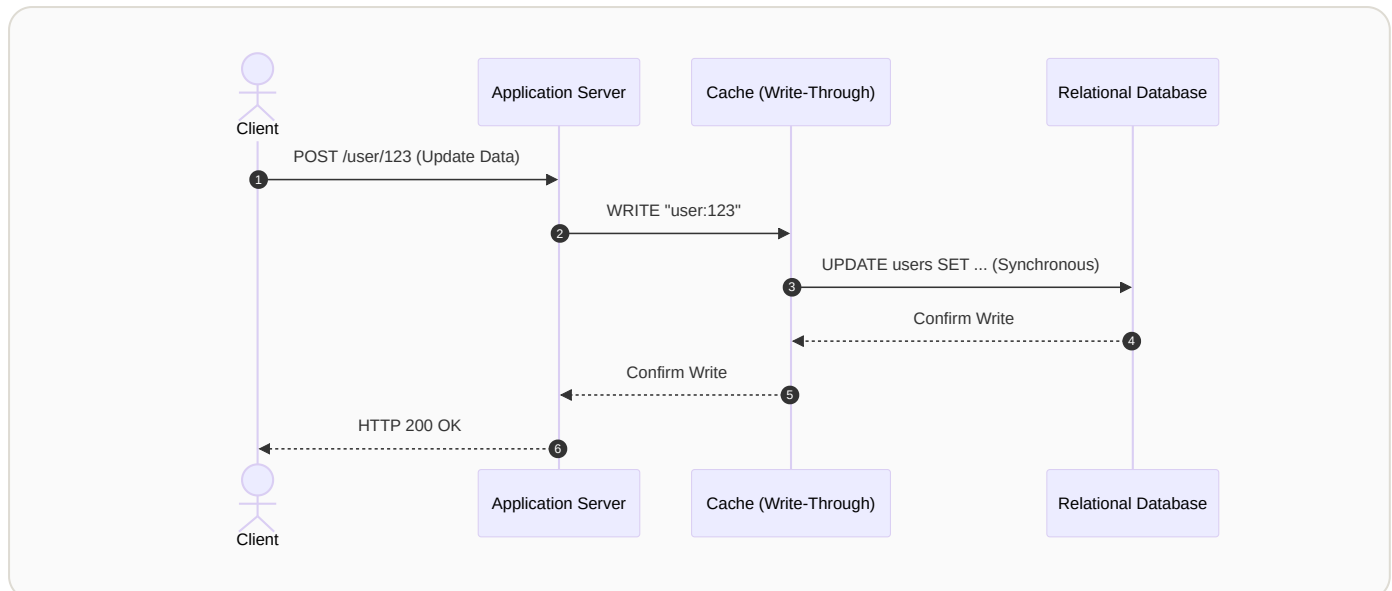
1. LAZY CACHING (CACHE-ASIDE)

In a Cache-Aside pattern, the application is responsible for managing both the cache and the database. The cache does not interact directly with the database.



2. WRITE-THROUGH CACHING

In a Write-Through pattern, the cache acts as the primary data interface. When the application writes data, it writes directly to the cache, which then synchronously updates the database.



Cache Eviction Policies

Because RAM is limited and expensive, caches eventually run out of capacity. When a cache reaches its memory limit, it uses an **eviction policy** to remove existing keys and make room for new data.

1. NO EVICTION (`NOEVICTION`)

2. LEAST RECENTLY USED (LRU)

3. LEAST FREQUENTLY USED (LFU)

4. TIME-TO-LIVE (TTL) EVICTION

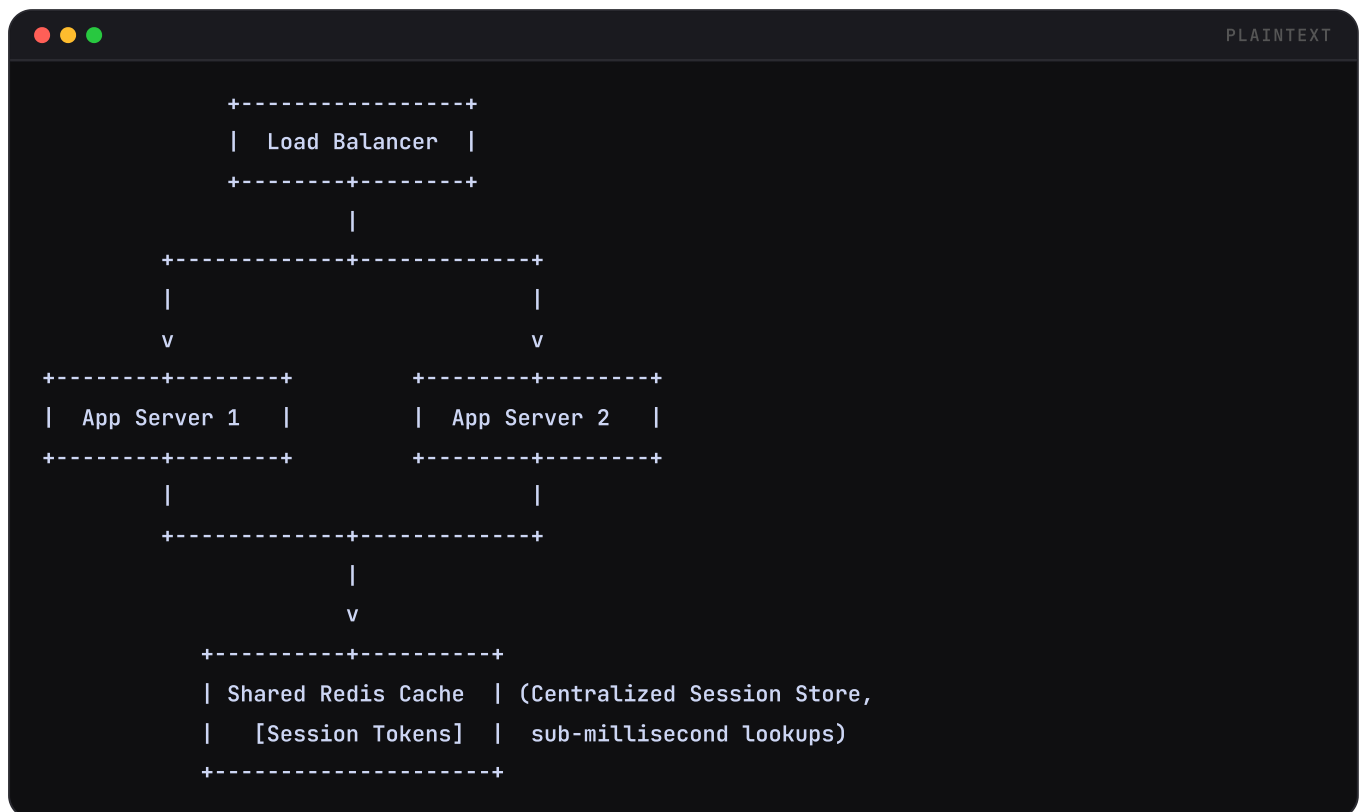
8. Real-World Backend Caching Patterns

Database Query Caching

Complex SQL queries—such as those involving multiple table joins, subqueries, and aggregations—can place significant load on relational databases. Caching the results of these queries is a common optimization.

Session Storage

In modern, horizontally scaled backend architectures, user sessions should not be stored in the local memory of a single application server. If a load balancer routes a subsequent request to a different server instance, the user's session state would be lost.



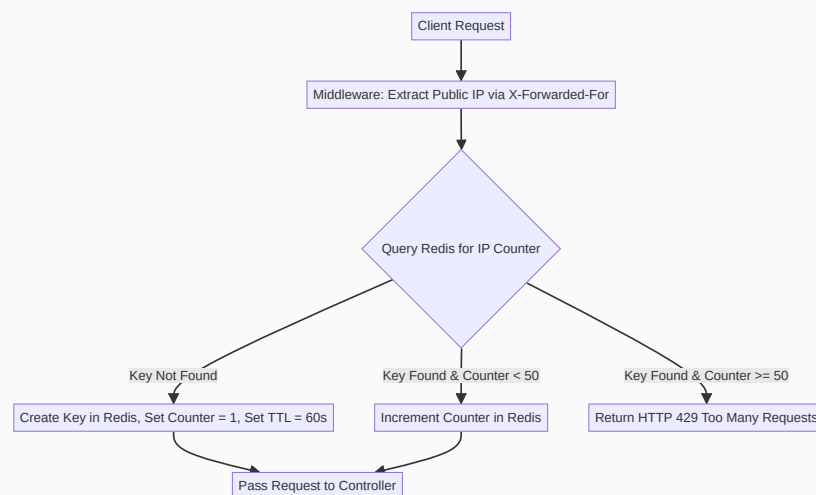
External API Caching

Integrating third-party services (such as weather APIs or financial data feeds) often introduces latency and cost considerations:

If the external data does not require real-time accuracy (e.g., weather forecasts), the backend can cache the API response for a set duration (e.g., 1 hour). Subsequent requests are served from the cache, protecting the application from rate limits and reducing external API costs.

Rate Limiting Middleware

Rate limiting protects APIs from abuse, denial-of-service (DoS) attacks, and brute-force attempts by limiting the number of requests a client can make within a given timeframe.



DETAILED IMPLEMENTATION MECHANICS: _____

Lecture 14: Task queues and background jobs

STUDY NOTES – LECTURE 14: TASK QUEUES AND BACKGROUND JOBS

1. Introduction to Background Tasks

In modern backend engineering, building highly scalable and responsive systems requires a deep understanding of **background tasks** (or background jobs).

What is a Background Task?

A background task is any piece of execution logic that runs **outside the synchronous request-response lifecycle** of a client-server interaction.

Synchronous Request-Response Lifecycle:

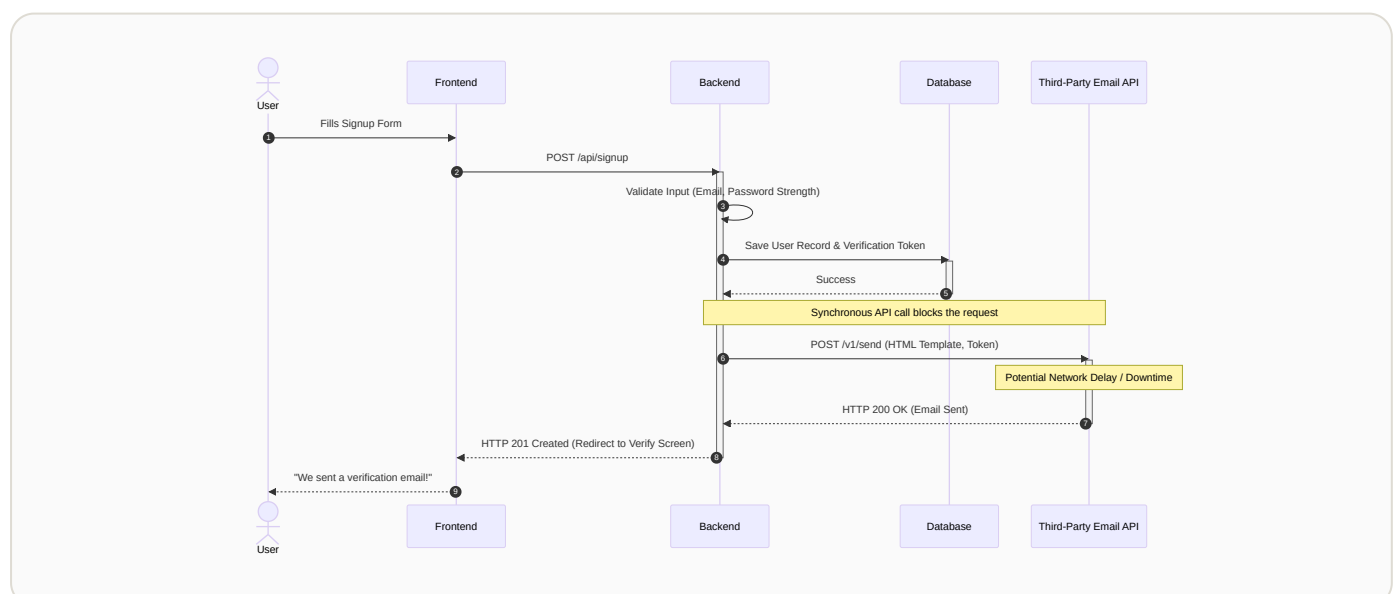
```
[ Client ] -----( Request )-----> [ Server ]  
[ Client ] <----- ( Response )----- [ Server ] (Blocks until complete)
```

In a traditional synchronous model, the client initiates a request, and the server blocks further execution until it processes the entire business logic and returns a response. If a request triggers heavy computation or external API calls, the connection remains open, consuming server resources and degrading the user experience.

By offloading these non-mission-critical workflows to a separate background process, the backend can immediately acknowledge the client's request, leaving the actual heavy lifting to be completed asynchronously.

2. The Anatomy of a Problem: Synchronous Pitfalls

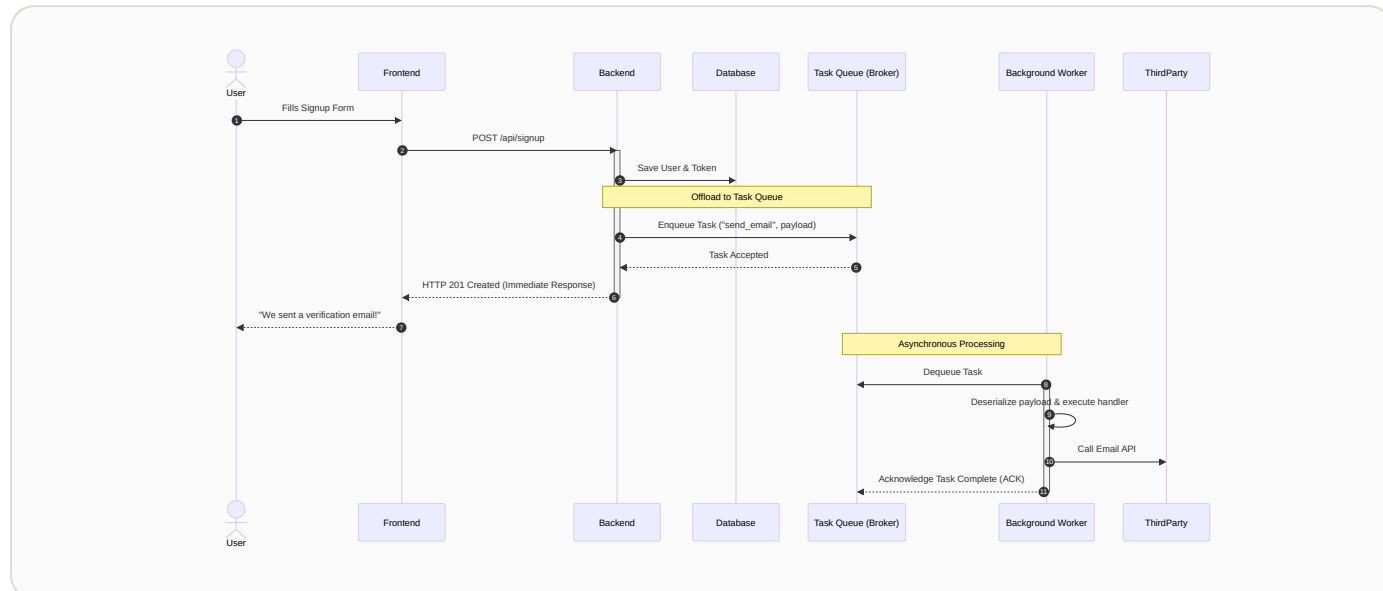
To understand why background execution is critical, consider a standard user signup workflow in a Software-as-a-Service (SaaS) application.



Failure Modes of the Synchronous Pattern

3. The Solution: Asynchronous Task Queues

Instead of executing the email-sending logic synchronously, the backend packages all necessary data, pushes it to an intermediate buffer (the queue), and immediately returns an HTTP success status to the client.



By decoupling task generation from task execution, the system achieves **high responsiveness** and **fault tolerance**.

4. System Architecture: The Producer-Broker-Consumer Model

A robust task queue architecture consists of three primary components: the **Producer**, the **Broker**, and the **Consumer (Worker)**.



Syntax error in text
mermaid version 10.9.6

1. The Producer

The producer is your main application backend (written in Node.js, Python, Go, etc.).

```
{
  "task_type": "send_verification_email",
  "payload": {
    "user_id": "usr_908123",
    "email": "user@example.com",
    "first_name": "Jane",
    "verification_code": "882194"
  }
}
```

2. The Broker (Task Queue)

The broker is the storage engine that acts as a temporary holding area for tasks until a worker is ready to process them.

3. The Consumer (Background Worker)

The consumer is an independent, long-running process (often running on separate physical/virtual nodes) that continually monitors the broker.

5. Core Queueing Mechanisms

Task Acknowledgements (ACKs)

To prevent data loss, brokers do not immediately delete a task when it is dequeued. Instead, the worker must explicitly send an **Acknowledgement (ACK)** signal back to the broker once the task has been successfully executed.

Visibility Timeout

The **visibility timeout** is the period of time during which the broker hides a dequeued task from other active workers.

Exponential Backoff and Retries

When an external dependency fails, retrying the task immediately can worsen the target system's load (a self-inflicted DDoS attack). Modern task runners utilize **exponential backoff** algorithms to space out subsequent retry attempts.

$$\text{Retry Delay} = \text{Base Delay} \times 2^{\text{attempt}}$$

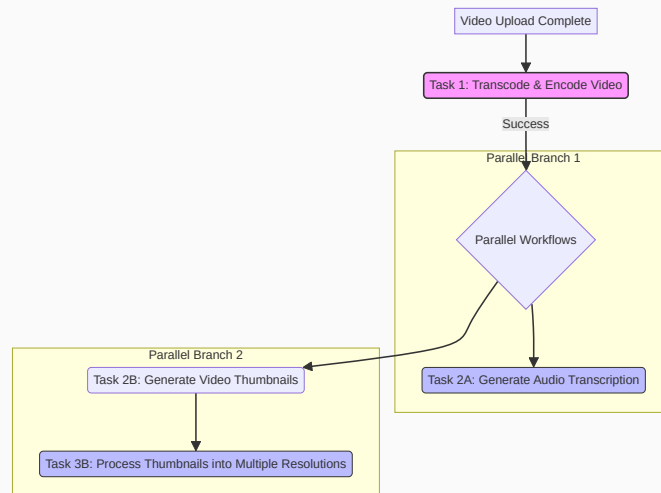
6. Types of Background Tasks

Modern SaaS architectures encounter several categories of background tasks:

Task Type	Description	Real-World SaaS Example
One-Off (Triggered) Tasks	Executed once in response to a specific application event.	Sending a verification email immediately after user registration; firing a single transactional push notification.
Recurring (Scheduled) Tasks	Executed periodically at predefined intervals (similar to OS-level Cron jobs).	Running database maintenance to delete expired user sessions; generating and emailing monthly billing statements.
Chained (Workflow) Tasks	Multi-step tasks with parent-child dependencies where downstream tasks require upstream success.	An LMS video upload pipeline (see workflow below).
Batch Tasks	Splitting a single high-volume operational request into thousands of independent, parallelized sub-tasks.	Deleting a enterprise account (which spawns thousands of sub-tasks to delete sharded database records, media assets, and logs across regions).

Chained Task Workflow: LMS Video Upload

In a Learning Management System (like Udemy), uploading a video requires a multi-stage, dependent workflow.



7. Key Design Considerations at Scale

When scaling a backend system to support millions of background tasks, engineers must design around several critical factors:

1. Idempotency

An operation is **idempotent** if performing it multiple times yields the exact same system state as performing it once.

$$f(f(x)) = f(x)$$

Because network partitions can cause duplicate task deliveries (e.g., a worker completes a task but crashes right before sending the ACK), workers must be resilient to executing the same task twice.

2. Robust Error Handling & Logging

Since background processes run asynchronously, unhandled exceptions will not bubble up to a client's browser.

3. Monitoring & Metrics Instrumentation

You cannot manage what you do not measure. A production-grade queueing system must expose metrics to monitoring systems like **Prometheus**, which are then visualized in tools like **Grafana**.

Key Metrics to Instrument:

1. Queue Depth (Number of pending tasks) → Critical for auto-scaling workers.
2. Processing Latency (Time elapsed from dequeue to ACK).
3. Error Rate (Percentage of tasks moving to the Dead Letter Queue).
4. Worker CPU & Memory Utilization.

4. Horizontal Scaling

If your queue depth (the number of pending tasks) begins to accumulate, the system should automatically spin up additional worker nodes. Design your workers to be stateless so that they can be scaled out horizontally across multiple servers or container environments (e.g., Kubernetes pods).

5. Rate Limiting External Services

Many background tasks involve interacting with external APIs (e.g., sending push notifications via Apple Push Notification service (APNs) or Google's Firebase Cloud Messaging (FCM)). Since these services enforce strict rate limits, your workers must be configured to throttle their output rate to avoid getting blocked or incurring excessive overage fees.

8. Best Practices for Backend Engineers

To ensure your background task architecture remains stable and maintainable, adhere to these four core development guidelines:

PLAINTEXT

BACKGROUND JOB BEST PRACTICES	
1. Keep Tasks Small	2. Avoid Long-Running Jobs
- One single processing unit per task.	- Break heavy tasks into smaller chunks.
3. Implement Robust Logs	4. Track Queue Depth
- Log task state changes and error traces.	- Set up alerts for unusual queue growth.

